



Saint-Mars-du-Désert

RAPPORT TECHNIQUE FIN DE PROJET

Projet transversal n°4

Collecteur de données animalières

Équipe projet :

Nolan Buchet, Davy Clark, Ewen Le Callet, Joris Menard, Jules Numba, Xueying Xu

Client :

Xavier Leprevost – Élu de la mairie de Saint-Mars-du-Désert

Encadrants :

Abdelhakim Saadane et Antoine Goullet

Date :

09/04/2026

Année universitaire 2025–2026

Remerciements

Nous tenons à remercier l'ensemble des personnes ayant contribué à la réalisation de ce projet transversal. Nous remercions particulièrement nos encadrants, Abdelhakim Saadane et Antoine Goulet, pour leur accompagnement, leurs conseils méthodologiques et leur disponibilité tout au long de l'année universitaire 2025–2026.

Nous remercions également notre client, Xavier Leprevost, élu de la mairie de Saint-Mars-du-Désert, pour sa confiance, la clarté de ses besoins et la qualité des échanges durant le développement du *collecteur de données animalières*.

Nous adressons aussi nos sincères remerciements à l'ensemble des experts que nous avons eu l'opportunité de contacter, pour leur disponibilité, leurs retours d'expérience et leurs précieux conseils qui ont contribué à orienter nos choix. Nous pensons notamment à Bruno Buffon GBODJINO (MSc in Remote Sensing and GIS, CURAT, Côte d'Ivoire), Luke Ditria (PhD en Machine Learning, ingénieur et développeur de dispositifs embarqués pour le suivi écologique par IA), Laurent Godet (Chargé de recherche CNRS, spécialiste en biogéographie littorale et conservation de la nature), ainsi que Julien Donzé (vidéaste animalier).

Table des matières

Remerciements

Table des figures

1 Présentation, contexte et spécifications

1.1	Présentation de l'entreprise	
1.2	Contexte général	
1.3	Présentation du sujet de projet et cahier des charges	
1.3.1	Cahier des charges fonctionnel	
1.3.2	Un projet construit sans cadre initial strict	
1.4	État de l'art	
1.4.1	Architecture générale des systèmes de pièges photographiques	
1.4.2	Critères de conception des systèmes embarqués	
1.4.3	Solutions existantes sur le marché	
1.4.4	Impact sur la faune et évolutions technologiques	

2 Work packages : objectifs, solutions et validations

2.1	WP 1.1 — Capteur de température et détection de mouvement (responsable : Jules Nounba)	
2.1.1	Spécification des tâches vis-à-vis du système	
2.1.2	Étude des composants du module	
2.1.3	Les capteurs détecteurs de mouvements	
2.1.4	Méthodologie de test	
2.1.5	Estimation de la consommation énergétique du composant	
2.1.6	Compétences acquises	
2.1.7	Conclusion	
2.1.8	Ce qu'il faut retenir	
2.2	WP 1.2 — Gestion des échanges données et du microcontrôleur (responsable : Nolan Buchet)	
2.2.1	Rappel des objectifs	
2.2.2	Tâche 0 : Choix du Microcontrôleur	
2.2.3	Tâche 1 : Gestion du stockage des données collectées	
2.2.4	Tâche 2 : Hors WP : Gestion de la luminosité pour WP 1.3	
2.2.5	Tâche 3 : Gestion de l'horodatage et de la localisation du module	
2.2.6	Tâche 4 : Définition d'un automate pour la gestion globale du module	
2.2.7	Tâche 5 : Gestion de l'alimentation du module	
2.2.8	Tâche 6 : Intégration des modules	
2.2.9	Tâche 7 : Développement d'une architecture électrique	
2.2.10	Tâche 8 : Création d'un PCB pour le module	
2.2.11	Ce qu'il faut retenir	
2.3	WP 1.3 — Caméra (responsable : Xueying Xu)	

2.3.1	Rappel des objectifs	
2.3.2	Solution envisagée	
2.3.3	Tests et Validation	
2.3.4	Ce qu'il faut retenir	
2.4	WP 1.4 — Conception et fabrication du boîtier (responsable : Joris Menard)	
2.5	WP bis — Transmission (responsable : Joris Menard)	
2.5.1	Rappel des objectifs	
2.5.2	Solution envisagée	
2.5.3	Mise en œuvre, tests et validation	
2.5.4	Ce qu'il faut retenir	
2.6	WP 2.1 — Stockage distant (responsable : Davy Clark)	
2.6.1	Rappel des objectifs	
2.6.2	Planification du projet	
2.6.3	Conception de la structure des données	
2.6.4	Solution envisagée	
2.6.5	Réalisation	
2.6.6	Limites actuelles et axes d'amélioration	
2.6.7	Tests et validation	
2.7	WP 2.2 — Interface web (responsable : Xueying Xu)	
2.7.1	Rappel des objectifs	
2.7.2	Solution envisagée	
2.7.3	Tests et Validation	
2.7.4	Ce qu'il faut retenir	
2.8	WP 2.3 — Classification animale par IA (responsable : Ewen Le Callet)	
2.8.1	Rappel des objectifs	
2.8.2	Solution envisagée	
2.8.3	Tests et Validation	
2.8.4	Ce qu'il faut retenir	

3 Bilan du projet

3.1	Industrialisation du Projet : Du Prototype à la Production en Série	
3.1.1	Organisation Industrielle	
3.1.2	Validation et Mise en Production	
3.1.3	Analyse Économique	
3.2	Bilan technique	
3.3	Bilan personnel	

Bibliographie

A Annexes

Annexes

A.1	Répartition détaillée des Work-Packages	
A.2	Exemples de données collectées	
A.3	Fichier JSON	

Table des figures

1.1	Zone Natura 2000 "Marais de l'Erdre" présente sur le territoire de Saint-Mars-du-Désert.
1.2	Déploiement national des Atlas de la Biodiversité Communale.
1.3	Photo du marais racheté(source Ouest France)
1.4	Classification animale réalisée par l'équipe avec le modèle <i>SpeciesNet</i> (basé sur <i>MegaDetector</i>).
1.5	Exemple d'un piège photographique.
1.6	Architecture générale du dispositif : capture, transmission et stockage distant.
1.7	Architecture fonctionnelle simplifiée d'un piège photographique.
1.8	Certaines especes restent sensible à la lumière émise par les LED IR...
2.1	Délimitation fonctionnelle du module d'acquisition
2.2	Carte électronique du PIR
2.3	Fonctionnement du capteur DHT22 et du PIR sur ESP32
2.4	Mesure de la température et de l'humidité
2.5	Zone de mesure du PIR
2.6	Module de carte SD utilisé pour le projet
2.7	Module de carte SD utilisé pour le projet
2.8	Résultat du test d'écriture sur la carte SD
2.9	Schéma du montage de diviseur de tension pour la photorésistance
2.10	Courbe de résistance en fonction de la luminosité pour la photorésistance
2.11	Couronne de LED Infrarouge utilisée pour le projet
2.12	Module GPS utilisé pour le projet
2.13	Exemple de trame NMEA envoyée par le module GPS
2.14	Trame GPRMC utilisée pour récupérer la localisation et l'horodatage
2.15	Schéma du principe de fonctionnement du module RTC pour maintenir l'heure en temps réel
2.16	Automate de gestion du module - version 1
2.17	Schéma du montage de diviseur de tension pour la mesure de la tension de la batterie
2.18	Automate de gestion du module - version finale
2.19	Consommation de base de l'ESP32 avec un code vierge
2.20	Consommation de l'ESP32 en fonction de la fréquence du processeur
2.21	Consommation de l'ESP32 en fonction du mode de veille
2.22	Consommation de l'ESP32 en mode Deep Sleep avec réveil par signal GPIO (simulant le PIR)
2.23	Prototype du système avec tous les modules intégrés
2.24	Différence entre la photographie de notre Caméra et de notre Téléphone
2.25	Autre section de notre architecture

2.26	Schéma de l'architecture du système avec l'ESP32 et la PSRAM pour le stockage des images	
2.27	Schéma électrique global du système réalisé avec Kicad	
2.28	Schéma de la partie microcontrôleur du système réalisé avec Kicad	
2.29	Schéma de la partie capteurs du système réalisé avec Kicad	
2.30	Schéma de la partie caméra du système réalisé avec Kicad	
2.31	Autre section de notre architecture	
2.32	Schéma du montage de contrôle d'alimentation avec un transistor MOSFET P-Channel	
2.33	Simulation du fonctionnement du montage de contrôle d'alimentation avec un transistor MOSFET P-Channel	
2.34	Schéma d'alimentation complet du système réalisé avec Kicad	
2.35	Empreintes des composants sur le PCB	
2.36	Rendu final du PCB réalisé avec Kicad	
2.37	Rendu final 3D du PCB réalisé avec Kicad	
2.38	Rendu final PCB après réception	
2.39	Rendu final	
2.40	Rendu final	
2.41	Chaîne d'acquisition et de traitement des images du module caméra	
2.42	Capteur OV2640 utilisé dans le système final	
2.43	Plateforme ESP32-CAM utilisée pour les tests expérimentaux	
2.44	Partie du boîtier accueillant la batterie	
2.45	Encoches pour les sangles de fixation	
2.46	Partie du boîtier accueillant la caméra et le capteur PIR	
2.47	Support pour la carte électronique	
2.48	Support pour le module de communication 4G	
2.49	Pièce de protection des composants électroniques	
2.50	Pièce de protection fixée sur le boîtier	
2.51	Partie du boîtier avec le tenon	
2.52	Partie du boîtier avec la mortaise	
2.53	Clips de fermeture du boîtier	
2.54	Image du boîtier final envisagé	
2.55	Capture d'écran des paramètres du point d'accès du téléphone	
2.56	Données envoyées depuis l'ESP32	
2.57	Données reçues depuis le téléphone	
2.58	Schéma de l'algorithme de calcul du CRC32	
2.59	CRC32 de l'image envoyée et code de retour	
2.60	CRC32 de l'image reçue	
2.61	Carte des antennes présentes dans la zone de Saint-Mars-du-Désert	
2.62	Module de communication TEL0177	
2.63	Vue d'ensemble du flux de stockage distant	
2.64	Schéma de la méthode en cycle en V	
2.65	Diagramme technique du flux de réception et de traitement	
2.66	Schéma relationnel simplifié du stockage distant	
2.67	Architecture simplifiée de l'interface web	
2.68	Exemple d'interface web développée pour le projet	

2.69	Page d'accueil de l'interface web (dashboard)	...
2.70	Visualisation des images avec métadonnées et détection	...
2.71	Interface de filtrage et tri des images	...
2.72	Fonction d'export des données	...
2.73	Journal des événements du système	...
2.74	Visualisation des caméras sur une carte	...
2.75	Image piège caméra du Zoo des Sables d'olonne.	...
2.76	Emplacement du WP "Classification animale par IA" dans notre architecture.	...
2.77	Image de référence (1920*1080).	...
2.78	Résolution limite (700*395).	...
2.79	Inférence effectuée sur YOLOv8 (le label 80 correspondant à "animal").	...
2.80	Même image que celle effectuée sur YOLOv8 mais cette fois-ci inférée sur SpeciesNet.	...
2.81	Méthodologie de taxonomie suivie par le modèle SpeciesNet.	...
2.82	Schéma résumé du traitement de l'image par l'IA.	...
2.83	Image tirée du site web, déplacée dans la section "indésirable" suite à la détection d'un humain.	...
3.1	Version 3D finale de notre PoC.	...
3.2	Visualisation 3D de notre PCB avec les composants.	...
3.3	Serveur AWS (datacenter Amazon).	...
3.4	Tableau des commandes du projet transversal.	...
3.5	Prix pièges caméras aux fonctionnalités similaires.	...

1

Présentation, contexte et spécifications

1.1 Présentation de l'entreprise

La mairie de Saint-Mars-du-Désert est une collectivité territoriale dont la mission première est de répondre aux besoins quotidiens de la population : gestion des services publics, aménagement du territoire, actions sociales, développement local et accompagnement des initiatives citoyennes.

Au-delà de ces responsabilités essentielles, la commune porte une attention particulière aux enjeux environnementaux et affirme un engagement fort en faveur de la transition écologique. Cette priorité s'inscrit dans une dynamique globale de protection du patrimoine naturel, renforcée par la présence, sur le territoire communal, de nombreuses zones classées *Natura 2000* (particulièrement les "Marais").

Le réseau *Natura 2000* regroupe des aires protégées désignées par les États membres de l'Union européenne pour préserver des habitats naturels ainsi que des espèces animales et végétales menacées, conformément aux directives « Oiseaux » et « Habitats, Faune, Flore ».

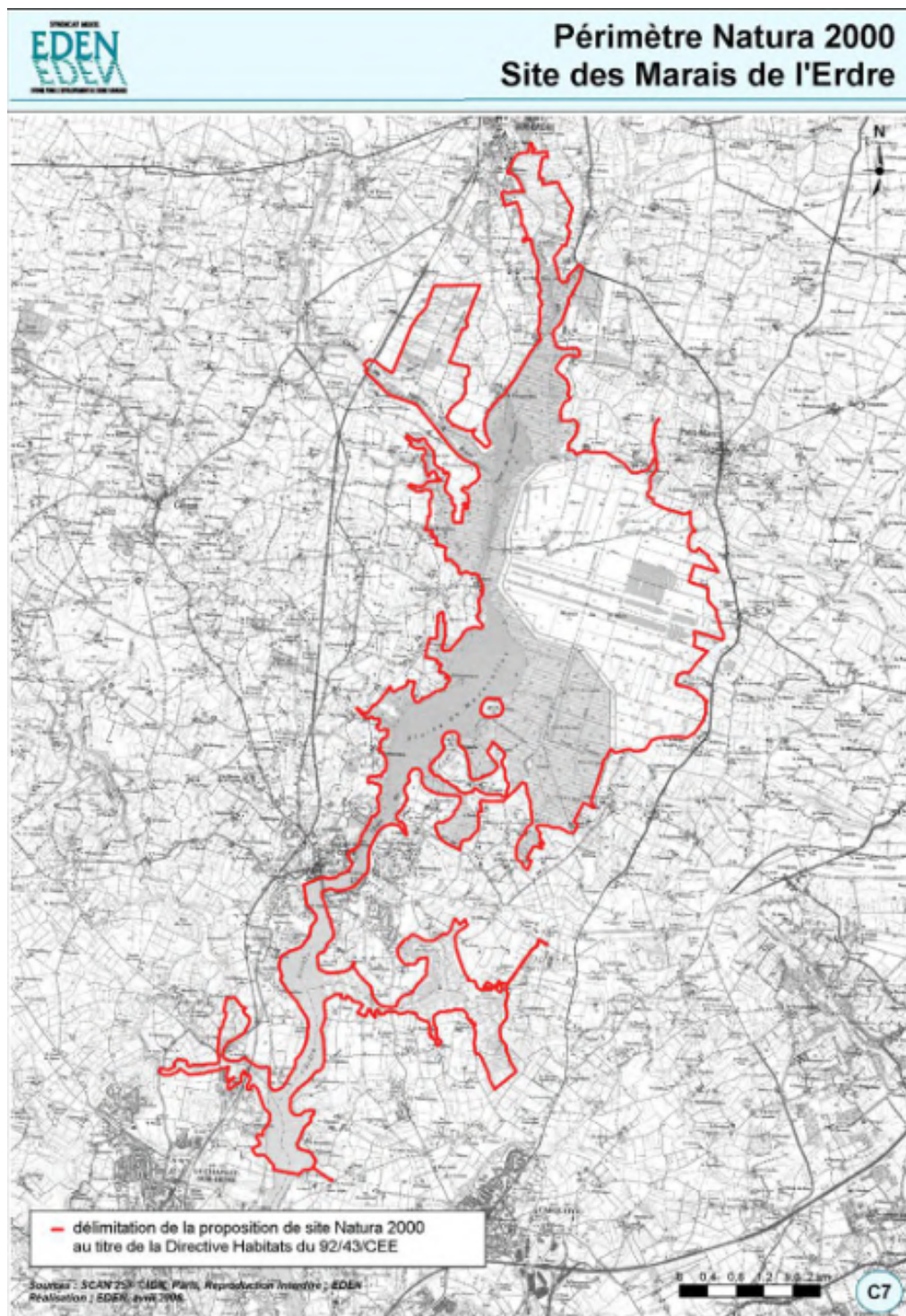


FIGURE 1.1 – Zone Natura 2000 "Marais de l'Erdre" présente sur le territoire de Saint-Mars-du-Désert.

La présence de ces zones de vigilance environnementale permet également à la commune de bénéficier de dispositifs de financement dédiés. Ces soutiens financiers facilitent la mise en œuvre de projets visant à améliorer la compréhension, le suivi et la protection de la faune et de la flore locales.

1.2 Contexte général

Dans le cadre de sa démarche de transition écologique, la commune de Saint-Mars-du-Désert souhaite renforcer ses connaissances sur la faune et la flore locales en élaborant son *Atlas de la Biodiversité Communale* (ABC). Cet outil constitue un support essentiel pour inventorier, comprendre et valoriser la richesse écologique du territoire.

Un ABC est un véritable diagnostic écologique qui met en évidence les principaux enjeux de biodiversité d'un territoire. Il mobilise l'ensemble des acteurs locaux — élus, citoyens, associations et entreprises — autour de la préservation du patrimoine naturel. Concrètement, il permet :

- de cartographier les enjeux de biodiversité à l'échelle communale grâce à des inventaires d'espèces et de milieux ;
- d'établir un état des lieux précis de la faune et de la flore ;
- de définir un plan d'actions opérationnel pour préserver et restaurer ces milieux ;
- d'intégrer plus facilement les enjeux écologiques dans les projets d'aménagement du territoire.

L'ABC constitue ainsi un outil d'information et d'aide à la décision pour la collectivité.

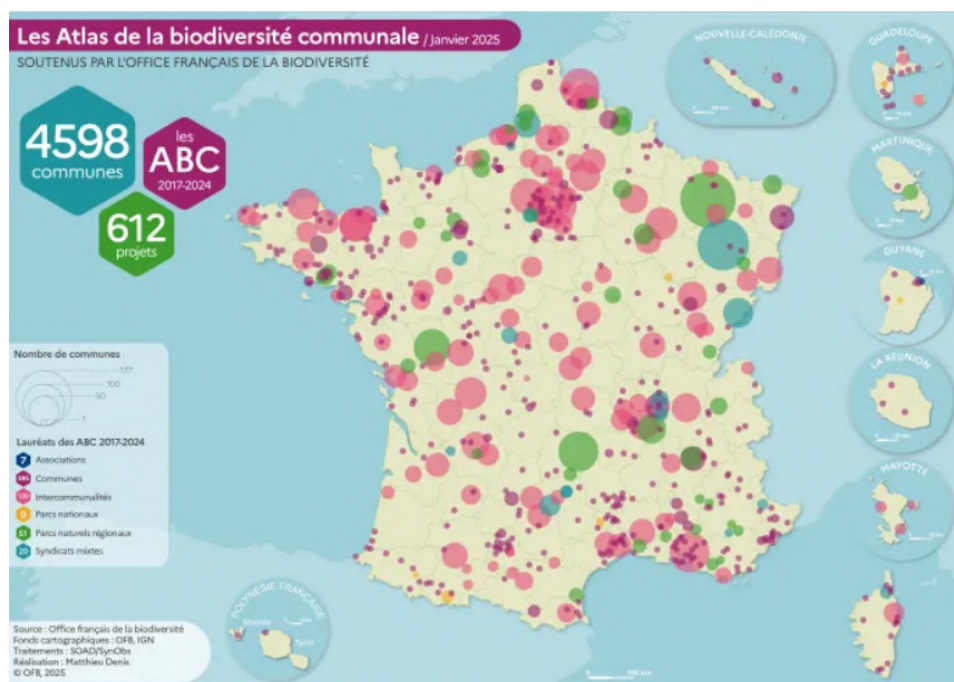


FIGURE 1.2 – Déploiement national des Atlas de la Biodiversité Communale.

Dans ce contexte, l'identification précise et documentée de la présence d'espèces animales constitue un enjeu stratégique. En effet, la collecte de preuves tangibles (observations de terrain, relevés naturalistes, données de suivi) permet d'attester de la richesse écologique d'un site. Ces données sont essentielles pour orienter les politiques publiques locales, mais également pour appuyer des démarches de protection à plus grande échelle ou d'acquisition foncière.

Un exemple concret de cette logique est illustré par l'acquisition récente de 659 hectares de marais dans la vallée de l'Erdre par le :*contentReference[oaicite :0/index=0* et

la :*contentReference[oaicite :1/index=1*. Situé en zone *Natura 2000*, ce site, reconnu pour sa forte biodiversité, la présence d'une avifaune importante et d'un couloir de migration, a été acquis pour environ 3,2 millions d'euros afin de créer un sanctuaire naturel dédié à la protection de la biodiversité et de la ressource en eau ([article Ouest-France](#)).



FIGURE 1.3 – Photo du marais racheté(source Ouest France)

Cette opération, rendue possible grâce au soutien d'acteurs publics tels que l'État et l'Agence de l'eau Loire-Bretagne, illustre l'importance de la capacité à démontrer concrètement la valeur écologique d'un territoire. La production de données fiables sur la faune et les habitats naturels devient ainsi un levier déterminant pour mobiliser des financements, convaincre des partenaires associatifs et, à terme, favoriser des acquisitions foncières ou des mesures de protection renforcées.

Lien avec le projet : genèse d'une problématique locale

Parallèlement, dans le cadre de leur formation à Polytech Nantes, des étudiants du département Électronique et Technologies Numériques (ETN 3A) ont mené des travaux exploratoires sur la classification animale par intelligence artificielle. Leurs expérimentations, réalisées sur ordinateur personnel à partir de données issues de pièges photographiques trouvés sur internet, ont montré le potentiel de ces technologies pour l'étude de la faune sauvage, permettant d'obtenir des informations fiables sans perturber les espèces (ou presque).

Ces travaux préliminaires ont retenu l'attention de plusieurs enseignants ainsi que de partenaires extérieurs, dont M. Xavier Leprévost (Microchip), expert en électronique et collaborateur régulier de la mairie de Saint-Mars-du-Désert. Convaincu de la pertinence du sujet pour le territoire et de son intérêt dans le cadre de l'ABC, il a facilité la mise en relation avec la commune. Cette dynamique a permis de transformer notre initiative

étudiante en un véritable projet concret, retenu comme Projet Transversal (PTrans) pour cette année.

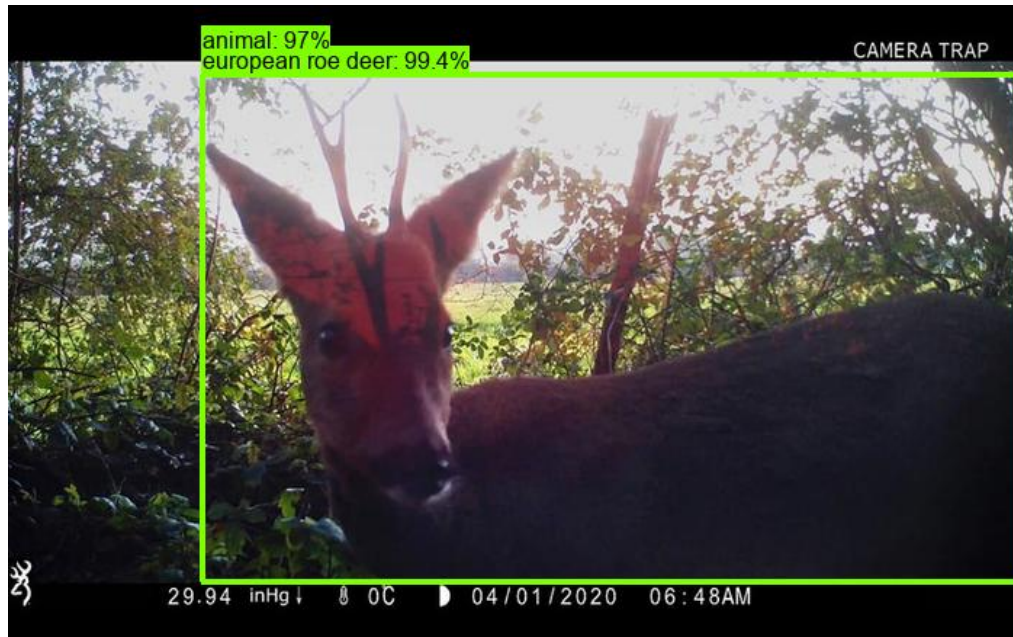


FIGURE 1.4 – Classification animale réalisée par l’équipe avec le modèle *SpeciesNet* (basé sur *MegaDetector*).

1.3 Présentation du sujet de projet et cahier des charges

À la suite des travaux exploratoires et de la mise en relation avec la mairie de Saint-Mars-du-Désert, un besoin concret a émergé : disposer d’un dispositif capable de collecter automatiquement des observations de faune sauvage afin d’alimenter l’Atlas de la Biodiversité Communale (ABC), dans une optique d’identification et de suivi des espèces. Dans la phase d’inventorisation, l’utilisation de *pièges photographiques* constitue une solution pertinente, permettant d’obtenir des données fiables, non intrusives et adaptées au suivi de la biodiversité locale. L’objectif du Projet Transversal (PTrans) est donc de concevoir un premier prototype fonctionnel répondant à ce besoin.



FIGURE 1.5 – Exemple d'un piège photographique.

Ce projet s'inscrit dans une démarche de *preuve de concept*, visant à démontrer la faisabilité d'un système complet de collecte et de transmission de données environnementales avant tout déploiement à plus grande échelle. L'architecture retenue privilégie la simplicité, la robustesse et la reproductibilité sur le terrain.

1.3.1 Cahier des charges fonctionnel

Le système doit répondre à plusieurs fonctionnalités principales :

- **Détection de mouvement animalier** : le dispositif repose sur une détection automatique de mouvements dans son champ de vision, optimisée pour limiter les déclenchements non pertinents (absence d'animaux).
- **Prise de vue automatique** : une photographie est capturée dès qu'un mouvement est détecté.
- **Enregistrement local temporaire** : les images sont stockées dans une mémoire tampon avec leurs métadonnées associées (date, heure, température, humidité, position GPS).
- **Transmission des données** : les informations collectées sont envoyées vers un stockage distant.
- **Autonomie énergétique** : le système, basé sur une alimentation par batterie, doit fonctionner de manière autonome avec une gestion optimisée de la consommation. Un suivi de l'état énergétique doit permettre d'anticiper les opérations de maintenance.
- **Stockage distant** : les données sont centralisées afin de permettre leur exploitation.
- **Interface utilisateur web** : une interface simple permet la consultation des images et des données associées (localisation, paramètres environnementaux, etc.).

Des fonctionnalités supplémentaires ont également été identifiées comme perspectives d'évolution :

- acquisition de données environnementales complémentaires (pression atmosphérique, qualité de l'air, acoustique, etc.) ;
- traitement des images par intelligence artificielle pour la classification des espèces ;
- filtrage automatique des images non pertinentes (floues, sans animal, présence humaine, etc.).

Les deux derniers points, à savoir le traitement des images par intelligence artificielle ainsi que le filtrage automatique des images non pertinentes, ont finalement pu être intégrés dans la preuve de concept finale. Cette intégration permet d'améliorer significativement la pertinence des données exploitées en réduisant le volume d'images inutiles et en facilitant l'identification des espèces.

1.3.2 Un projet construit sans cadre initial strict

Il est important de souligner que le projet ne s'est pas inscrit dès le départ dans un cadre technique entièrement défini. Contrairement à un cahier des charges classique fourni en amont, les premières phases ont été marquées par une absence de directives précises.

L'équipe a ainsi dû adopter une posture autonome, en devenant en quelque sorte son propre chef de projet. Cette situation a nécessité un important travail d'analyse et de structuration afin de transformer un besoin global — la collecte automatisée de données faunistiques — en un cadre technique clair et exploitable.

Cette phase a inclus :

- l'identification des besoins réels du terrain et des utilisateurs finaux ;
- la définition progressive des fonctionnalités prioritaires et secondaires ;
- l'étude des contraintes techniques (autonomie, transmission, stockage, robustesse) ;
- la construction d'une architecture système cohérente à partir de zéro.

Ce travail d'étude et de formalisation a constitué une étape essentielle du projet, permettant de passer d'une idée initiale floue à un système structuré, compatible avec les objectifs de l'Atlas de la Biodiversité Communale.

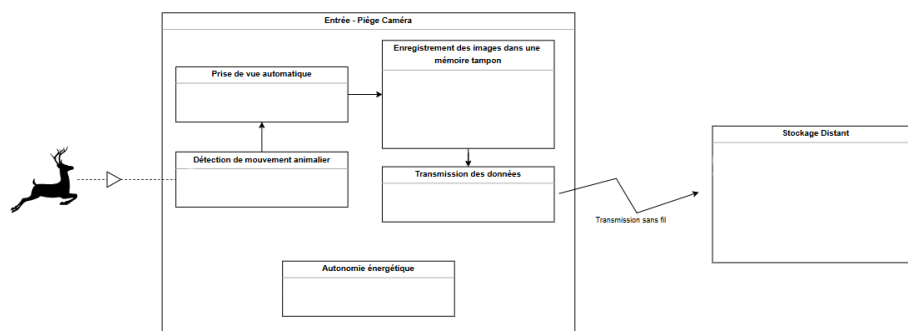


FIGURE 1.6 – Architecture générale du dispositif : capture, transmission et stockage distant.

1.4 État de l'art

Avant de nous lancer dans la conception puis commandes des différents éléments de notre piège caméra, il était important de faire une synthèse ou un état de l'art de l'architecture

principale des pièges caméras aujourd'hui, en prenant en compte leurs atouts et leurs contraintes.

1.4.1 Architecture générale des systèmes de pièges photographiques

Un piège photographique moderne repose sur une architecture modulaire composée de plusieurs blocs fonctionnels interdépendants.

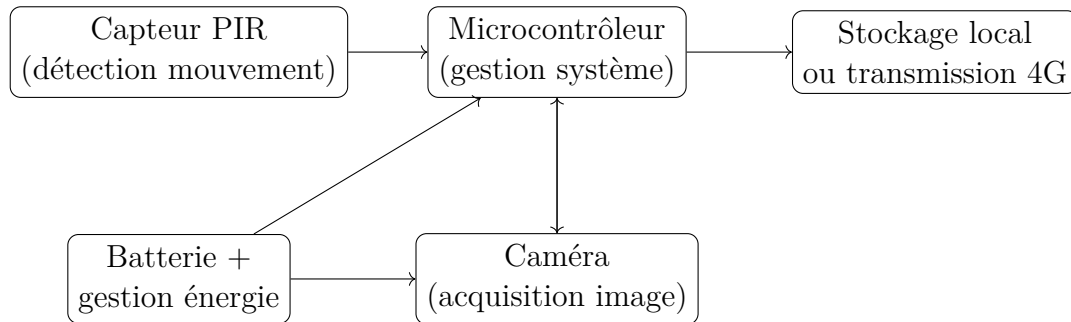


FIGURE 1.7 – Architecture fonctionnelle simplifiée d'un piège photographique.

Le capteur infrarouge passif (PIR) assure la détection des variations thermiques. Le microcontrôleur constitue le cœur du système embarqué et orchestre les différents modules. Les caméras utilisées reposent généralement sur des capteurs CMOS optimisés pour les faibles luminosités, tandis que les données sont soit stockées localement (carte SD), soit transmises sans fils (4G, LORA...).

On appliquera cette architecture pour la décomposition fonctionnelle de notre projet en tâches.

1.4.2 Critères de conception des systèmes embarqués

La conception d'un piège photographique est fortement contrainte par son environnement d'utilisation. Les principaux critères techniques sont :

- autonomie énergétique (fonctionnement longue durée sur batterie) ;
- latence de déclenchement (réactivité entre détection et capture) ;
- robustesse environnementale (humidité, pluie, température) ;
- consommation énergétique globale ;
- capacité de stockage ou transmission des données.

Ces contraintes influencent directement le choix des composants, notamment le microcontrôleur, le capteur image et le mode de communication.

Critère	Contraintes	Choix technologiques associés
Autonomie énergétique	Fonctionnement plusieurs semaines sans maintenance	Microcontrôleur basse consommation, modes sleep, capteur PIR passif
Détection rapide	Réduire la perte de l'animal dans le champ	Microcontrôleur réactif, caméra à faible shutter lag
Qualité d'image	Exploitation des données, qualité minimum pour classification IA	Caméra bonne résolution, appliquer un mode nuit (Caméra IR ou leds)
Connectivité	Réduction des déplacements terrain	Module 4G, transmission cloud
Robustesse	Conditions extérieures difficiles	Boîtier étanche et matériaux résistants
Stockage	Volume élevé d'images	Compression, stockage distant

TABLE 1.1 – Correspondance entre critères de conception et choix technologiques.

1.4.3 Solutions existantes sur le marché

Les pièges photographiques commerciaux se différencient principalement par leur niveau de connectivité et d'intelligence embarquée.

Les modèles de première génération, tels que ceux de *Bushnell*, fonctionnent de manière totalement autonome avec stockage local sur carte SD. Ils sont simples, robustes et économes en énergie, mais nécessitent une récupération manuelle des données.

Des fabricants comme *Reconyx* proposent des systèmes haut de gamme optimisés pour la réactivité et la qualité d'image, avec une très faible latence de déclenchement et une grande fiabilité en conditions extrêmes, mais toujours sans connectivité avancée.

Enfin, les solutions récentes dites "cellulaires" intègrent des modules 4G/LTE permettant l'envoi automatique des images vers le cloud. Ces systèmes facilitent le suivi en temps réel mais augmentent fortement la consommation énergétique et le coût global du dispositif.

1.4.4 Impact sur la faune et évolutions technologiques

Bien que non intrusifs, les pièges photographiques peuvent influencer le comportement animal, notamment via les dispositifs d'éclairage nocturne. Les systèmes modernes privilégient les LED infrarouges invisibles (ou presque) afin de limiter toute perturbation comportementale.



FIGURE 1.8 – Certaines especes restent sensible à la lumière émise par les LED IR...

Les évolutions récentes s'orientent vers des systèmes intelligents intégrant du traitement d'image par intelligence artificielle. Ces approches permettent la classification automatique des espèces et la suppression des images non pertinentes, réduisant ainsi la charge de traitement et améliorant la qualité des données exploitables. L'idée d'intégrer une classification animale par IA directement sur le piège caméra (*in-place*) a été longuement discuté avec les coachs et le client. Il a finalement été décidé de garder cette tâche sur un serveur distant, ramenant cette tâche à un niveau de priorité secondaire.

2

Work packages : objectifs, solutions et validations

Ce chapitre présente la décomposition du projet en work packages, ainsi que les objectifs, les solutions retenues et les validations associées pour chaque lot de travail effectivement documenté dans ce rapport.

2.1 WP 1.1 — Capteur de température et détection de mouvement(responsable : Jules Nomba)

2.1.1 Spécification des tâches vis-à-vis du système

Délimitation du projet Notre mission vise à mettre en place l'acquisition de l'ensemble des données nécessaires pour notre client. Suite à l'étude du cahier des charges, nous retenons que le module d'acquisition traite des données telles que : la température, l'humidité, l'image et les coordonnées géographiques. Tous ces éléments sont transmis, stockés et classifiés dans une base de données à distance, à disposition des utilisateurs, comme l'illustre la figure 2.1 ci-dessous.

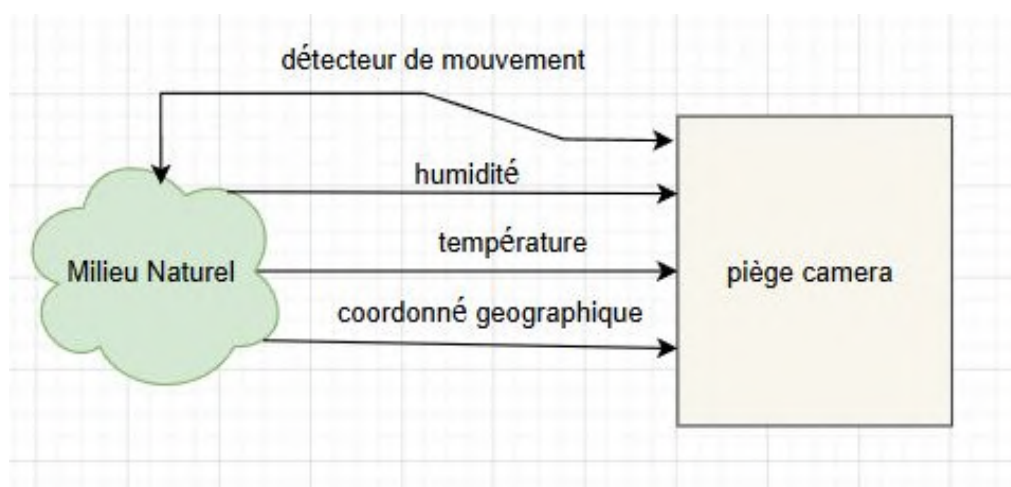


FIGURE 2.1 – Délimitation fonctionnelle du module d'acquisition

La figure 2.1 illustre l'ensemble des interactions entre le système étudié et son milieu extérieur. Chaque élément du milieu naturel est représenté par un capteur, dont le rôle est de convertir des signaux analogiques issus de l'environnement en signaux numériques exploitables par un ordinateur. Cette conversion permet de produire des informations utiles au traitement des données relatives aux animaux.

Ainsi, l'objectif de cette partie est d'étudier les différents capteurs utilisés pour l'acquisition de données animales et environnementales. Dans ce contexte, l'utilisation d'un piège photographique doit être conçue de manière à ne pas perturber l'équilibre des écosystèmes dans lesquels il est installé.

Définitions des contraintes environnementales L'utilisation des capteurs peut avoir une influence significative sur le mode de vie des animaux, notamment lorsqu'ils sont conçus à partir de matériaux ou de composants potentiellement dangereux pour certaines espèces. Ainsi, en fonction des paramètres présentés à la figure 2.1, une attention particulière doit être portée aux éléments suivants :

- l'utilisation des ondes infrarouges ou sonores ;
- l'éclairage ;
- les risques de pollution.

2.1.2 Étude des composants du module

Définition Un capteur est un composant actif ou passif capable de transformer des grandeurs physiques en grandeur électrique (analogique ou numérique).

Capteur de température et d’humidité Le choix du capteur DHT22 repose sur un compromis entre simplicité d’utilisation, faible consommation énergétique, précision suffisante et facilité d’intégration. Contrairement à certains capteurs plus précis mais plus complexes (comme le Pt100 ou les thermocouples), le DHT22 permet une acquisition directe de la température et de l’humidité avec un minimum de composants externes, ce qui réduit le coût, la consommation et l’impact environnemental global du système. De plus, il ne nécessite ni amplification de signal ni circuit de conditionnement complexe, contrairement à des capteurs analogiques comme le LM35 ou les thermistances.

TABLE 2.1 – Comparaison des capteurs (Tableau 1)

Tableau comparatif des capteurs

Capteur	Type	Précision	Consommation	Complexité d'intégration	Avantages	Inconvénients	℄
Pt100	Résistif	Très élevée	Moyenne	Élevée	Très précis, stable	Nécessite conditionnement (pont de Wheatstone, amplification)	
NTC / PTC	Résistif	Moyenne	Faible	Moyenne	Peu coûteux	Non linéaire, calibration nécessaire	
Thermocouple K/J/T	Électrique	Moyenne à élevée	Très faible	Élevée	Large plage de température	Nécessite compensation de jonction froide	
LM35	Analogique	Bonne	Faible	Moyenne	Sortie linéaire simple	Ne mesure que la température	
DS18B20	Numérique	Bonne ($\pm 0.5^{\circ}\text{C}$)	Très faible	Faible	Numérique, facile à utiliser	Temps de conversion plus lent	
Capteur infrarouge	Sans contact	Variable	Faible	Moyenne	Mesure sans contact	Sensible aux conditions environnementales	
DHT22	Numérique	Bonne ($\pm 0.5^{\circ}\text{C}$, $\pm 2-5\% \text{ HR}$)	Très faible	Très faible	Température + humidité, simple, peu coûteux	Fréquence d'échantillonnage limitée	

Suite au tableau 2.1 comparatif des capteurs, le DHT22 est retenu car il combine plusieurs avantages essentiels pour un système embarqué écologique : faible consommation énergétique, intégration simple sans composants externes, mesure simultanée de la température et de l’humidité, et coût réduit. Ces caractéristiques en font un choix optimal pour des dispositifs autonomes tels que les systèmes de surveillance environnementale ou les pièges photographiques écologiques.

Étude du fonctionnement Le capteur DHT22 est un dispositif électronique qui transforme les grandeurs physiques telles que la température et l’humidité en grandeur électrique sous forme de donnée numérique exploitable par ordinateur.

Taille des données Les données reçues par le microcontrôleur (MCU) en provenance du capteur DHT22 sont transmises sous forme d’une trame numérique. Cette trame est composée de 40 bits, répartis comme suit :

DATA = 8 bits (partie entière humidité) + 8 bits (partie décimale humidité) + 8 bits (partie entière température) + 8 bits (partie décimale température) + 8 bits (checksum).

Les 16 premiers bits correspondent à l'humidité relative (partie entière et décimale), tandis que les 16 bits suivants représentent la température (partie entière et décimale). Enfin, les 8 derniers bits constituent le checksum, utilisé pour vérifier l'intégrité des données transmises.

Ainsi, cette organisation permet au microcontrôleur de reconstruire précisément les valeurs mesurées. Le checksum joue un rôle essentiel en assurant que les données reçues ne sont pas corrompues lors de la transmission entre le capteur et le microcontrôleur.

Temps de réponse Un système embarqué à microprocesseur se compose d'une unité de traitement (MCU) et de modules périphériques tels que les capteurs, chargés de l'acquisition des données physiques.

Le temps de réponse correspond à la latence entre la requête du MCU et la réception des données, et dépend du temps de conversion, de la communication et du traitement interne :

$$T = T_{\text{acquisition}} + T_{\text{transmission}} + T_{\text{traitement}}$$

Par conséquent, la datasheet du DHT22 donne un temps de réponse de $T = 5$ ms.

En conclusion, le capteur DHT22 offre un bon compromis entre précision, faible consommation et simplicité d'intégration. Il constitue ainsi une solution adaptée aux systèmes embarqués avec un impact environnemental limité.

2.1.3 Les capteurs détecteurs de mouvements

Besoin fonctionnel Détecter automatiquement la présence d'un animal sans utiliser la caméra. Comme l'indique la figure 2.1, le capteur détecteur de mouvement est l'élément responsable du déclenchement de l'ensemble du système du piège caméra.

Fonctionnement du capteur détecteur de mouvement Ce capteur est constitué de deux cellules sensibles au rayonnement infrarouge. Il se déclenche lorsque le signal reçu (lumière infrarouge émise ou réfléchi) dépasse un seuil de tension prédéfini dans le circuit. Ce seuil dépend principalement de la distance de l'objet, de la nature de sa surface (réflectivité) et de la puissance de la LED infrarouge.

Remarque : ce capteur de mouvement pilote le fonctionnement global du système. Il assure sa mise en marche et synchronise son activation dès qu'un mouvement est détecté.

Caractéristiques principales

- **Portée :** 5 à 12 mètres en général
- **Angle de détection :** 110° à 180°
- **Faible consommation :** fonctionne souvent sur pile
- **Capteur passif :** n'émet rien, il ne fait que recevoir

Choix du capteur détecteur de mouvement Le capteur infrarouge passif (PIR) détecte les variations de chaleur émises par les êtres vivants, comme les humains et les animaux. Il présente plusieurs avantages : il consomme peu d'énergie, génère peu de fausses alertes et, surtout, il n'est pas intrusif, ce qui signifie qu'il n'influence pas le comportement des animaux observés.

Cependant, il reste sensible à la température ambiante, ce qui peut parfois limiter sa précision dans certains environnements.

À l'inverse, les capteurs à ultrasons et les capteurs micro-ondes fonctionnent en émettant des ondes et en analysant leur retour. Même s'ils sont efficaces, ils consomment davantage d'énergie. Les capteurs micro-ondes sont aussi plus coûteux et moins sélectifs. De plus, l'émission d'ondes peut perturber certains animaux et modifier leur comportement, ce qui est un inconvénient dans un contexte d'étude environnementale.

Module capteur PIR — étude du fonctionnement Le module capteur est une unité compacte qui intègre un capteur infrarouge passif (PIR) ainsi que les composants électroniques associés. Le tableau 2.2 ci-dessous décrit le fonctionnement du capteur PIR ainsi que l'ensemble de ses circuits d'interaction.

TABLE 2.2 – Description des circuits fonctionnels du PIR (Tableau 2)

Composant	Fonction principale	Rôle et fonctionnalités	Remarques
Capteur infrarouge passif	Détecte le rayonnement infrarouge	L'élément central du module ; détecte les changements de chaleur causés par les mouvements humains/animaux	Habituellement une petite boîte en métal
Circuit d'amplification et de traitement du signal	Amplifie et filtre les signaux	Amplifie les signaux faibles et supprime le bruit pour garantir une détection de mouvement précise	Utilise des amplificateurs opérationnels ou des circuits intégrés dédiés
Comparateur ou circuit intégré de contrôle	Jugement et traitement du signal	Détermine si un mouvement valide est détecté et émet une commande de contrôle	Améliore la fiabilité du module
Potentiomètre réglable	Réglage de la sensibilité et du délai	Permet aux utilisateurs de définir de manière flexible la plage de détection, la sensibilité et le délai de sortie	Réglage par bouton rotatif ou par vis
Interface de sortie	Sortie du signal	Fournit une sortie numérique HAUT/BAS simple pour les microcontrôleurs, relais, etc.	Généralement une interface à 3 broches
Alimentation	Fournit la tension de fonctionnement	Généralement 3–5 V CC, compatible avec de nombreux circuits de commande	Notez la polarité et la tension
Lentille de Fresnel (dans certains modules)	Se concentre sur les signaux infrarouges	Améliore la portée et l'angle de détection, permettant ainsi au PIR capteur pour détecter efficacement les sources de chaleur dans la zone	Souvent un dôme en plastique blanc

La figure 2.2 présentée ci-dessous est constituée des éléments suivants :

- *Hold Delay* : définit la période de déclenchement du PIR ;
- *Sensitivity* : utilisé pour régler le degré de sensibilité du système ;
- *Vcc* : tension d'alimentation du système, ici 3 V ou 5 V.

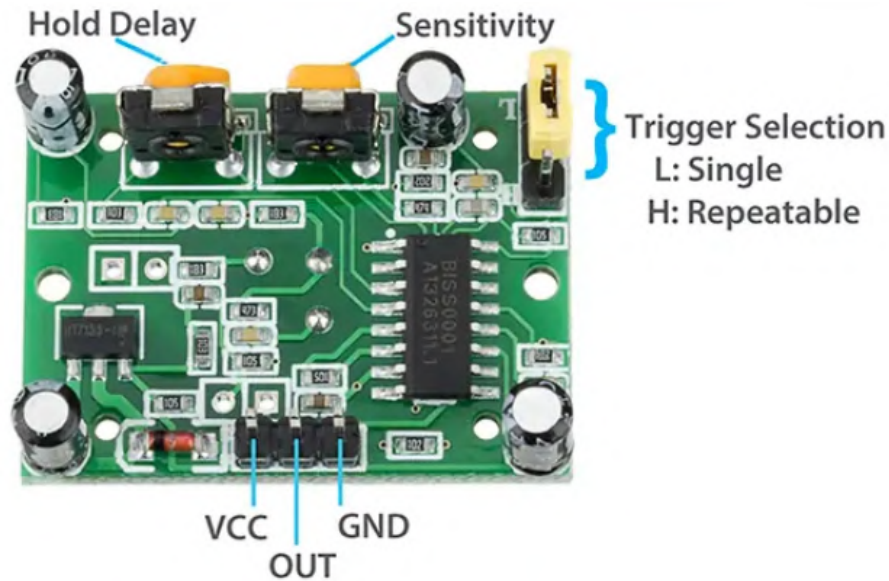


FIGURE 2.2 – Carte électronique du PIR

Conclusion partielle : les tests réalisés montrent que le système fonctionne bien, de manière stable et fiable, avec une bonne détection du capteur PIR et des mesures cohérentes du DHT22. Il peut donc être utilisé en milieu naturel sans perturber l'environnement, même s'il reste légèrement sensible aux variations de température ambiante.

2.1.4 Méthodologie de test

Deux tests seront effectués dans cette étude : un test de fonctionnement unitaire et un test du niveau de tension.

Test unitaire sur chaque composant

- **Test d'alimentation :** les composants tels que le PIR et le DHT22 sont alimentés en 5 V. Ce test consiste à alimenter le composant en 5 V et à tester son fonctionnement en vérifiant sa stabilité sans dysfonctionnement. Cette vérification a été effective lors des tests de fonctionnement où l'on a constaté un transfert continu d'information de déclenchement du PIR ou d'affichage de la température et de l'humidité.
- **Test de fonctionnement :**

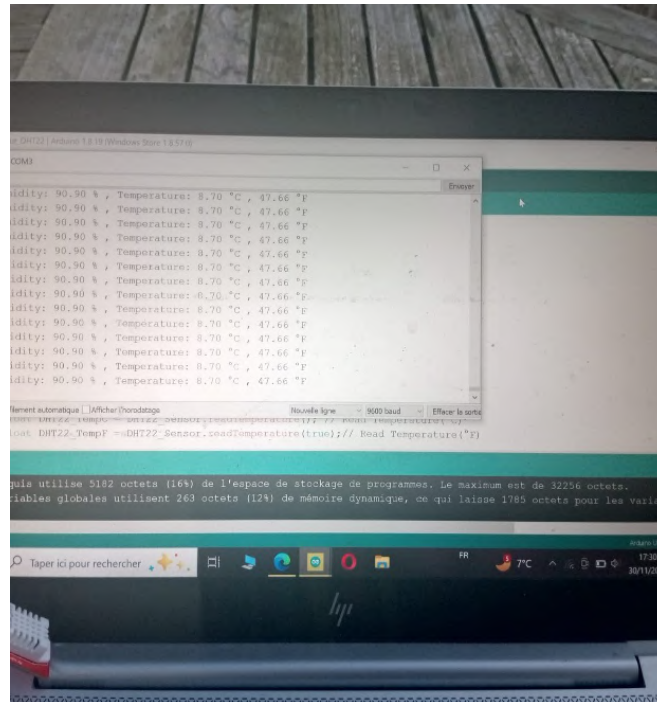


FIGURE 2.3 – Fonctionnement du capteur DHT22 et du PIR sur ESP32

Suite à ce test, à chaque activation du capteur PIR, le capteur DHT22 est mis en fonctionnement. Cela permet alors de récupérer les valeurs de température et d'humidité associées à l'environnement mesuré.

Test longue durée (stabilité) Nous ne constatons aucune perte de communication entre le DHT22 et le microcontrôleur au cours de son fonctionnement. Les données de température et d'humidité restent stables sur une durée de 50 minutes d'utilisation du système. Les figures 2.3 et 2.4, respectivement prises au début de l'expérience et après 50 minutes, illustrent la stabilité du capteur DHT22 dans le temps.

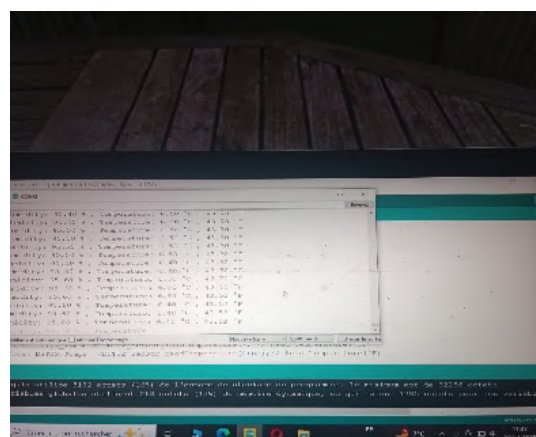


FIGURE 2.4 – Mesure de la température et de l'humidité

Test général du système

Test de distance Nous cherchons à déterminer la distance maximale de fonctionnement du capteur PIR pour la détection d'un objet.

Objectif : identifier les distances minimale et maximale de déclenchement correspondant à un changement d'état du capteur PIR.

Méthode : observation et mesure.

Mesure de la distance Nous nous déplaçons progressivement vers le capteur. Lorsqu'une variation liée à notre présence est détectée, un observateur nous signale immédiatement afin de stopper le déplacement. Le changement d'état du système interne est alors relevé pour déterminer la distance de déclenchement.

Pour ce test, la moyenne est de 4,8 m à 7 m selon la sensibilité du capteur et le niveau de luminosité.

Nous mesurons l'angle de couverture nécessaire pour le fonctionnement de notre piège photographique. Lors de déplacements latéraux par rapport au capteur, nous observons une variation du signal dès que le corps entre dans la zone de détection. En relevant les positions limites de détection de part et d'autre, nous estimons que l'angle de couverture du capteur PIR est d'environ 60°.

La figure 2.5 présente la zone d'observation du capteur. Elle correspond approximativement à une surface de :

$$S = \frac{60}{360} \times \pi r^2 \approx 13 \text{ m}^2$$

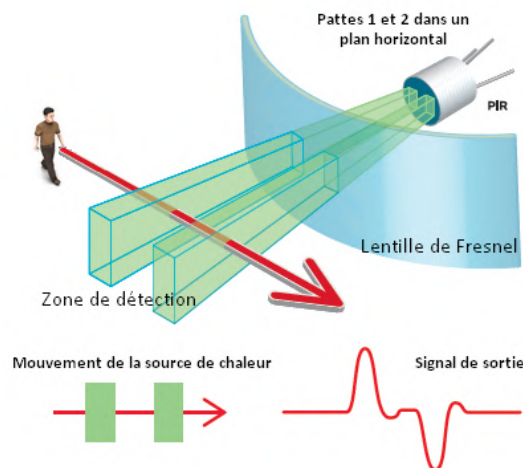


FIGURE 2.5 – Zone de mesure du PIR

Risques et perturbations Le rôle du capteur PIR est de détecter le mouvement d'un animal dans une zone définie afin de déclencher les autres éléments du piège photographique. Cependant, il est important de noter que les variations de température ambiante peuvent perturber son fonctionnement et influencer la qualité de la détection.

Ce capteur est particulièrement efficace car il est passif : il ne génère aucune onde, contrairement à un radar actif. De ce fait, sa consommation énergétique reste très faible.

2.1.5 Estimation de la consommation énergétique du composant

Comprendre les notions de base

- Puissance électrique (P) : énergie consommée par unité de temps.

$$P = V \times I$$

avec V la tension aux bornes du composant (en volts) et I le courant qui traverse le composant (en ampères).

- Énergie consommée (E) : intégrale de la puissance sur le temps.

$$E = \int P(t) dt$$

ou, si P est constante sur une durée Δt :

$$E = P \times \Delta t$$

Consommation statique vs dynamique

Statique (repos) Consommation lorsque le composant est sous tension mais sans activité.

Mode veille : les seuls composants en fonctionnement sont le capteur détecteur de mouvement PIR et le microcontrôleur en veille.

TABLE 2.3 – Mode de fonctionnement de l'ESP32 en veille

Composant	Tension	Courant	Durée de fonctionnement	Mode
Micro-processeur	5 V	3 mA	—	Deep sleep (10–150 μ A)
PIR	5 V	10 à 20 mA	—	—

Consommation du système sur 24 h :

$$E_s = 5 \times 3 \times 24 \times 3600 + 5 \times 20$$

$$E_s = 1296 + 8640$$

$$E_s = 9936 \text{ J} \approx 0,1145 \text{ Wh}$$

Dynamique (fonctionnement actif) Consommation liée au changement d'état du composant (commutation de transistors, calculs, transmission de données). Pour les circuits numériques, la puissance dynamique est :

$$P_{\text{dyn}} = C_{\text{eff}} \times V^2 \times f$$

où C_{eff} est la capacité effective chargée/déchargée à chaque cycle, V la tension d'alimentation et f la fréquence de commutation.

Dans les tableaux ci-dessous, nous associons cette consommation énergétique à chaque composant du système afin d'évaluer la consommation globale.

TABLE 2.4 – Récapitulatif tension/courant de tous les composants liés au piège caméra (Tableau 3)

Composant	Temps	Tension	Courant max.	Énergie
ESP32 (WiFi)	3 s	5 V	300 mA	4,5 J
PIR	2 s	5 V	135 μ A	13,5 μ J
DHT22	3 s	5 V	3 mA	0,045 J
Caméra	0,5 s	5 V	120 mA	0,6 J
LED infrarouge	1 s	12 V	260 mA	6,144 J
GPS	60 s	5 V	60 mA	18 J
Temps total				69,5 s
Énergie totale consommée par le module				34,067 J
Temps total sans GPS				9,5 s
Énergie totale sans GPS				671 mJ

Dimensionnement de l'alimentation Le dimensionnement de la batterie repose sur les caractéristiques électriques des différents composants du système embarqué, notamment la tension d'alimentation requise et le courant maximal consommé.

À partir de l'analyse récapitulative des tensions et courants (Tableau 2.4), le courant maximal global du système est évalué à :

- $I_{\max} = 0,743 \text{ A}$
- $V = 12 \text{ V}$

Ainsi, le système nécessite une alimentation stabilisée de **12 V**, capable de fournir un courant minimal de **0,75 A** afin d'assurer un fonctionnement nominal sans chute de tension ni limitation en courant.

Analyse de la consommation énergétique de l'ESP32

Hypothèses de fonctionnement L'ESP32 est étudié en mode de fonctionnement sans communication Wi-Fi active. Les paramètres considérés sont :

- Courant moyen : $I = 80 \text{ mA}$
- Tension d'alimentation : $V = 5 \text{ V}$
- Temps de réveil : $t = 0,3 \text{ s}$

Énergie consommée lors d'un cycle

$$E = V \times I \times t$$

$$E = 5 \times 0,08 \times 0,3 = 0,12 \text{ J}$$

Ainsi, l'ESP32 présente une très faible consommation énergétique par cycle de réveil.

Analyse temporelle et énergétique du fonctionnement de l'ESP32 L'ESP32 fonctionne à une fréquence de 240 MHz, soit une période :

$$T = 4,16 \text{ ns}$$

Pour environ 165 instructions simples exécutées :

- Temps de calcul : $6,87 \times 10^{-7}$ s
- Temps d'attente (délais cumulés) : 0,5 s
- Temps total d'exécution : $T_{\text{total}} \approx 0,500000687$ s

Énergie associée à l'exécution

$$E = V \times I \times t = 5 \times 0,08 \times 0,500000687$$
$$E \approx 0,2 \text{ J}$$

Énergie en mode de fonctionnement global En prenant en compte les phases d'attente et de communication interne, l'énergie globale estimée est :

$$E_{\text{fonctionnement}} \approx 10,6 \text{ J}$$

Bilan énergétique global

$$E_{\text{total}} = 10,6 + 0,2 + 0,12 + 37,067 = 47,74 \text{ J}$$

$$E_{\text{total}} \approx 0,0132 \text{ Wh}$$

Conclusion partielle Le système présente une consommation énergétique faible, permettant une autonomie théorique d'environ **28 jours**, soit approximativement un mois de fonctionnement continu dans des conditions optimales.

2.1.6 Compétences acquises

Au cours de ce projet, j'ai pu développer plusieurs compétences, aussi bien techniques que méthodologiques.

Sur le plan technique, j'ai appris à mieux comprendre et utiliser des systèmes embarqués, notamment avec l'ESP32 et différents capteurs comme le PIR et le DHT22. J'ai également acquis des connaissances en gestion de l'énergie, en particulier pour dimensionner une alimentation et optimiser la consommation d'un système autonome.

Ce projet m'a aussi permis de mettre en pratique des méthodes d'analyse, en réalisant des tests concrets afin d'évaluer les performances du système, comme le temps de réponse, la stabilité des mesures et la consommation énergétique.

De plus, j'ai développé des compétences en gestion du temps, ainsi que des compétences en communication, notamment à travers l'écoute active et la conception de supports visuels tels que des posters.

Enfin, j'ai amélioré ma manière de travailler, en étant plus rigoureux dans l'analyse, en structurant mieux mes idées dans un rapport technique, et en prenant en compte des contraintes réelles comme l'impact environnemental et l'autonomie du système.

2.1.7 Conclusion

Cette étude a permis de mieux comprendre le fonctionnement énergétique du système et de valider son choix d'alimentation. Avec une batterie de 12 V capable de fournir environ 0,75 A, l'ensemble des composants peut fonctionner correctement sans contrainte particulière.

Les différents calculs montrent que la consommation globale reste faible, notamment grâce à l'utilisation de composants à basse consommation comme l'ESP32 et à une gestion efficace des phases d'activité et de repos. L'énergie totale estimée est d'environ 47,74 J (soit 0,0132 Wh), ce qui confirme que le système est très peu énergivore.

Au final, ces résultats montrent que le dispositif peut fonctionner de manière autonome pendant environ un mois dans des conditions normales d'utilisation. Cela confirme la pertinence du choix des composants pour une application de surveillance en milieu naturel, où l'autonomie et la faible consommation sont des critères essentiels.

2.1.8 Ce qu'il faut retenir

- Le module d'acquisition traite quatre types de données : température, humidité, image et coordonnées géographiques, transmises à une base de données distante.
- Le capteur **DHT22** est retenu pour sa simplicité d'intégration, sa faible consommation et sa capacité à mesurer simultanément la température et l'humidité sans circuit de conditionnement externe.
- Le capteur **PIR** est choisi comme déclencheur principal du système : passif, non intrusif pour la faune, avec une portée de 5 à 12 m et un angle de couverture d'environ 60°.
- Les tests unitaires et de longue durée (50 min) confirment la **stabilité et la fiabilité** du système en conditions réelles.
- La consommation énergétique totale est estimée à **47,74 J** (0,0132 Wh) par cycle, permettant une autonomie théorique d'environ **un mois** avec une alimentation 12 V / 0,75 A.
- L'ensemble du dispositif est conçu pour minimiser son impact sur les écosystèmes : aucune émission d'onde, faible consommation, composants à basse tension.

2.2 WP 1.2 — Gestion des échanges données et du microcontrôleur (responsable : Nolan Buchet)

2.2.1 Rappel des objectifs

Pour recontextualiser le projet et les objectifs de ce Work Package, le WP 1.2 fait partie du Work Package 1 qui est dédié à la conception et au développement du système de collecte de données animalières. Le WP 1.2 se concentre spécifiquement sur la gestion des échanges de données entre les différents composants du système, ainsi que sur l'intégration et la programmation du microcontrôleur qui servira de cerveau central pour le projet. De plus celui-ci s'occupera de la partie énergétique du module, ainsi que de la partie stockage des données collectées.

Voici la liste des tâches qui seront réalisées dans le cadre de ce WP :

- Tâche 0 (Équipe) : Choix du microcontrôleur et de la carte SD
- Tâche 1 : Gestion du stockage des données collectées
- Tâche 2 (hors WP) : Gestion de la luminosité pour WP 1.3
- Tâche 3 : Gestion de l'horodatage et de la localisation du module
- Tâche 4 : Définition d'un automate pour la gestion globale du module
- Tâche 5 : Gestion de l'alimentation du module
- Tâche 6 : Intégration des modules (prototypage)
- Tâche 7 : Développement d'une architecture électrique
- Tâche 8 : Création d'un PCB pour le module

Pour chacune des tâches réalisées ci-dessus, il y a eu une phase de test réalisée, ainsi qu'en amont une phase de concertation avec l'équipe du WP afin que les solutions envisagées soient en adéquation avec les besoins du projet.

Certaines tâches de ce module ne sont pas dans l'ordre de conception, comme la tâche 4 ou 5 qui ont été en constante évolution durant tout le projet. Et certaines tâches non essentielles, mais logiques, comme l'optimisation du code ou le rapport / documentation, ont été réalisées en parallèle de la conception du module.

2.2.2 Tâche 0 : Choix du Microcontrôleur

Solution envisagée

À la suite d'une phase de recherche (benchmark du début de projet) concernant le contrôleur à utiliser pour ce projet, nous avons identifié deux solutions susceptibles de répondre à nos besoins : l'ESP32 et le Raspberry Pi Zero.

Pour rappel, les besoins principaux concernant le contrôleur étaient les suivants :

Critère	Description
Consommation énergétique	Minimiser la consommation pour un usage sur batterie.
Performances	Garantir une puissance suffisante pour le traitement des données.
Stockage	Disposer d'une capacité suffisante pour enregistrer temporairement les mesures locales et la photographie.
Système	Offrir un environnement logiciel simple avec facilité sur les outils de développement utilisés.
Interfaces	Proposer des entrées/sorties adaptées aux capteurs et modules externes.
Connectivités	Intégrer ou supporter le Wi-Fi, le Bluetooth ou d'autres moyens de communication.
Prix	Rester dans une gamme de coût compatible avec le budget du projet.

Ces critères nous ont permis de comparer plusieurs solutions techniques lors de la phase de benchmark.

Les critères suivants ont été considérés lors du choix final du microcontrôleur :

- Architecture matérielle
- Connectivité
- Consommation énergétique
- Nombre et type de GPIO
- Facilité de développement
- Coût global

À l'issue de l'analyse comparative réalisée entre l'ESP32 et le Raspberry Pi Zero, le choix s'est porté sur l'ESP32 comme microcontrôleur principal pour le collecteur de données animalières. Ce choix est motivé par plusieurs facteurs clés. Tout d'abord, sa faible consommation énergétique constitue un avantage décisif pour une utilisation sur batterie dans un environnement isolé. L'ESP32 peut fonctionner plusieurs jours, voire semaines, en mode basse consommation, ce qui garantit une autonomie optimale sur le terrain.

Ensuite, son intégration native du Wi-Fi et du Bluetooth simplifie la conception matérielle et réduit le besoin de modules externes, tout en permettant la transmission sans

fil des données et la configuration à distance du système.

Enfin, son coût réduit et sa disponibilité sur le marché en font une solution économique et accessible, sans compromis majeur sur les performances nécessaires au projet. En revanche, il est à noter que le Raspberry Pi Zero offre une puissance de calcul et une mémoire supérieures, mais celles-ci sont jugées excessives au regard des besoins du projet et pénalisantes en termes de consommation énergétique et de gestion logicielle.

En conclusion, l'ESP32 répond pleinement aux exigences du projet en matière de consommation, de connectivité et de simplicité de déploiement, ce qui en fait la solution la plus adaptée pour le collecteur de données animalières.

Notes : Un document détaillé présentant l'analyse comparative entre l'ESP32 et le Raspberry Pi Zero, ainsi que les critères de sélection, a été rédigé.

2.2.3 Tâche 1 : Gestion du stockage des données collectées

Rappel des objectifs

Commençons par la tâche 1, qui est la gestion du stockage des données collectées. Pour cette tâche, il a été décidé d'utiliser une carte SD (par cahier des charges) pour stocker les données collectées, notamment les métadonnées et les images. La carte SD offre une grande capacité de stockage, ce qui est essentiel pour notre projet. De plus, les cartes SD sont relativement faciles à interfacer avec un microcontrôleur, ce qui facilite l'intégration dans notre système. Enfin, celle-ci est facilement accessible pour les utilisateurs, ce qui permet une récupération facile des données collectées.

Donc nous avons utilisé un module de carte SD utilisable via le moyen de communication SPI, qui est un protocole de communication série rapide et efficace. Le microcontrôleur peut ainsi envoyer des commandes à la carte SD pour lire et écrire des données de manière rapide et fiable. Le module utilisé est un module personnel (**AZDelivery SPI Reader**), celui-ci a été choisi au début du projet car aucune commande n'était envisageable de la part du département ou du client.

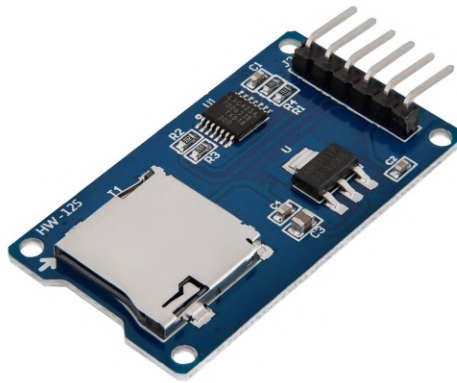


FIGURE 2.6 – Module de carte SD utilisé pour le projet

Solution envisagée

Pour utiliser le module, nous avons connecté notre ESP32 à ce module en liaison SPI, en utilisant les broches suivantes :

- MOSI (Master Out Slave In : Écriture) : GPIO 13
- MISO (Master In Slave Out : Lecture) : GPIO 12
- SCK (Serial Clock) : GPIO 14
- CS (Chip Select) : GPIO 15
- VCC : 5V
- GND : GND

Nous utiliserons le module HSPI de l'ESP32 pour la communication SPI, qui permet des vitesses de transfert élevées. Voici un exemple de communication en SPI pour décrire le moyen de communication entre l'ESP32 et la carte SD.

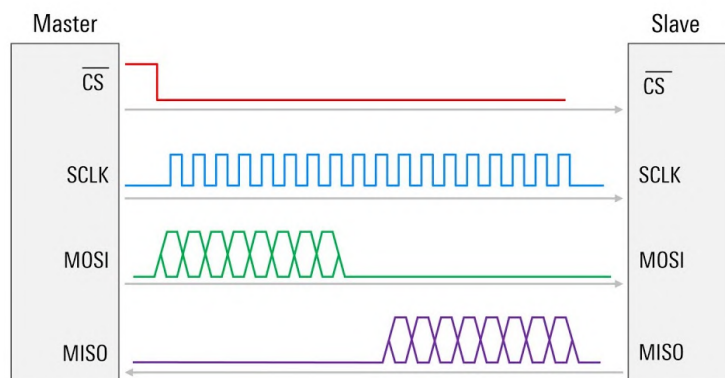


FIGURE 2.7 – Module de carte SD utilisé pour le projet

Pour utiliser ce module, une librairie toute faite a déjà été développée, qu'il a suffi d'implémenter dans notre projet pour pouvoir utiliser la carte SD. Cette librairie est la librairie `SD.h` qui est une librairie standard pour la gestion des cartes SD avec les

microcontrôleurs Arduino et ESP32. Elle offre des fonctions pour initialiser la carte SD, lire et écrire des fichiers, et gérer les répertoires. Nous l'avons utilisée pour créer notre propre bibliothèque visant à simplifier l'utilisation de cette carte, notamment pour le WPBis qui est dédié à la gestion de la transmission des données collectées.

Tests et Validation

Pour la phase de test, nous avons juste réalisé un test afin de savoir si nous pouvions écrire un fichier texte sur la carte SD, en donnant comme nom de fichier les métadonnées (comme température, humidité, etc), afin de simuler un buffer de réception d'images avec les données correspondantes.

Voici un exemple de code utilisé pour ce test :

```
1 #include <SD_PIEGE.h>
2
3 void setup()
4 {
5     initSD();
6
7     // Temperature_Humidite_Latitude_Longitude_Heure_Minute_Seconde
8     String filename = "20_60_47-2833012_1-5152623_10-30-30.txt";
9
10    File file = SD.open(filename, FILE_WRITE);
11
12    // Exemple de donnees JPEG
13    uint8_t image[] = {0xFF, 0xD8, 0xFF, 0xE0, 0x00, 0x10, 0x4A, 0x46};
14
15    file.write(image, sizeof(image));
16    file.close();
17 }
```

Listing 2.1 – Exemple utilisation module SD

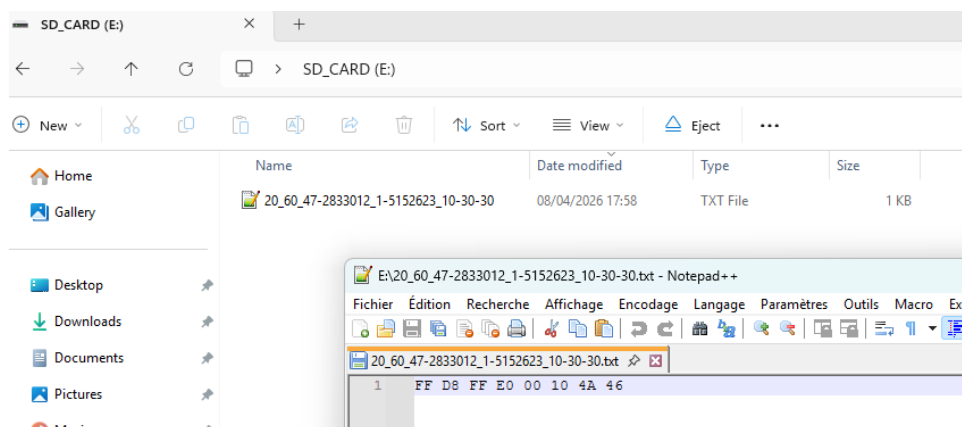


FIGURE 2.8 – Résultat du test d'écriture sur la carte SD

2.2.4 Tâche 2 : Hors WP : Gestion de la luminosité pour WP 1.3

Rappel des objectifs

Pour cette partie nous avons décidé de nous séparer de notre WP afin d'aider le WP 1.3 qui est dédié à la gestion de la caméra, notamment pour la partie de la luminosité. En effet, la luminosité est un facteur important pour la prise de vue des images, car d'après notre cahier des charges, nous voulions prendre des photographies de jour comme de nuit.

Avoir la possibilité de connaître la luminosité de l'environnement nous permettrait de pouvoir allumer une source de lumière (comme des LED infrarouges) pour pouvoir capturer des éléments de nuit.

Solution envisagée

Pour cela nous avons décidé d'ajouter un capteur de luminosité (une photorésistance (LDR)) afin de capturer la luminosité de l'environnement. Ce capteur est un composant électronique qui change de résistance en fonction de la quantité de lumière. C'est un capteur très simple à mettre en œuvre et à interfacer avec un microcontrôleur comme l'ESP32. En mesurant la résistance de la photorésistance, nous pouvons déterminer la luminosité ambiante et ajuster le comportement de notre système.

Pour connaître la résistance de ce capteur, nous utilisons un montage de diviseur de tension, qui nous permet de convertir la résistance variable de la photorésistance en une tension mesurable par l'ESP32. Voici un schéma de ce montage :

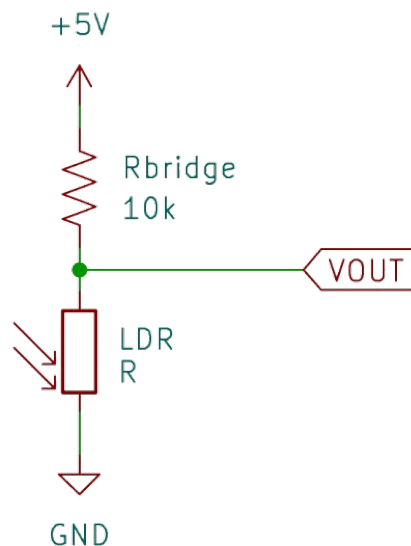


FIGURE 2.9 – Schéma du montage de diviseur de tension pour la photorésistance

Dans ce montage, la photorésistance (LDR) est connectée en série avec une résistance fixe (R_1). Lorsque la lumière frappe la photorésistance, sa résistance change, ce qui modifie la tension de sortie (V_{out}) du diviseur de tension. En mesurant cette tension de sortie avec l'ESP32, nous pouvons calculer la résistance de la photorésistance et ainsi

déterminer la luminosité ambiante.

$$R_{LDR} = ((R_{serie} * 3.3) / Tension_{Vout}) - R_{serie};$$

À partir de la documentation technique de la LDR, nous pouvons facilement retrouver la valeur de la luminosité (en LUX) à partir de la résistance, en utilisant la courbe suivante :

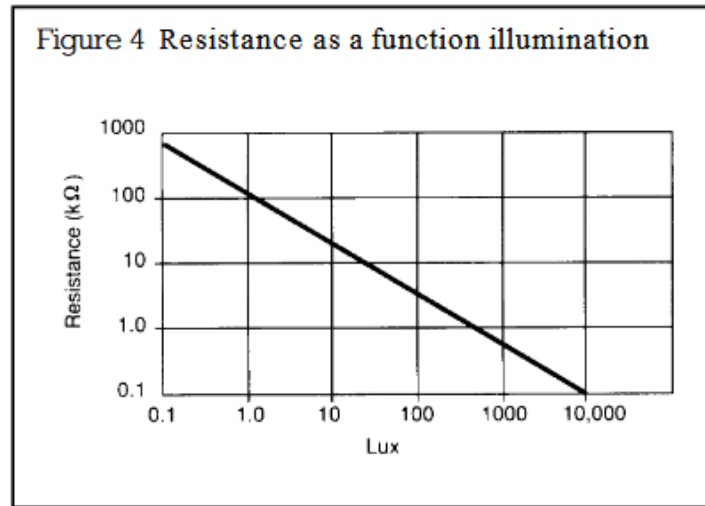


FIGURE 2.10 – Courbe de résistance en fonction de la luminosité pour la photorésistance

$$Lux = \left(\frac{1000}{Resistance} \right)^{1.4}$$

Tests et Validation

Pour la phase de test de cette LDR, nous avons tout simplement effectué plusieurs mesures avec des taux de luminosité différents, en utilisant une lampe, un chiffon, un rideau, etc. pour faire varier la luminosité, et en mesurant la tension de sortie du diviseur de tension.

Pour lire la valeur numérique nous avons tout simplement utilisé la fonction `analogRead()` de l'ESP32, qui nous permet de lire la tension de sortie.

Voici un exemple de code utilisé pour ce test :

```

1 #define LDR_PIN 34 // Broche analogique pour la LDR
2 #define RESISTOR_BRIDGE 10000 // Resistance diviseur de tension
3
4 void setup()
5 {
6     Serial.begin(115200);
7 }
8
9 void loop()
```

```

10 {
11     // Calcul de la tension ( ADC 12 bits )
12     ldrValue = (analogRead(LDR_PIN) * 3.3) / (1.0 * 4095.0);
13
14     // Calcul de la resistance
15     float resistance = (( RESISTOR_BRIDGE * 3.3 ) / ldrValue ) -
16         RESISTOR_BRIDGE;
17
18     // Calcul de la luminosite en LUX
19     float lux = pow((1000 / resistance), 1.4);
20
21     Serial.print("LDR Value: ");
22     Serial.println(ldrValue); // Affichage de la valeur lue
23
24     Serial.print("Resistance: ");
25     Serial.println(resistance); // Affichage de la resistance calculee
26
27     Serial.print("Luminosite (LUX): ");
28     Serial.println(lux); // Affichage de la luminosite calculee
29
30     delay(1000); // Attente d'une seconde avant la prochaine lecture
31 }

```

Listing 2.2 – Exemple utilisation LDR

Avec ce code nous avons pu vérifier que notre capteur LDR fonctionnait comme un luxmètre, en affichant la valeur de luminosité en LUX dans le moniteur série de l'ESP32. Nous avons en même temps vérifié à l'aide d'un luxmètre (téléphone) que les valeurs de luminosité affichées étaient cohérentes avec les mesures réelles. Malheureusement le téléphone utilisant une application, celui-ci n'était pas très précis.

Mais dans l'ensemble, les résultats obtenus étaient cohérents, nous avons obtenu :

	Theorie (Calcul)			Numerique (Code)			Pratique (Multimetre + Telephone)		
	Tension	Resistance	Lux	Tension	Resistance	Lux	Tension	Resistance	Lux (PAS FIABLE)
Noir (Torchon)	0	1M	0,1	0	1M	1	0	17,16M	0
Sombre				0,69	5668	2636	0,7	5300	100
Gris				2	996	5413	2,13	890	500
Lumiere				2,71	329	7312	2,8	237	1900
Flash	3,3	0,1	10000	3,3	0	9820	3,22	72,3	

TABLE 2.6 – Comparaison des mesures de luminosite : theorie, calcul numerique et mesures pratiques

Nous avons pour principe de construire notre propre couronne de LED infrarouges, avec ce capteur LDR, afin de pouvoir capturer des animaux durant la nuit, mais par facilité nous avons décidé d'acheter une couronne de LED infrarouges déjà faite, qui s'allume à partir d'une certaine luminosité, ce qui nous a permis de ne pas avoir à gérer l'allumage de ces LED, et de se concentrer sur d'autres aspects du projet.

Cette couronne de LED a une détection de luminosité automatique, si le seuil est assez élevé, alors les LED s'allumeront, et si le seuil est bas alors les LED seront éteintes, ce principe est aussi géré par une photorésistance, mais celle-ci est intégrée dans la cou-

ronne de LED, ce qui nous a permis de ne pas avoir à gérer cet aspect et de simplement alimenter ce module.



Couronne de LED Infrarouge utilisée pour le projet - face avant



Couronne de LED Infrarouge utilisée pour le projet - face arrière

FIGURE 2.11 – Couronne de LED Infrarouge utilisée pour le projet

Malheureusement, avec les différents problèmes rencontrés durant le projet, nous n'avons pas pu faire de vrais tests dans la nature afin de vérifier que nous puissions capturer les animaux de nuit, mais nous avons pu faire des tests en intérieur, en nous prenant en photo dans le noir.

2.2.5 Tâche 3 : Gestion de l'horodatage et de la localisation du module

Rappel des objectifs

Pour cette tâche, nous avons décidé d'ajouter un module GPS afin de pouvoir récupérer la localisation du module, ainsi que l'heure grâce à ce module. En effet, les données d'horodatage sont nécessaires pour pouvoir obtenir des statistiques sur le taux de passage dans la zone en question, et la localisation est nécessaire dans le cas où nous aurions plusieurs modules à déployer, afin d'observer les zones de passage.

Nous avons décidé d'utiliser un module GPS personnel (dans le même cas que pour la carte SD, aucune commande n'était envisageable de la part du département ou du client), qui est un module basique destiné plutôt aux drones FPV.



PIN	PIN Name	I/O	Description
1	GND	G	Ground
2	TX	O	Serial Data Output.
3	RX	I	Serial Data Input.
4	VCC	I	DC 3.6V - 5.5V supply input, Typical: 5.0V

FIGURE 2.12 – Module GPS utilisé pour le projet

Solution envisagée - GPS

Ce module est basé sur une communication Série UART, qui est un protocole de communication simple et largement utilisé pour les modules GPS. Celui-ci envoie des données de localisation et d'horodatage au format NMEA, qui est un format standard pour les données GPS. Différentes trames NMEA contiennent différentes informations, telles que la position, la vitesse, l'heure, etc.

Par exemple, la trame la plus connue est celle de GGA, qui est :

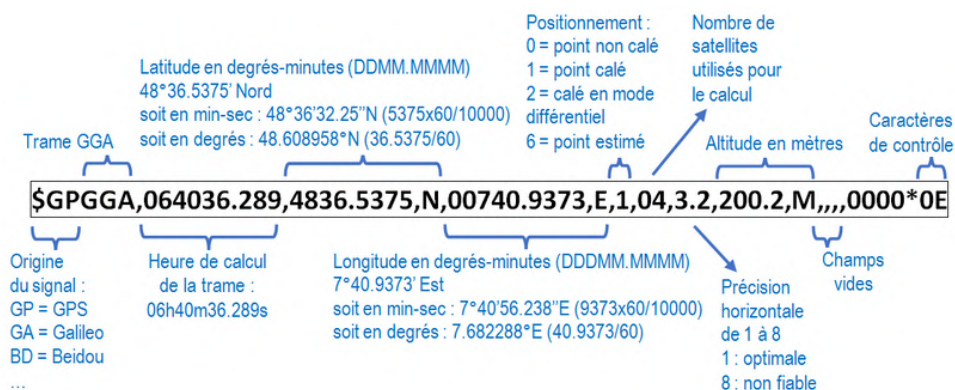


FIGURE 2.13 – Exemple de trame NMEA envoyée par le module GPS

Dans notre cas nous allons utiliser la trame GPRMC :

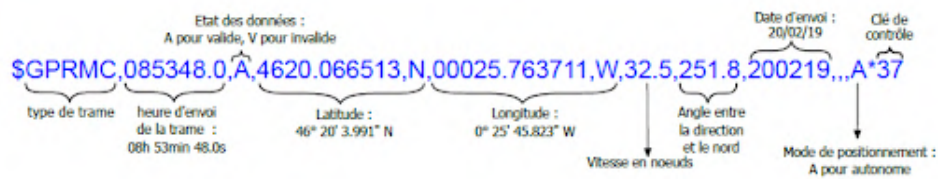


FIGURE 2.14 – Trame GPRMC utilisée pour récupérer la localisation et l'horodatage

Grâce à cette trame, nous allons pouvoir récupérer l'heure (UTC), la latitude et la longitude.

Pour l'implémentation de ce module, nous avons créé notre librairie Arduino visant à récupérer les informations que nous souhaitons à partir des informations de communication en liaison UART.

Le principe est simple : nous attendons qu'une trame complète soit arrivée, nous analysons si le caractère de début est bien `$GPRMC`, et nous analysons les caractères qui sont séparés par des virgules pour récupérer les différentes informations. À la suite de ça nous avons créé une classe en C permettant de pouvoir récupérer les valeurs directement via une simple fonction, `get()`.

Solution envisagée - RTC

Pour la partie du GPS nous avons pu voir que nous pouvions récupérer toutes les informations nécessaires à notre projet, cependant il y avait un problème : le GPS met du temps à capter la localisation car celui-ci dépend de la position des satellites, et dans le cas où le module est dans une zone isolée, il peut mettre du temps à capter la localisation.

Une solution était d'utiliser un module RTC (Real Time Clock), celui-ci permettrait de continuer à obtenir l'heure sans pour autant avoir besoin d'utiliser le module GPS, ce qui nous permettrait de gagner du temps sur tout le cheminement de la capture à la transmission.

Un module RTC est un composant électronique qui maintient une horloge en temps réel, même lorsque le microcontrôleur est éteint ou redémarré. Voici un schéma du principe que nous voulions implémenter :

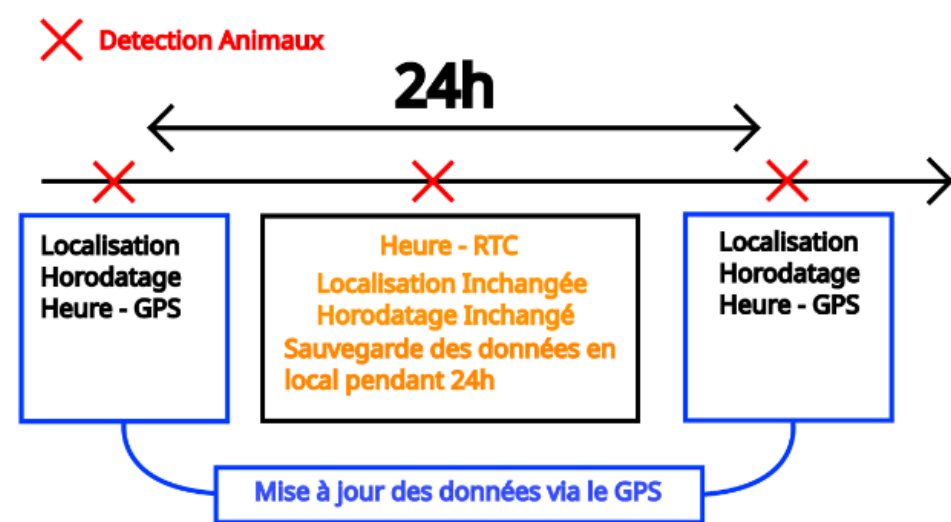


FIGURE 2.15 – Schéma du principe de fonctionnement du module RTC pour maintenir l'heure en temps réel

Pour l'implémentation de ce module, nous avons implémenté notre propre librairie qui vise à initialiser le module RTC avec les données du GPS, et ensuite, comme pour le GPS, à avoir des fonctions `get()` pour récupérer les différentes informations de date et d'heure.

Donc après l'implémentation de ce code, nous avons au total 10 données qui sont sauvegardées dans le module RTC, qui sont :

- Seconde : String
- Minute : String
- Heure : String
- Jour : String
- Mois : String
- Année : String
- Latitude : char
- Longitude : char
- activation du GPS : bool (permet de connaître l'activation de la première fois du GPS pour la synchronisation de l'heure)
- lastRefreshDay : int (permet d'enregistrer le jour auquel les métadonnées ont été rafraîchies par le GPS)

Tests et Validation

Dans un premier temps nous avons vérifié que nous recevions bien les trames NMEA du module GPS, en affichant les données reçues dans le moniteur série de l'ESP32.

```
1 void loop()
2 {
3     gps.update();
4     Serial.println("Donnees GPS
5         recues :");
6
7     Serial.print("Latitude: ");
8     Serial.println(
9         gps.getLatitude());
10    Serial.print("Longitude: ");
11    Serial.println(
12        gps.getLongitude());
13    Serial.print("Heure UTC: ");
14    Serial.print(
15        gps.getHours());
16    Serial.print(":");
17    Serial.print(
18        gps.getMinutes());
19    Serial.print(":");
20    Serial.println(
21        gps.getSeconds());
22    Serial.print("Date
23        (JJ/MM/AAAA): ");
24    Serial.print(
25        gps.getDay());
26    Serial.print("/");
27    Serial.print(
28        gps.getMonth());
29    Serial.print("/");
30    Serial.println(
31        gps.getYear());
32
33    delay(10000); // Attente 10s
34 }
```

Listing 2.3 – Exemple utilisation module GPS

À la suite de cette validation nous avons pu vérifier que le module RTC fonctionnait correctement, en affichant les données de date et d’heure dans le moniteur série de l’ESP32, et en vérifiant que l’heure était maintenue.

```
1 Donnees GPS recues :
2 Latitude: 47.2833012
3 Longitude: 1.5152623
4 Heure UTC: 10:38:32
5 Date (JJ/MM/AAAA): 09/04/2026
6
7 Donnees GPS recues :
8 Latitude: 47.2833012
9 Longitude: 1.5152623
10 Heure UTC: 10:38:42
11 Date (JJ/MM/AAAA): 09/04/2026
12
13 Donnees GPS recues :
14 Latitude: 47.2833012
15 Longitude: 1.5152623
16 Heure UTC: 10:38:52
17 Date (JJ/MM/AAAA): 09/04/2026
18
19 Donnees GPS recues :
20 Latitude: 47.2833012
21 Longitude: 1.5152623
22 Heure UTC: 10:39:02
23 Date (JJ/MM/AAAA): 09/04/2026
24
25 Donnees GPS recues :
26 Latitude: 47.2833012
27 Longitude: 1.5152623
28 Heure UTC: 10:39:12
29 Date (JJ/MM/AAAA): 09/04/2026
```

Listing 2.4 – Moniteur Série de l’ESP32

```
1 #include <Arduino.h>
2 #include "GPS.hpp"
3 #include "RTC.hpp"
4
5 GPS gps(16, 17); // RX, TX
6 RTC rtc;
7
8 void setup()
9 {
10     Serial.begin(115200);
11
12     gps.update();
13     rtc.updateFromGPS(...);
14     rtc.getDateTime();
15 }
16
17 void loop()
18 {
19     delay(10000); // Attendre
20                   10 secondes
21
22     rtc.getDateTime();
23 }
```

Listing 2.5 – Exemple utilisation module GPS + module RTC

```
1 =====
2 Donnees GPS recues :
3 Lat: 47.2833012
4 Long: 1.5152623
5 Date: 09/04/2026
6 Heure: 15:42:26
7 =====
8
9 =====
10 RTC interne uniquement :
11 Lat: 47.2833012
12 Long: 1.5152623
13 Date: 09/04/2026
14 Heure: 15:42:36
15 =====
16
17 =====
18 RTC interne uniquement :
19 Lat: 47.2833012
20 Long: 1.5152623
21 Date: 09/04/2026
22 Heure: 15:42:56
23 =====
```

Listing 2.6 – Moniteur Série de l'ESP32

Ces deux tests nous ont permis de vérifier que le module GPS nous transmettait bien les données de localisation et d'horodatage, et que le module RTC maintenait correctement l'heure en temps réel, même après la synchronisation initiale avec le GPS. Les tests ont été effectués plusieurs fois sur plusieurs heures, afin de vérifier que le module RTC suffisait à maintenir l'heure sans besoin de resynchronisation fréquente avec le GPS, ce qui est essentiel pour l'efficacité énergétique du système.

2.2.6 Tâche 4 : Définition d'un automate pour la gestion globale du module

Rappel des objectifs

Cette tâche a été réalisée une première fois, puis modifiée plusieurs fois durant le projet, afin de pouvoir s'adapter au mieux à l'évolution du projet et aux différentes contraintes rencontrées. Au départ nous avons seulement défini tous les éléments à prendre en compte et faire un déroulement d'une prise de photo et récupération des données puis la transmission.

Mais au vu des différents défis qui sont apparus, comme la gestion du GPS, et puis la gestion des LED infrarouges, nous avons dû le modifier pour faire apparaître des branches en cas de besoin / non-besoin d'allumer ces éléments.

Solution envisagée

Au départ nous étions partis sur un modèle simple : l'ESP32 devait être en veille, donc avoir une phase de réveil, de réception de données, puis de transmission :

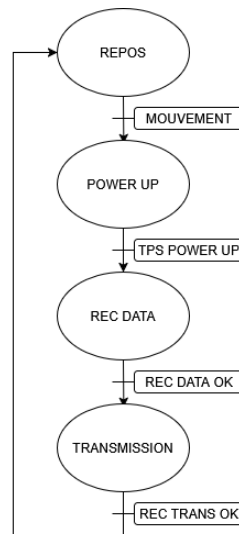


FIGURE 2.16 – Automate de gestion du module - version 1

Nous avons aussi décidé d'ajouter un module de capture de la tension de la batterie, afin de pouvoir surveiller la tension de la batterie, pouvoir faire des statistiques et aussi de permettre à l'utilisateur de savoir quand il faut changer la batterie, ou de faire des prévisions sur l'autonomie restante.

Celui-ci repose sur le même principe que la photorésistance, c'est à dire un montage de diviseur de tension, qui nous permet de convertir la tension de la batterie en une tension mesurable par l'ESP32. Nous choisissons une forte valeur de résistance pour moins consommer.

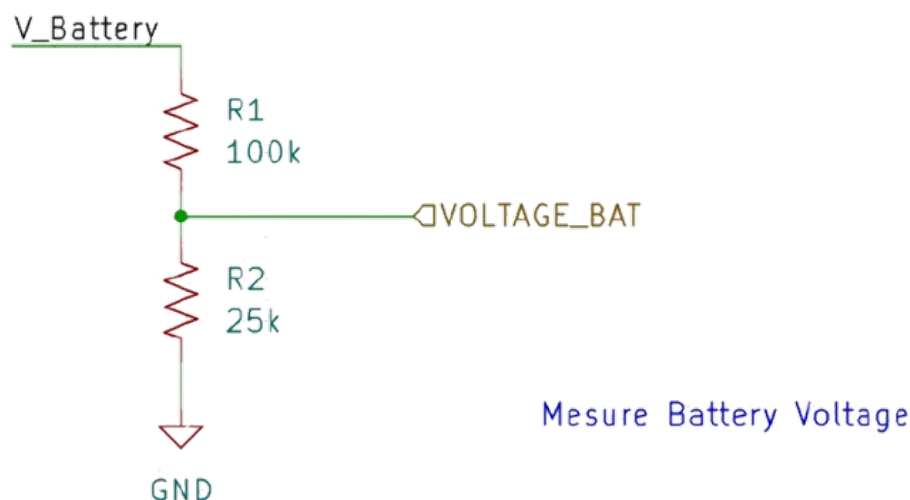


FIGURE 2.17 – Schéma du montage de diviseur de tension pour la mesure de la tension de la batterie

La deuxième résistance permet de connaître la division de ce montage, nous avons décidé

de diviser par 4 notre système, car la tension de la batterie sera proche de 12V, celle-ci ne peut pas être directement mesurée par l'ESP32 qui ne supporte que 5V. En divisant par 4 nous nous assurons que la tension mesurée ne dépassera pas 3V, ce qui est sécuritaire pour l'ESP32.

$$V_{out} = \frac{R2}{R1 + R2} \times V_{in}$$

La lecture de notre tension se fait grâce à un ADC (Analog to Digital Converter), qui convertit la tension analogique en une valeur numérique que l'ESP32 peut traiter.

Voici la version finale que nous avons mise en place pour la finalité de notre projet :

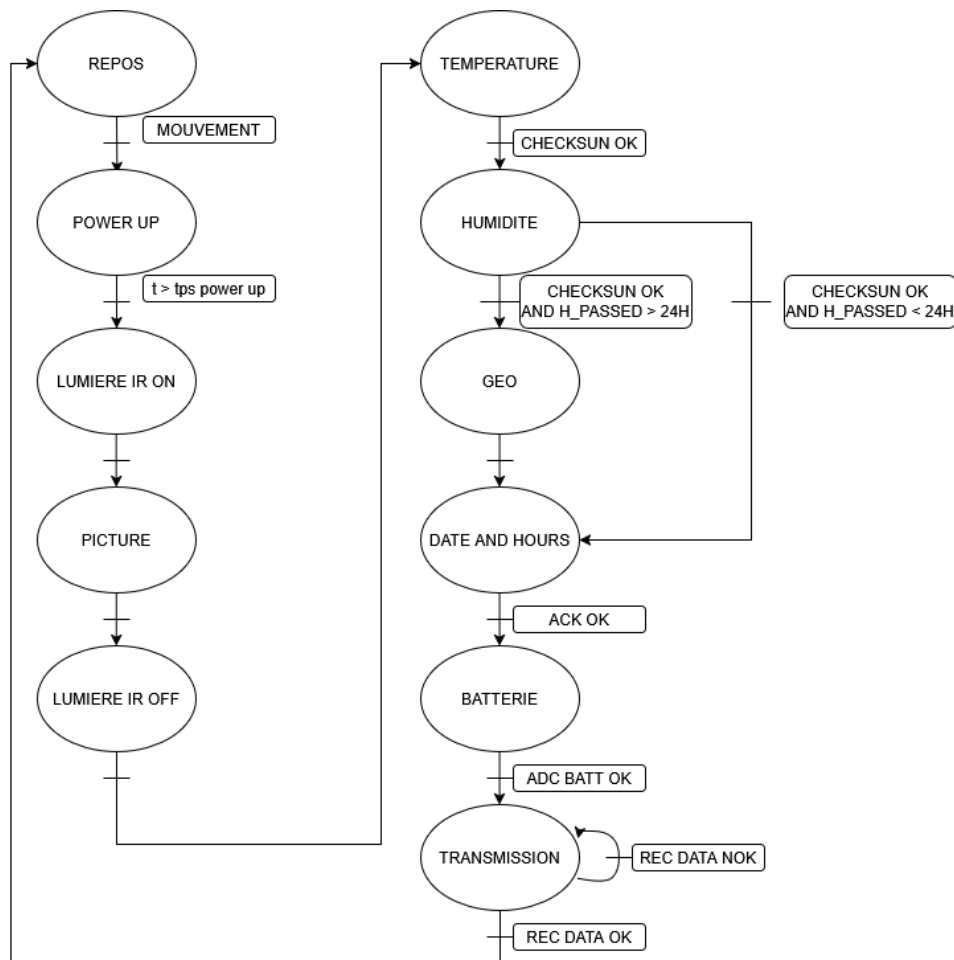


FIGURE 2.18 – Automate de gestion du module - version finale

La description de cet automate n'est pas finie, car il nous manque les étapes de transition, mais dans l'ensemble nous avons la structure de base de notre automate, qui nous permettra de gérer les différentes étapes de notre projet.

Nous observons toutes les étapes en commençant par le réveil de notre système par un mouvement. Une phase de réveil est nécessaire pour que le système puisse s'initialiser. Tout de suite après, nous allumons la couronne de LED, puis prenons une photo, puis

réalisons l'extinction de celle-ci. Nous avons effectué cela car notre couronne est en 12V et celle-ci consomme énormément d'énergie. À la suite de la prise de photo, nous prenons nos données de température, d'humidité, et de localisation et d'horodatage, puis nous éteignons le GPS pour économiser de l'énergie, et ensuite nous transmettons les données collectées. Le GPS est activé seulement une fois toutes les 24h pour économiser l'énergie nécessaire à la recherche.

2.2.7 Tâche 5 : Gestion de l'alimentation du module

Rappel des objectifs

Dans cette tâche, j'ai dû réaliser un bilan des meilleurs moyens d'alimenter les modules afin de moins consommer d'énergie et d'avoir une autonomie plus longue. En effet notre projet est destiné à être utilisé dans des zones isolées, et il est donc essentiel d'avoir une autonomie suffisante pour que le système puisse fonctionner pendant une longue période sans besoin de maintenance fréquente.

Solution envisagée

Pour la première étape il faut pouvoir mettre en veille les modules afin de pouvoir mieux gérer l'énergie : Commençons par l'ESP32, celui-ci offre plusieurs modes de veille qui permettent de réduire considérablement la consommation d'énergie lorsque le système n'est pas actif.

Voici les différents modes de veille disponibles pour l'ESP32 :

Mode	Consommation typique	Ce qui est éteint	Ce qui reste actif	Comment se réveiller
Actif	~ 40 mA – 240 mA	Rien	Tout	-
Modem Sleep	~ 20 mA	WiFi / BT / Radio	CPU, mémoire, périphériques	Timer, interruption
Light Sleep	~ 0,8 mA	CPU, PLL, radio	RTC, mémoire RTC, ULP co-processor	Timer, interruption (GPIO, UART, etc.)
Deep Sleep	~ 10 μ A – 150 μ A	Presque tout : CPU, WiFi, BT, RAM principale	RTC, mémoire RTC lente (8 KB), ULP co-processor	Timer, épingles RTC, GPIO, touch
Hibernation	~ 2,5 μ A – 5 μ A	Tout, même le RTC est ralenti	Mémoire RTC lente (8 KB)	Timer RTC externe uniquement

TABLE 2.8 – Principaux modes de veille de l'ESP32

Dans notre projet nous avons pour objectif de consommer le moins possible mais aussi de pouvoir utiliser le module RTC pour la mémoire de la localisation et l'horodatage, ce qui nous a permis de choisir le mode de veille "Deep Sleep", qui offre une consommation très faible tout en permettant au RTC de continuer à fonctionner, ce qui est essentiel

pour notre projet.

Plusieurs autres méthodes sont envisagées pour pouvoir réduire la consommation d'énergie, comme l'arrêt de différents périphériques de ce microcontrôleur, ou encore la mise en place de pull-down sur les GPIO pour éviter les fuites de courant, mais après la phase de test nous nous sommes rendu compte que, à part le Wi-Fi et le Bluetooth, le reste est négligeable en termes de consommation.

Ce qui a le plus d'impact sur la consommation, c'est surtout de réduire la fréquence d'utilisation du processeur du microcontrôleur, mais cela aura un impact sur le temps de traitement et la rapidité d'exécution de notre code.

Tests et Validation

Nous pouvons observer tout d'abord, afin d'avoir une idée de la consommation de base, un code vierge injecté dans l'ESP32 :

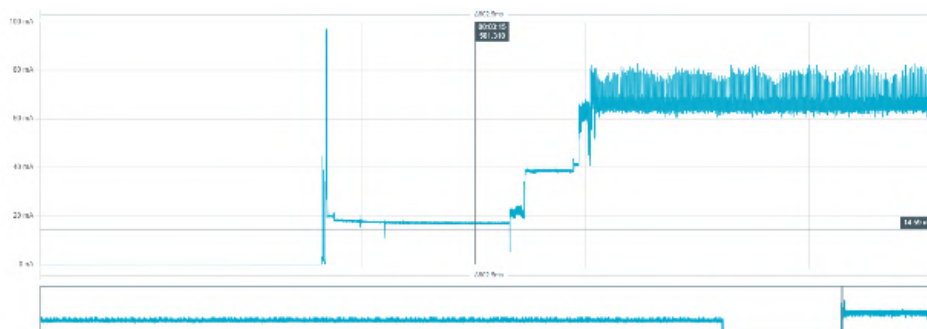


FIGURE 2.19 – Consommation de base de l’ESP32 avec un code vierge

Celui-ci montre une consommation de 80mA environ.

Passons maintenant à la réduction de la fréquence de notre microcontrôleur :

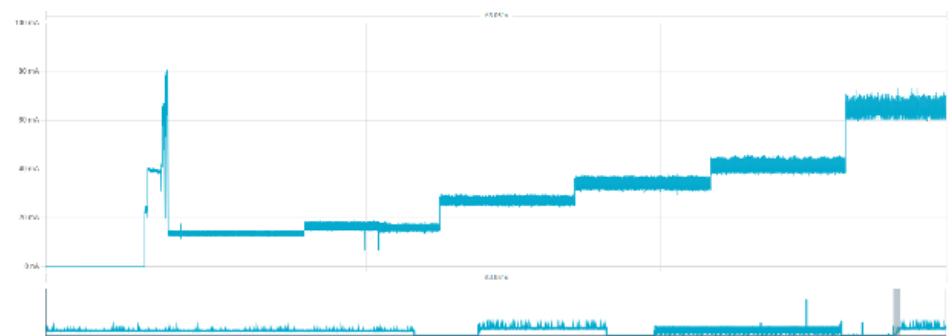


FIGURE 2.20 – Consommation de l’ESP32 en fonction de la fréquence du processeur

Dans ce graphe nous observons différents paliers qui correspondent à différentes fréquences du processeur, en commençant par 20, 40, 80, 160 et 240 MHz. Déjà nous avons une nette différence entre toutes ces fréquences.

Continuons avec le mode de veille : les modes Deep :

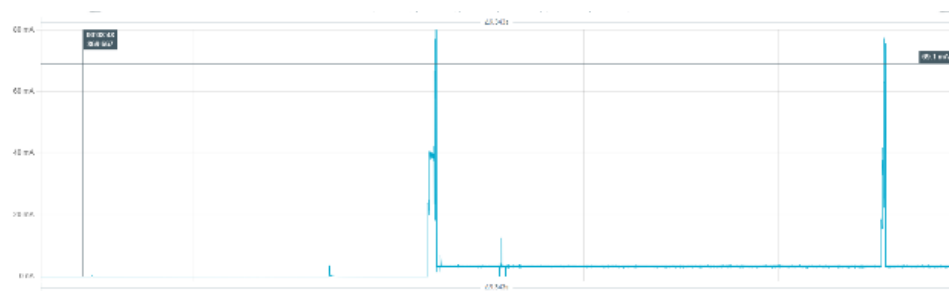


FIGURE 2.21 – Consommation de l'ESP32 en fonction du mode de veille

La consommation en mode Deep Sleep est de l'ordre de 5mA en moyenne, ce qui sera un avantage pour notre projet.

Pour aller plus loin dans nos tests nous avons testé la consommation en mode Deep Sleep, avec réveil par une simulation de mouvement, donc en envoyant un signal sur un GPIO (simulant le PIR).

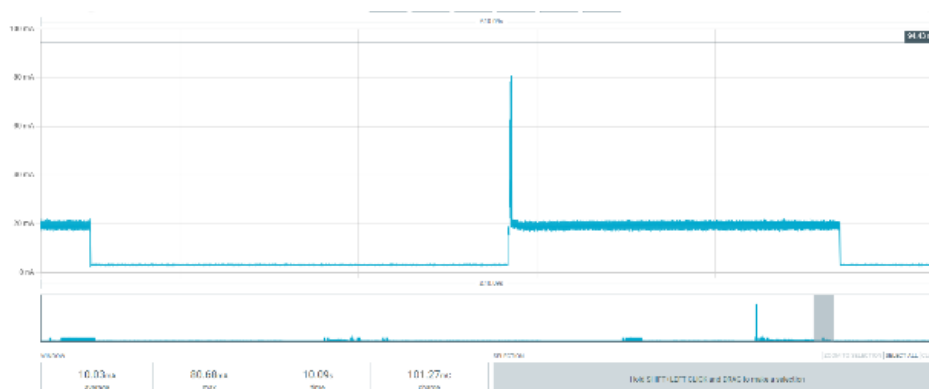


FIGURE 2.22 – Consommation de l’ESP32 en mode Deep Sleep avec réveil par signal GPIO (simulant le PIR)

Celui-ci montre un système concluant, avec une faible consommation. Mais nous observons dans tous nos graphes un problème majeur, c'est un pic au démarrage, celui-ci correspond à l'initialisation de tous les périphériques de base de l'ESP32.

Voici un exemple de code utilisé pour le test du mode de veille :

```
1 #define LED_PIN 2
2 #define WAKE_PIN 33
3
4 void actionAuReveil()
5 {
6     // Se declenche UNE SEULE FOIS par front montant
7     digitalWrite(LED_PIN, HIGH);
8     delay(150);
9     digitalWrite(LED_PIN, LOW);
10    delay(150);
11 }
12
13 void setup()
```

```
14 {
15     setCpuFrequencyMhz(80);
16
17     pinMode(LED_PIN, OUTPUT);
18     pinMode(WAKE_PIN, INPUT_PULLDOWN);
19
20     digitalWrite(LED_PIN, LOW);
21
22     // Cause de reveil
23     esp_sleep_wakeup_cause_t cause = esp_sleep_get_wakeup_cause();
24
25     if (cause == ESP_SLEEP_WAKEUP_EXT0)
26     {
27         // Execute l'action UNE seule fois
28         actionAuReveil();
29         while (digitalRead(WAKE_PIN) == HIGH) {}
30     }
31
32     // Rearmer le front montant UNIQUEMENT quand la pin est basse
33     esp_sleep_enable_ext0_wakeup((gpio_num_t)WAKE_PIN, 1);
34
35     esp_deep_sleep_start();
36 }
37
38 void loop() {}
```

Listing 2.7 – Exemple de code pour le test du mode de veille

2.2.8 Tâche 6 : Intégration des modules

Rappel des objectifs

Dans cette phase nous avons eu pour objectif d’assembler nos modules de tout le WP1, celui-ci correspond à notre premier prototype de notre système. Il s’agit de rassembler le travail effectué par chacun, notamment sur la partie de récupération d’images et de métadonnées, la partie de stockage sur la carte SD, la partie de localisation et d’horodatage, et la partie transmission.

Solution envisagée

Pour cela, nous avons décidé que chacun d’entre nous effectuerait une bibliothèque pour que je puisse simplement appeler les fonctions telles que l’automate défini précédemment.

Donc pour résumer la partie de notre système, le WP possède un total de 7 bibliothèques, qui sont :

- CAM.hpp : bibliothèque de gestion de la caméra, qui permet de prendre des photos
- GPS.hpp : bibliothèque de gestion du module GPS, qui permet de récupérer la localisation et l’horodatage
- RTC.hpp : bibliothèque de gestion du module RTC, qui permet de maintenir l’heure en temps réel

- SD.hpp : bibliothèque de gestion de la carte SD, qui permet de stocker les données collectées
- SENDWIFI.hpp : bibliothèque de gestion de la transmission des données en Wi-Fi
- BAT.hpp : bibliothèque de récupération de la tension de la batterie
- CONFIG.h : fichier de configuration du projet, qui contient les différentes constantes et paramètres du projet

Le code étant conséquent, il sera disponible en annexe. :code_{prototype}

Tests et Validation

Au départ pour notre système nous avons comme objectif de faire 10 photographies et de les sauvegarder sur la carte SD, avec les métadonnées de localisation et d'horodatage, ainsi que les données de température et d'humidité.

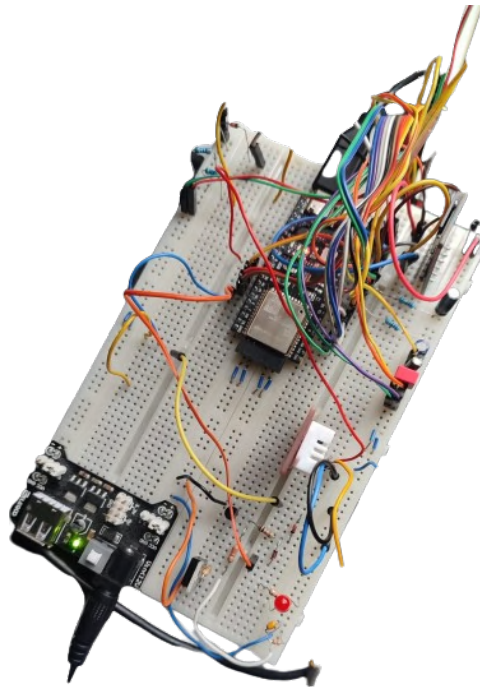


FIGURE 2.23 – Prototype du système avec tous les modules intégrés



Photographie Téléphone



Photographie Caméra

FIGURE 2.24 – Différence entre la photographie de notre Caméra et de notre Téléphone



Photographie intérieure



Photographie Humain

FIGURE 2.25 – Autre section de notre architecture

Cependant notre équipe a été assez performante, et à peine la réalisation de ce prototype terminée, nous avons pu faire des tests de transmission en Wi-Fi et de réception sur le serveur de stockage distant.

Un problème majeur est survenu lors de nos phases de test, c'était tout simplement la

résolution de nos photos. De base la bibliothèque de la caméra nous permettait de choisir l'emplacement du buffer de l'image, mais malheureusement notre ESP32 personnel n'était pas assez puissant et nous étions obligés de stocker l'image dans la DRAM (stockage principal de l'ESP32), qui est limitée en termes de capacité, et qui ne nous permettait pas de stocker des images de haute résolution. De plus nous ne pouvions pas utiliser la carte SD pour stocker le buffer car le moyen de récupération de l'image (Méthode DMA (Direct Memory Access)) n'est pas compatible avec la carte SD.

Pour faire face à ce problème nous avons donc décidé de regarder la bibliothèque en détail, et nous avons trouvé une solution adaptée à notre problème, qui a été de prendre un ESP32 avec une PSRAM. Une PSRAM (pseudo-static RAM) est une mémoire externe qui peut être utilisée pour stocker des données, et qui offre une capacité plus grande que la DRAM interne de l'ESP32 ; celle-ci est directement connectée à un périphérique de l'ESP32 (I2S), ce qui nous a permis de stocker des images de haute résolution sans problème de stockage.

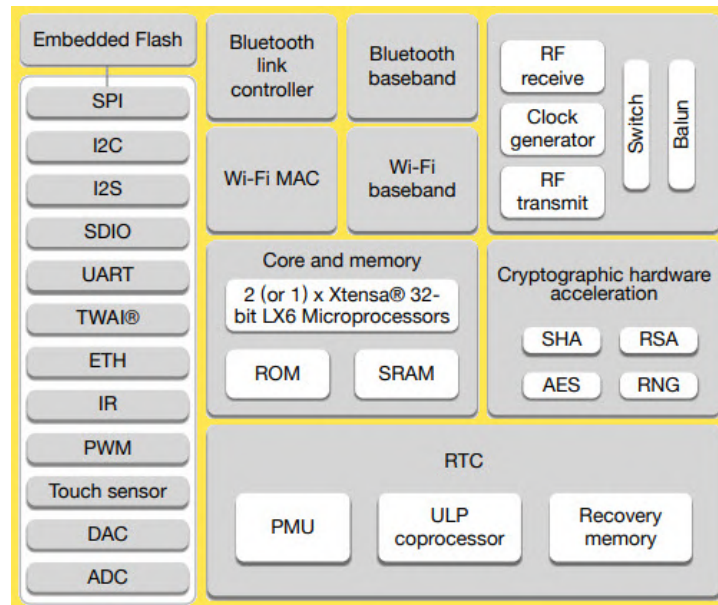


FIGURE 2.26 – Schéma de l'architecture du système avec l'ESP32 et la PSRAM pour le stockage des images

Donc nous sommes partis sur un modèle plus haut de gamme de l'ESP32, qui est l'ESP32-WROVER, qui intègre une PSRAM de 8MB. Une amélioration ne vient pas toute seule, ce modèle possède aussi plus de pins GPIO que notre ancien modèle qui arrivait à saturation.

2.2.9 Tâche 7 : Développement d'une architecture électrique

Rappel des objectifs

Dans cette section il s'agit d'effectuer une étude de notre schéma réalisé grâce au prototype, car celui-ci étant concluant, il nous permet de construire notre schéma électrique,

qui nous permettra de construire potentiellement un PCB à l’avenir, ou de faire une meilleure gestion de l’alimentation et de la consommation de notre système.

Solution envisagée

Dans un premier temps, nous recopions juste notre système grâce au logiciel de CAO, **Kicad**. Pour que cela soit plus parlant et mieux en terme de gestion de projet, nous mettons des blocs pour que chaque partie soit clairement identifiée.

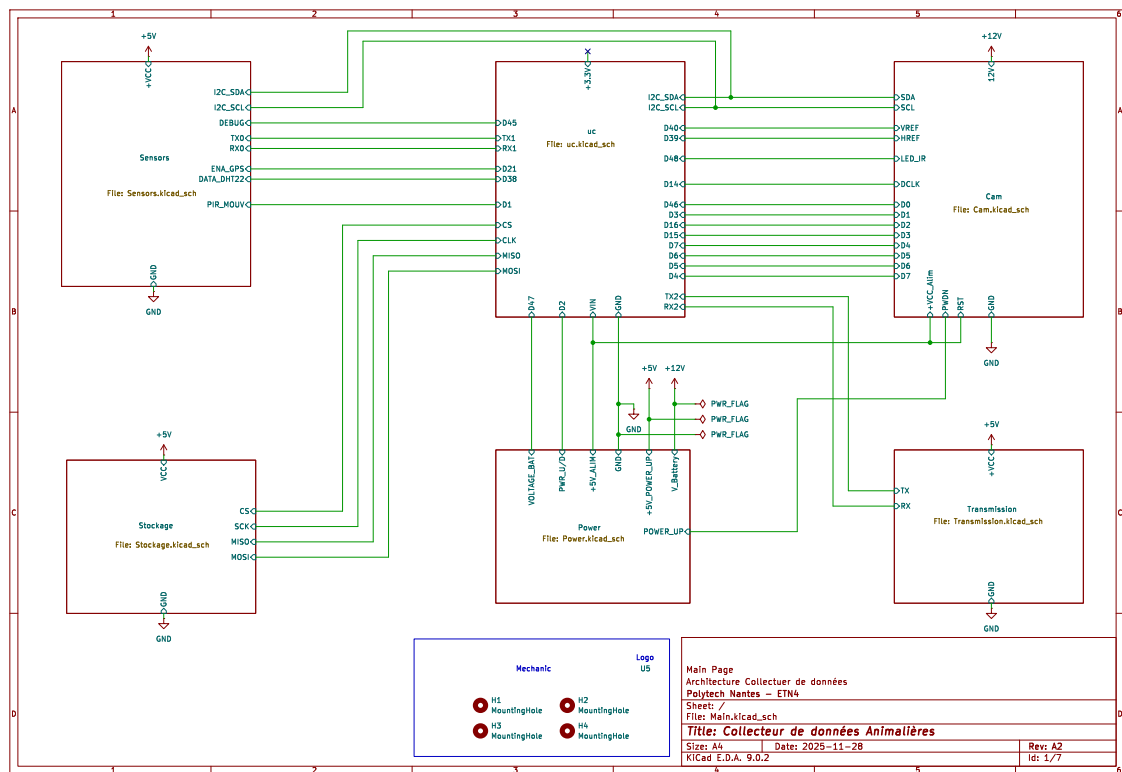


FIGURE 2.27 – Schéma électrique global du système réalisé avec Kicad

Celui-ci montre notre WP1 dans son ensemble avec les différents modules. Commençons par notre partie principale qui est notre microcontrôleur. Celle-ci montre notre microcontrôleur avec toutes les pins utilisables. Nous ajoutons des pull-up sur les signaux I2C et des condensateurs de découplage pour éviter les problèmes de bruit et de stabilité du système. Une diode est rajoutée, car la pin VIN est aussi la pin de sortie de tension du câble de programmation, et cela nous permet d’éviter les problèmes de court-circuit lors de la programmation du module.

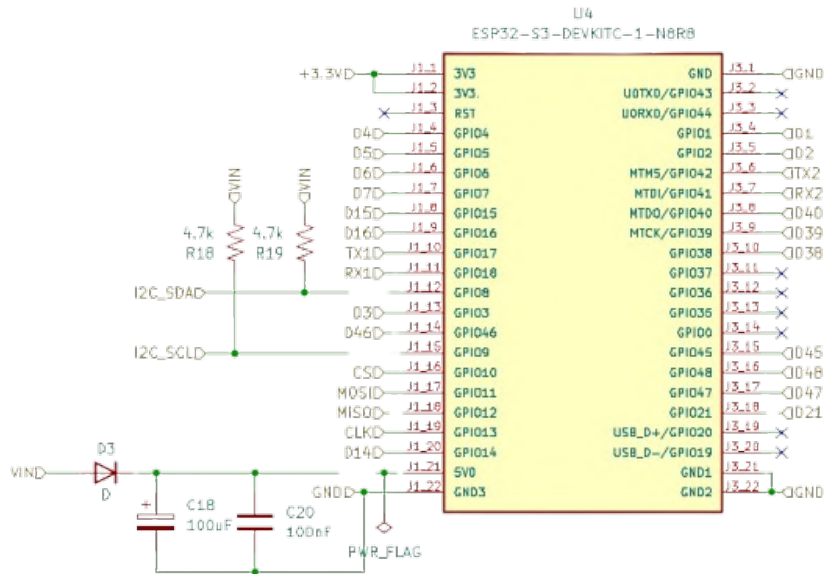


FIGURE 2.28 – Schéma de la partie microcontrôleur du système réalisé avec Kicad

Dans un second cas nous prenons la partie capteur, nous avons au total 3 capteurs : le GPS, le module DHT22 (température et humidité) et le PIR. Nous avons décidé d'ajouter des connecteurs supplémentaires, comme un double connecteur pour notre module personnel mais aussi pour un autre module GPS que nous avons sélectionné à commander. Nous avons aussi ajouté un transistor MOSFET pour pouvoir contrôler son alimentation.

Nous avons aussi rajouté une LED de DEBUG, qui nous permettra de faire du débogage visuel de notre système, en indiquant par exemple l'état du système ou les différentes étapes de notre automate.

Et enfin d'autres connecteurs comme deux connecteurs Grove (pour les modules I2C) et un connecteur pour un autre capteur de température, humidité et pression (BME280) que nous avons sélectionné à commander. Ce capteur de pression et de qualité de l'air était dans la commande initiale, mais n'est pas arrivé à temps pour les tests. Il nous permettra d'avoir des données supplémentaires sur les conditions environnementales de la zone.

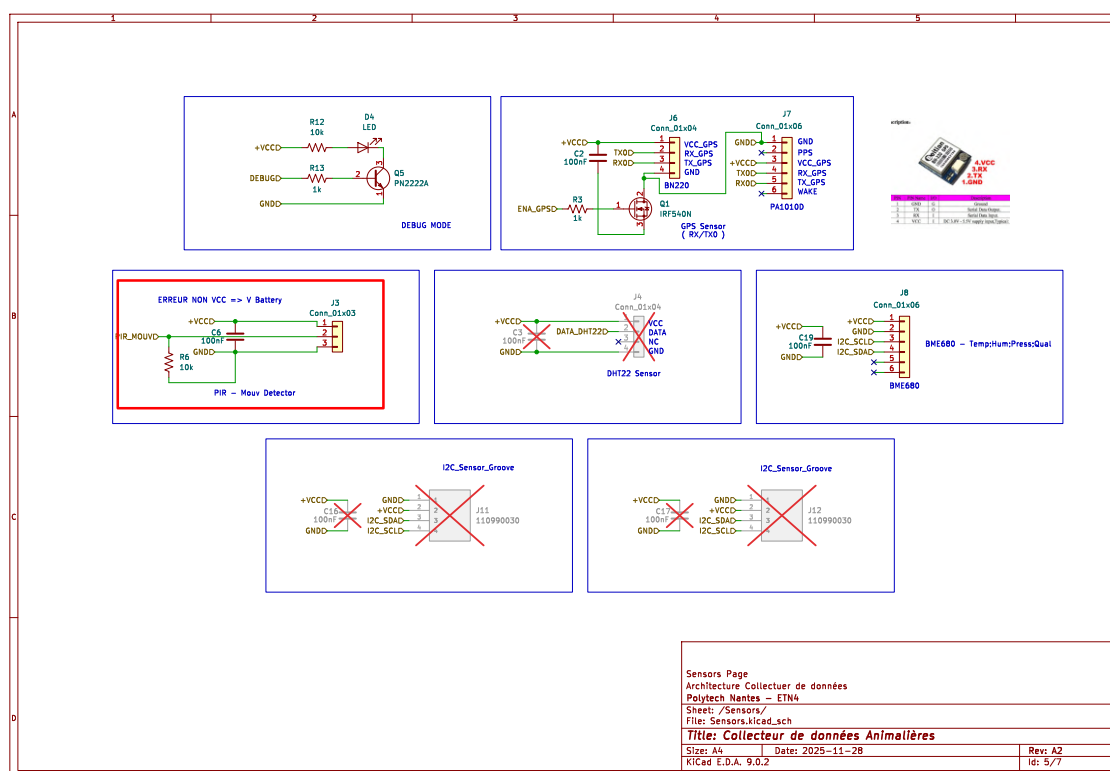


FIGURE 2.29 – Schéma de la partie capteurs du système réalisé avec Kicad

Passons à la partie caméra, qui quant à elle possède un connecteur pour la caméra OV2640, mais aussi un connecteur pour la couronne de LED infrarouges. Nous avons aussi rajouté un transistor MOSFET pour pouvoir contrôler son alimentation.

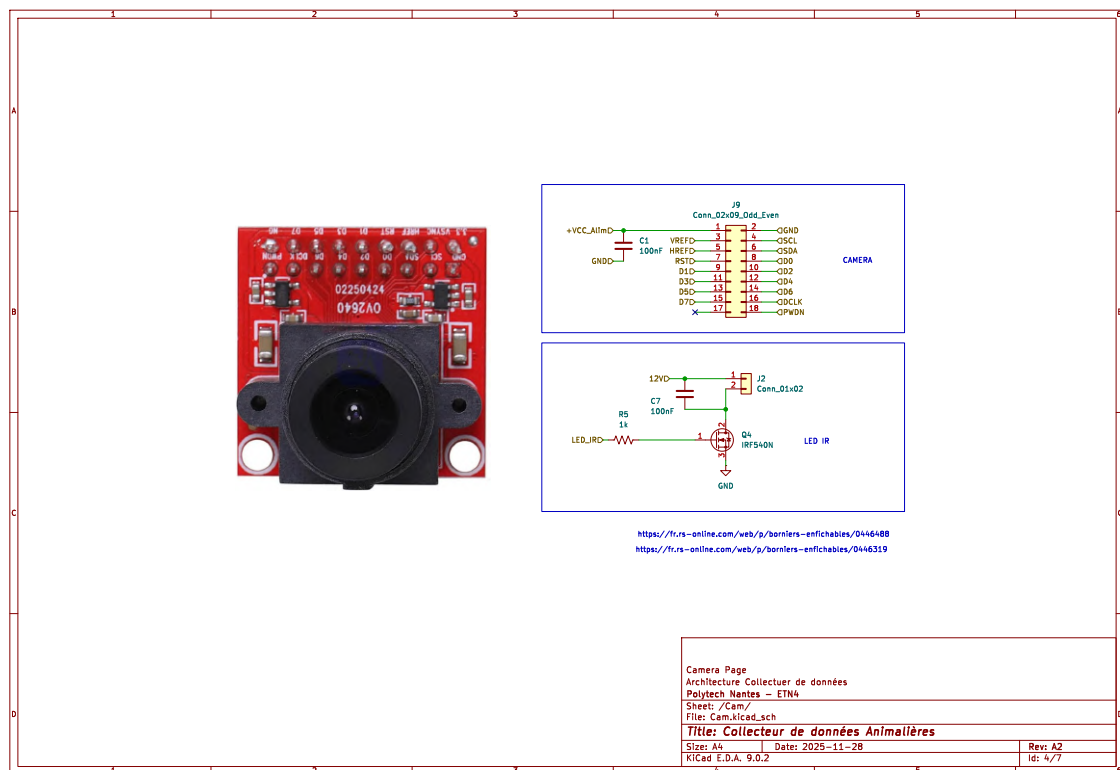


FIGURE 2.30 – Schéma de la partie caméra du système réalisé avec Kicad

Nous avons ensuite les connecteurs pour le module de carte SD ainsi que le futur module de carte SIM pour une transmission en 4G :

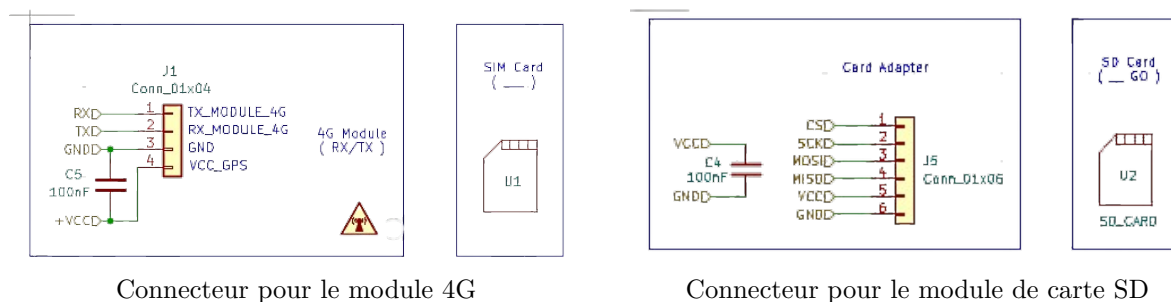


FIGURE 2.31 – Autre section de notre architecture

Enfin pour finir nous avons la partie d'alimentation, celle-ci nous a donné un peu plus de réflexion étant donné que notre système fonctionnait à 12V, mais que notre microcontrôleur et les autres modules fonctionnaient à 5V, nous avons donc décidé d'utiliser un régulateur de tension pour abaisser la tension de 12V à 5V.

De plus il a fallu réfléchir à un moyen de pouvoir avoir une alimentation qui alimente notre ESP32 et notre PIR tout le temps afin de détecter ou non, et une autre activable seulement lors de la prise de photo pour alimenter la couronne de LED et la caméra, afin d'économiser de l'énergie, nous avons donc décidé d'utiliser un transistor MOSFET pour pouvoir contrôler l'alimentation de ces éléments.

Le montage suivant est réalisé à l'aide d'un transistor MOSFET P-Channel, qui agit comme un relais en reliant le 5V du régulateur avec les différents modules.

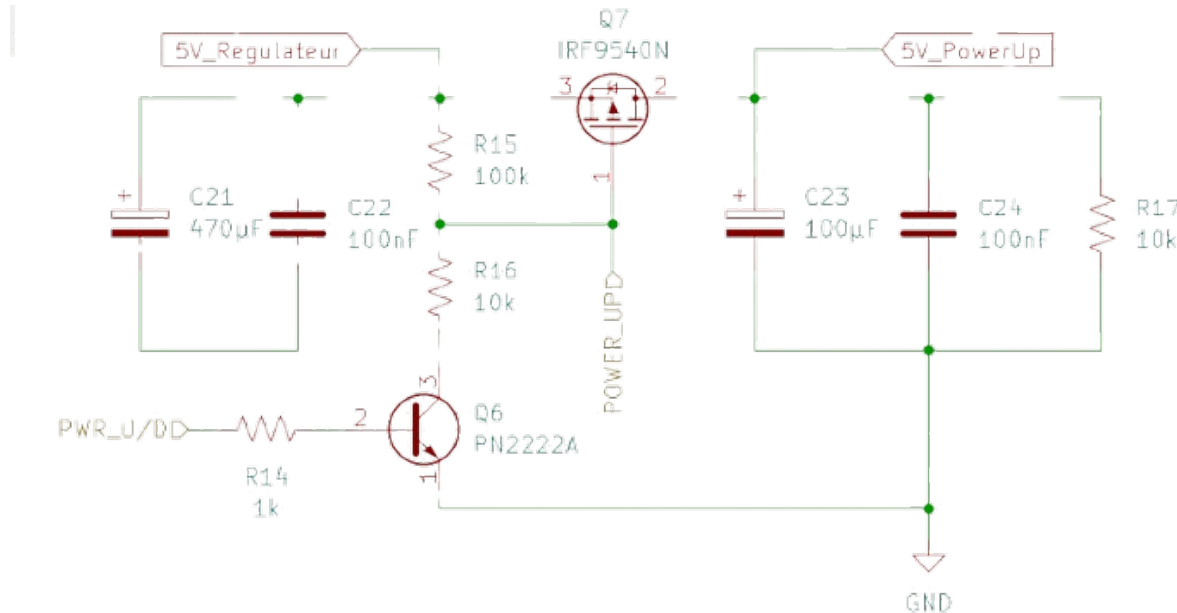


FIGURE 2.32 – Schéma du montage de contrôle d'alimentation avec un transistor MOSFET P-Channel

Dans ce montage si une tension à la sortie du régulateur est présente, alors le transistor est éteint et ne laisse pas passer le courant, mais lorsque la pin de contrôle du transistor est mise à LOW, alors le transistor s'active et laisse passer le courant, ce qui permet d'alimenter les modules connectés à la sortie du transistor.

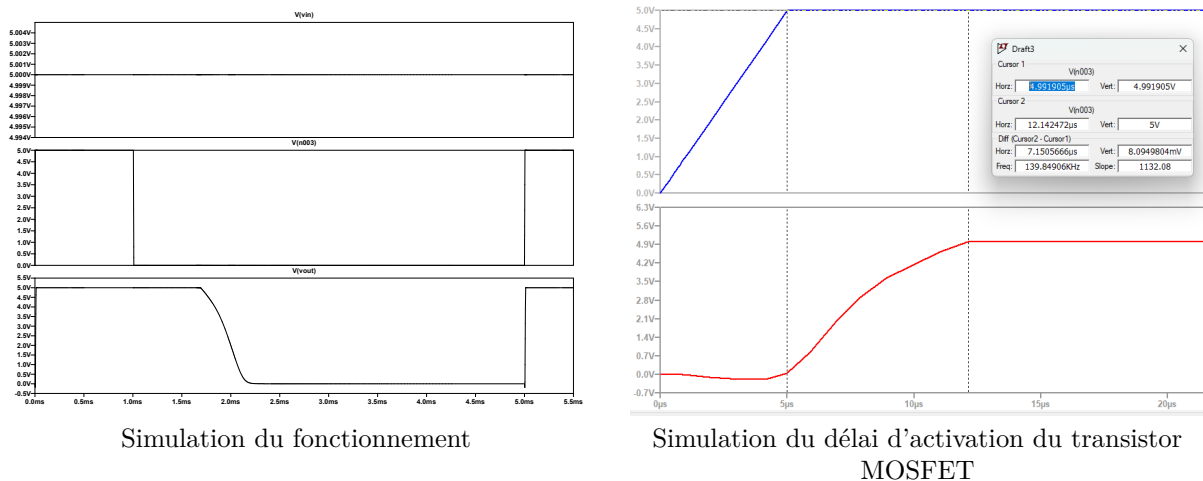


FIGURE 2.33 – Simulation du fonctionnement du montage de contrôle d'alimentation avec un transistor MOSFET P-Channel

Nous avons aussi observé le délai d'activation entre nos deux tensions qui est de 10µs, ce qui est largement suffisant pour notre projet.

Pour résumer notre schéma d'alimentation, voici un tableau récapitulatif de notre architecture d'alimentation :

12V	5V – Sortie Régulateur	5V – Power Up
Couronne LED	ESP32	DHT22
Régulateur	PIR	Carte SD
	Caméra (Mode Veille)	GPS
		4G
		DEBUG
		Autre Capteur

TABLE 2.10 – Répartition des éléments selon les différentes alimentations du système

Et voici le schéma d'alimentation de notre système complet, celui-ci possède un régulateur 12V vers 5V, de chez Microchip (Demande du client), avec le système de power up / down, et le pont diviseur de notre système de mesure de la tension de la batterie :

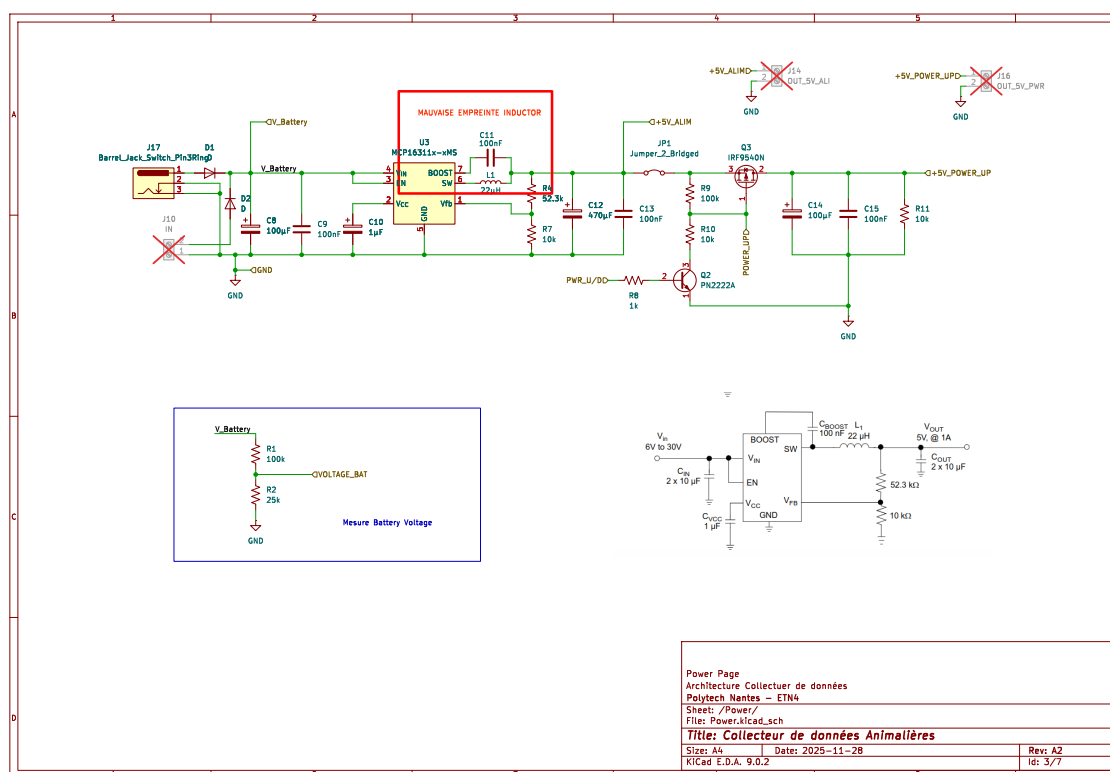


FIGURE 2.34 – Schéma d'alimentation complet du système réalisé avec Kicad

2.2.10 Tâche 8 : Création d'un PCB pour le module

Rappel des objectifs

Jusqu'à présent nous avons réalisé notre système sur une breadboard, ce qui est pratique pour les phases de test et de développement, mais qui n'est pas adapté pour une utilisation finale, car cela peut être fragile et peu fiable. De plus cela peut engendrer des problèmes d'interférence, notamment sur la caméra qui possède des signaux de synchronisation très sensibles. De plus notre système jusqu'à présent était alimenté via une batterie externe classique, ce qui n'est pas la finalité de notre projet.

Pour aller plus loin dans notre projet, nous avons décidé de réaliser un PCB (Printed Circuit Board) pour notre système, ce qui nous permettra d'avoir un système plus compact, plus fiable, et plus facile à utiliser, sans câble volant partout. Nous continuons à utiliser le logiciel Kicad qui permet la réalisation de PCB.

Solution envisagée

Pour la première partie de conception d'un circuit imprimé nous devons attribuer une empreinte (Footprint) à chaque composant de notre schéma, cela permet de définir la forme et les dimensions de chaque composant sur le PCB, ainsi que les points de soudure pour les connexions électriques. La plupart des composants basiques comme les résistances, inductances ou condensateurs possèdent des empreintes standard, mais pour les composants plus spécifiques comme notre microcontrôleur ou notre module de caméra, nous avons dû créer nos propres empreintes, en respectant les dimensions et les caractéristiques de ces composants.

Pour le cas du microcontrôleur, deux connecteurs femelles pour les broches, et pour la caméra un seul connecteur 2x9 broches, qui correspond à la nappe de la caméra OV2640. L'objectif étant de supprimer les câbles, nous avons décidé de connecter la caméra sur le PCB :

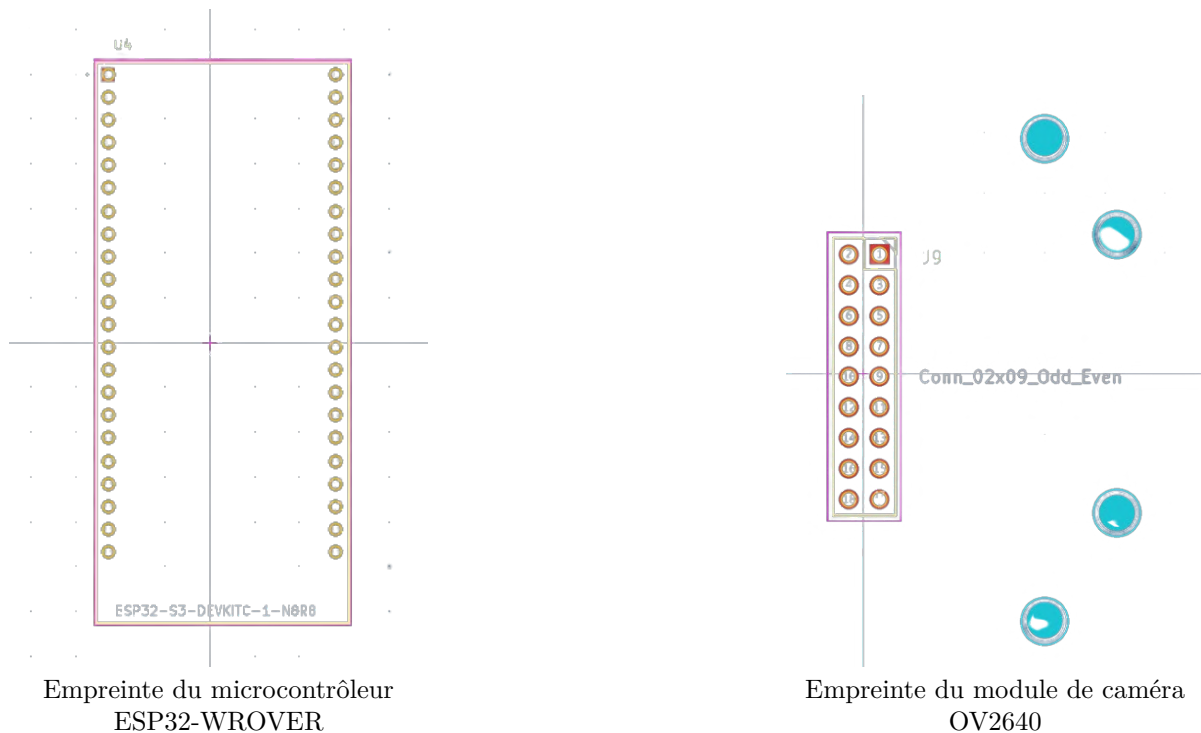


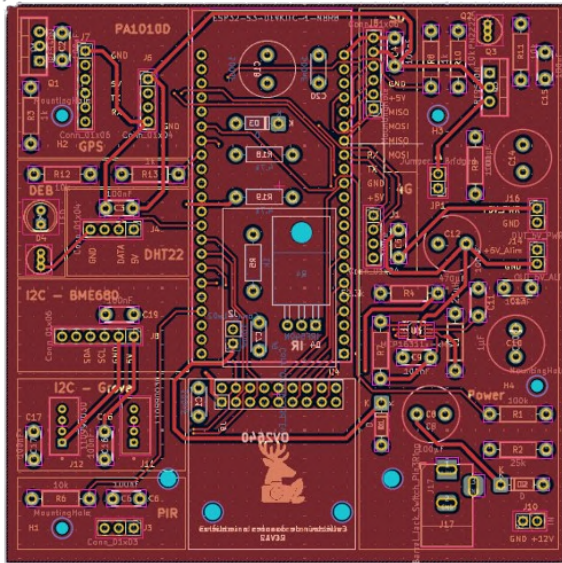
FIGURE 2.35 – Empreintes des composants sur le PCB

Pour la caméra nous choisissons d'ajouter 4 trous de fixation mécanique, deux pour la caméra, et deux autres pour la couronne de LED qui viendra par-dessus.

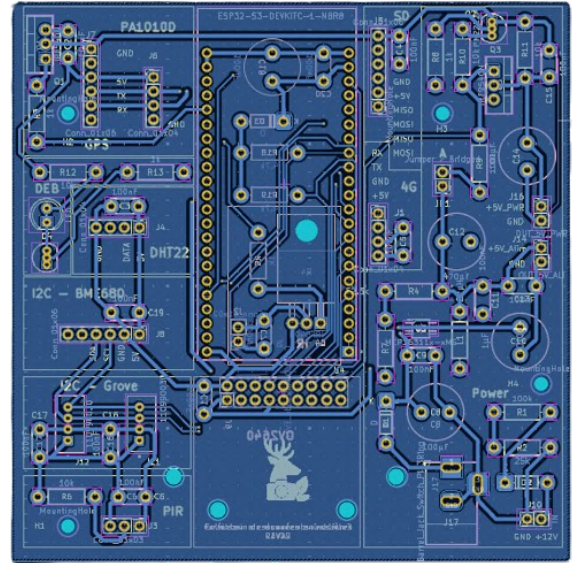
À la suite de ça nous avons réalisé le placement des composants, celui-ci en concertation avec le WP Boîtier, qui s'occupe de la partie mécanique du projet, afin de pouvoir faire un placement optimal des composants sur le PCB, en prenant en compte les contraintes mécaniques et d'encombrement du boîtier.

À la suite nous avons réalisé le routage de la carte. Pour les règles de routage, quelque chose de basique a été réalisé : des pistes de 0.5mm pour l'alimentation et des pistes de 0.3mm pour les signaux. Aucune puissance n'étant réellement élevée, nous n'avons pas besoin de faire des pistes plus larges, et cela nous permet de gagner de la place sur le PCB.

Aucun plan de masse n'a été réalisé, car notre système n'est pas sensible au bruit électromagnétique, l'isolement des pistes via des condensateurs de découplage est suffisant pour assurer la stabilité du système, et cela nous permet de gagner de la place sur le PCB.



Vue du haut du PCB



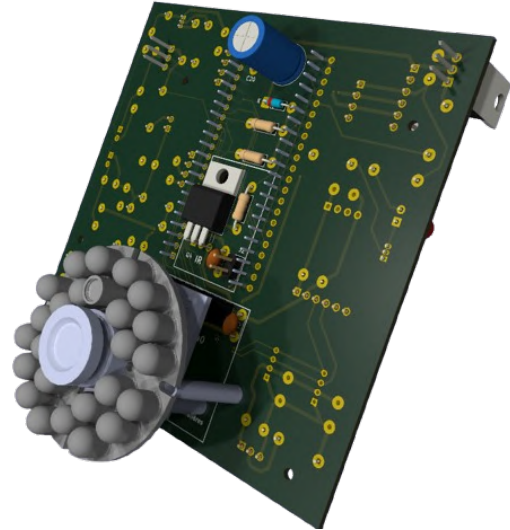
Vue du bas du PCB

FIGURE 2.36 – Rendu final du PCB réalisé avec Kicad

Voici le rendu final de notre PCB, celui-ci est réalisé en double couche, avec les composants montés des deux côtés du PCB, ce qui nous permet de gagner de la place et d'avoir un système plus compact.



Vue du haut du PCB



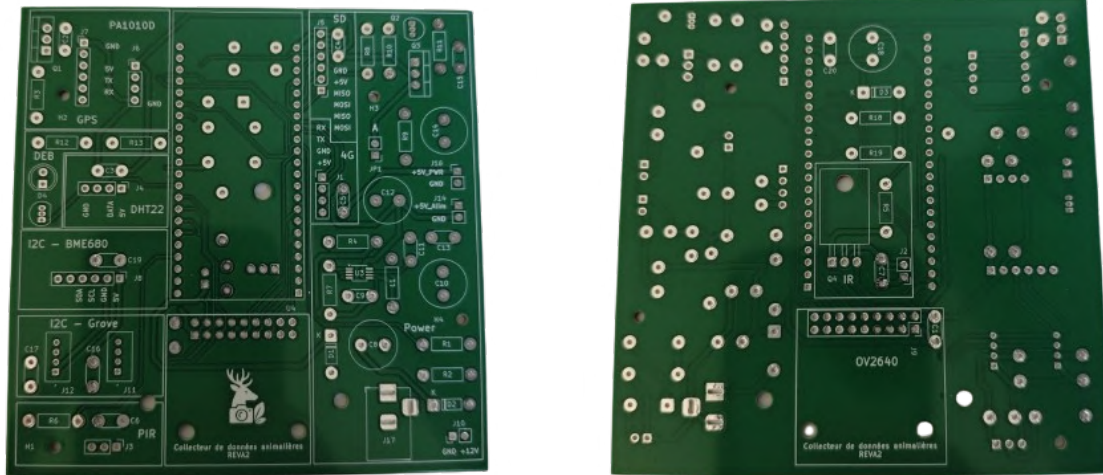
Vue du bas du PCB

FIGURE 2.37 – Rendu final 3D du PCB réalisé avec Kicad

Voici le rendu final en 3D de notre PCB, celui-ci nous permet de visualiser notre système dans son ensemble, et de vérifier que tous les composants sont correctement placés et connectés, ainsi que de vérifier les dimensions et l'encombrement du système, ce qui est essentiel pour la suite de notre projet, notamment pour la partie mécanique du boîtier.

Tests et Validation

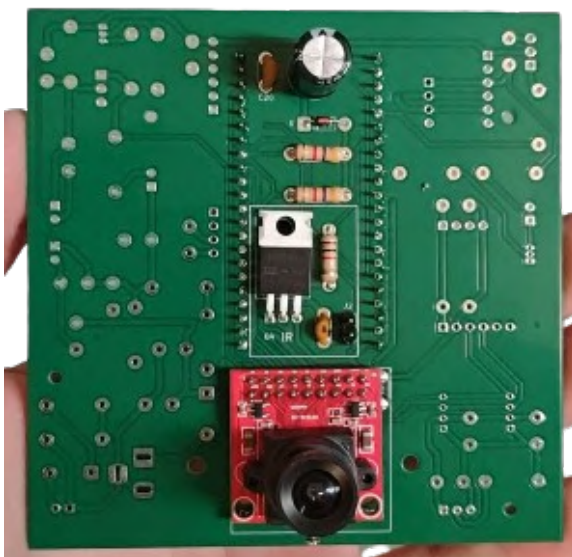
Après réception de notre PCB nous l'avons monté et soudé, avec le matériel que nous avons personnellement, et nous avons effectué des tests de tension.



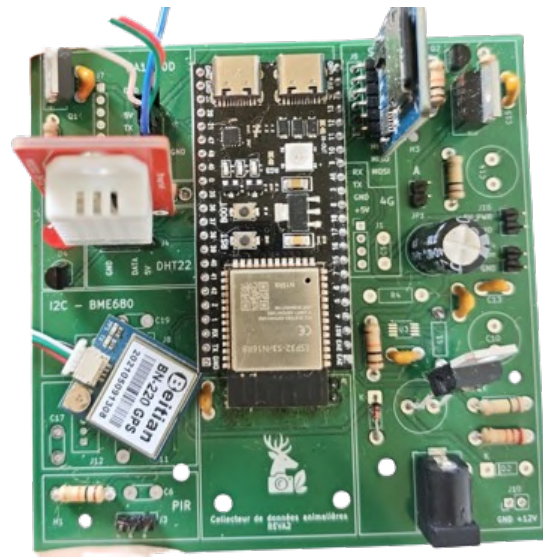
Vue du haut du PCB après réception

Vue du bas du PCB après réception

FIGURE 2.38 – Rendu final PCB après réception



Vue du bas du PCB après soudage



Vue du haut du PCB après soudage

FIGURE 2.39 – Rendu final

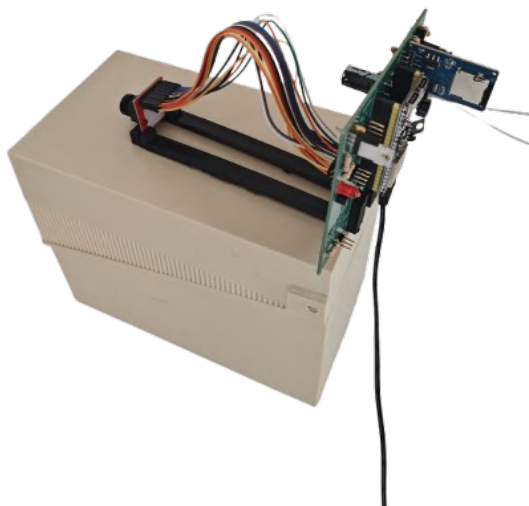
Après vérification de notre montage, nous avons remarqué deux problèmes majeurs, l'un

étant plus grave que l'autre :

Le problème numéro 1 est que l'empreinte de la caméra était inversée. Il y a deux lignes de broches et, sur notre PCB, selon la manière dont nous voulions la positionner, elles étaient interchangées. Cela a eu pour conséquence qu'une fois la caméra soudée et le système mis sous tension, nous avons observé une chute de tension. La solution que nous avons mise en place a consisté à dessouder la caméra et à mettre des câbles volants. Le PCB avait pourtant été prévu pour cela... dommage.

Le deuxième problème concerne l'alimentation du PIR. Dans le schéma, une erreur s'est glissée et le PIR est connecté au 5V POWERUP, ce qui est incorrect ; il devrait être connecté au 5V REGULATOR. Par conséquent, le PIR n'était pas alimenté et ne pouvait donc pas détecter les mouvements. La solution temporaire a été de le connecter au connecteur d'alimentation prévu pour d'autres modules, au cas où.

Malgré ces deux erreurs, il ne s'agit que d'erreurs mineures, ce qui rend tout de même notre PCB fonctionnel. Nous avons ainsi pu réaliser les tests de tension et de fonctionnement de notre système avec le PCB, ce qui constitue une grande réussite pour notre projet.



Prototype final avec le PCB soudé et fonctionnel



Vue du haut du PCB après soudage

FIGURE 2.40 – Rendu final

2.2.11 Ce qu'il faut retenir

Dans ce WP1.2 nous avons réalisé la partie de développement du module, qui correspond à la partie électronique et logicielle de notre projet. Nous avons réalisé un prototype fonctionnel de notre système, qui nous a permis de valider les différentes fonctionnalités de notre système, ainsi que de tester les différentes solutions envisagées pour la gestion de l'alimentation et de la consommation d'énergie.

Nous avons aussi réalisé un schéma électrique de notre système, ainsi que la réalisation du PCB, ce qui était un objectif majeur.

Sur le plan personnel je me suis beaucoup épanoui dans ce WP, j'étais au centre du Work Package, avec au début une sorte de mini pression pour ne pas décevoir mon chef de projet, mais cela s'est plutôt bien déroulé.

Le matériel a été le plus gros problème de ce WP, nous avons eu beaucoup de problèmes de commande, nous avons dû utiliser, ou même commander par nous-mêmes le matériel nécessaire pour la réalisation de notre projet, ce qui a engendré des retards dans notre planning, mais nous avons réussi à faire face à ces problèmes et à avancer dans notre projet malgré tout.

2.3 WP 1.3 — Caméra (responsable : Xueying Xu)

2.3.1 Rappel des objectifs

Le work package WP1.3 correspond au module caméra du système.

Son objectif principal est de permettre la prise d'images dans le cadre du piège caméra, afin de récupérer des données exploitables sur les animaux détectés.

Plus concrètement, les tâches associées sont :

- Capturer une image lorsqu'un signal de déclenchement est reçu (capteur ou commande)
- Générer des images dans un format adapté, notamment le format JPEG
- Assurer un fonctionnement en conditions de jour comme de nuit, avec un mode nocturne basé sur un éclairage infrarouge
- Obtenir une qualité d'image suffisante pour permettre une observation correcte
- Assurer que les images puissent être utilisées par les autres modules, en particulier le module MCU (WP1.2) et l'interface web (WP2.2)
- Étudier l'influence de certains paramètres (résolution, distance, etc.) sur la qualité et la taille des images

Au début du projet, plusieurs solutions ont été testées (Raspberry Pi, ESP32-CAM) afin de valider le fonctionnement global. La solution matérielle finale étant traitée dans le WP1.2, cette partie se concentre surtout sur la gestion des images et leur format.

Dans ce contexte, nous avons mené une étude spécifique sur le choix du format d'image ainsi que sur son impact sur le système global. En particulier, le format JPEG a été analysé en lien avec la taille des données, la qualité de l'image et les contraintes de transmission. Parallèlement, nous avons également réalisé plusieurs tests afin d'évaluer l'influence de la résolution et des conditions de prise de vue sur le module caméra et la qualité de l'image.

2.3.2 Solution envisagée

Afin de répondre aux objectifs du WP1.3, une solution basée sur l'acquisition et la gestion d'images au format JPEG a été retenue.

Une phase de prototypage a d'abord été réalisée à l'aide de différentes plateformes (Raspberry Pi puis ESP32-CAM) afin de valider le principe de capture d'image et de transmission. La solution matérielle finale étant traitée dans le WP1.2, cette partie se concentre principalement sur le traitement des images.

La chaîne de fonctionnement mise en place est illustrée sur la figure ci-dessous. Elle comprend le déclenchement de la capture, l'acquisition de l'image, la génération d'une image compressée au format JPEG, puis son stockage temporaire avant exploitation par les autres modules du système.

Dans le cadre du développement du module caméra, deux configurations ont été utilisées. D'une part, le capteur OV2640 intégré au système final, connecté au microcontrôleur ESP32, constitue la solution retenue pour le produit final.

D'autre part, une plateforme ESP32-CAM a été utilisée lors de la phase de prototypage. Cette plateforme intègre directement un capteur OV2640 et permet de réaliser rapidement des tests de capture, de transmission et de paramétrage de l'image.

Cette approche a permis de valider le fonctionnement global du module caméra, tout en facilitant la réalisation de mesures expérimentales dans un environnement contrôlé.

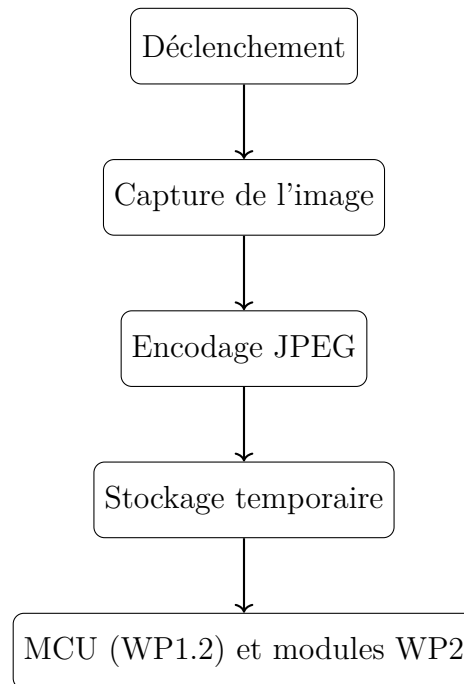


FIGURE 2.41 – Chaîne d’acquisition et de traitement des images du module caméra

Choix du capteur OV2640 Le capteur OV2640 a été retenu car il est bien adapté à une application embarquée de faible coût. Il est compatible avec l’ESP32 et permet de produire directement des images exploitables dans le système.

Son fonctionnement repose sur la conversion de la lumière captée par le capteur en données numériques. Ces données peuvent ensuite être traitées afin de générer une image compressée au format JPEG, qui sera récupérée par le microcontrôleur pour être stockée ou transmise.

Ce choix présente plusieurs avantages dans le cadre du projet :

- génération directe d’images JPEG ;
- réduction de la charge de traitement côté microcontrôleur ;
- taille de données plus adaptée à la transmission ;
- bonne intégration dans une architecture embarquée.

Les tests réalisés ont également confirmé ce choix, en montrant que le module permet de générer des images exploitables avec des tailles et des temps de capture compatibles avec les contraintes du système.

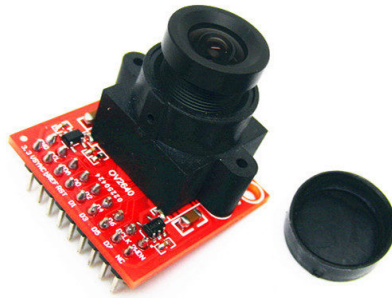


FIGURE 2.42 – Capteur OV2640 utilisé dans le système final

Principe du format JPEG Le format JPEG est un standard de compression d'image largement utilisé, permettant de réduire la taille des fichiers tout en conservant une qualité visuelle acceptable. Il repose sur une compression avec perte, ce qui signifie que certaines informations de l'image sont volontairement simplifiées ou supprimées afin de diminuer la quantité de données.

Son principe consiste à représenter l'image en séparant les informations les plus importantes de celles qui le sont moins pour l'œil humain. D'un point de vue théorique, l'image est découpée en blocs, puis chaque bloc est transformé dans le domaine fréquentiel, généralement par une transformée en cosinus discrète (DCT). Les coefficients obtenus sont ensuite quantifiés : les composantes les moins importantes, souvent associées aux hautes fréquences, sont réduites ou supprimées. Cette étape permet de diminuer fortement la quantité de données à stocker ou à transmettre.

Dans le cadre de ce projet, le choix du format JPEG est particulièrement pertinent. En effet, il permet de limiter la taille des images produites par le système, ce qui réduit les contraintes de stockage et facilite la transmission via le réseau WiFi. Ce compromis entre qualité visuelle et volume de données est essentiel pour garantir le bon fonctionnement global du système.

Dans notre cas, le choix du format JPEG est également motivé par les contraintes du microcontrôleur ESP32, qui dispose de ressources limitées en mémoire et en puissance de calcul.

2.3.3 Tests et Validation

Les mesures présentées dans cette section ont été réalisées à l'aide de la plateforme ESP32-CAM, en utilisant une interface de capture basée sur une requête HTTP de type GET (/capture). Chaque requête déclenche la capture d'une image unique, permettant ainsi de contrôler précisément les conditions d'acquisition.

Pour chaque résolution, plusieurs captures consécutives ont été effectuées dans des conditions identiques, afin d'obtenir des valeurs représentatives. Les paramètres de la caméra ont été maintenus constants pendant toute la durée des tests.

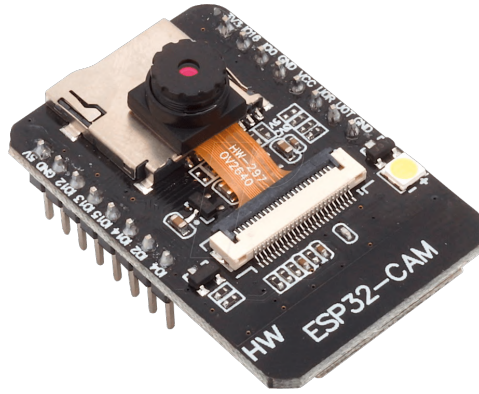


FIGURE 2.43 – Plateforme ESP32-CAM utilisée pour les tests expérimentaux

Afin d'évaluer les performances de la chaîne d'acquisition d'image sur plateforme ESP32, nous avons réalisé une série de tests.

Tout d'abord, nous avons vérifié le format des images générées. Les résultats montrent que le module peut correctement générer des images au format JPEG, et que ces images peuvent être utilisées par les autres modules du système.

De plus, nous avons étudié l'influence de la résolution sur la taille des images. Les résultats montrent qu'avec l'augmentation de la résolution, la taille des images augmente également, passant de quelques kilooctets en basse résolution (QQVGA) jusqu'à environ 30 kB en haute résolution (XGA). Comme montré sur les tableaux ci-dessous, une augmentation de la résolution entraîne une augmentation de la taille des images ainsi qu'un impact sur le temps de capture.

Les résultats obtenus pour différentes résolutions sont présentés dans les tableaux suivants.

TABLE 2.11 – Taille des images JPEG selon la résolution

Résolution	Largeur×Hauteur	Img1 (KB)	Img2	Img3	Img4	Moy. (KB)
QQVGA	160×120	1.4	2.9	2.9	3.0	2.55
QVGA	320×240	4.3	4.3	4.3	4.3	4.30
VGA	640×480	11.7	7.2	12.0	21.6	13.13
SVGA	800×600	32.4	35.0	24.4	23.4	28.80
XGA	1024×768	29.8	34.4	30.9	34.4	32.38

TABLE 2.12 – Temps de capture selon la résolution

Résolution	Largeur×Hauteur	Essai1 (μs)	Essai2	Essai3	Essai4	Moy. (μs)
QQVGA	160×120	25	25	23	28	25.25
QVGA	320×240	30	26	27	36	29.75
VGA	640×480	216	144	231	236	206.80
SVGA	800×600	89	98	126	211	131.0
XGA	1024×768	219	249	254	219	235.25

On observe que l'augmentation de la résolution entraîne une croissance significative de la taille des images ainsi qu'une augmentation du temps de capture. Pour une même

résolution, des variations peuvent également apparaître, car la compression JPEG dépend du contenu visuel de la scène, notamment du niveau de détail et des contrastes. Cela met en évidence un compromis entre la qualité de l'image et les performances du système, qui doit être pris en compte dans le choix des paramètres de fonctionnement.

Nous avons également analysé le temps de capture des images. Les résultats montrent que le temps de capture est globalement faible, mais qu'il varie en fonction de la résolution. Cependant, la latence globale provient principalement de la transmission WiFi ainsi que du traitement côté client.

Enfin, nous avons testé la qualité des images à différentes distances de prise de vue. Les résultats montrent que la qualité est bonne à courte distance, mais qu'elle diminue progressivement lorsque la distance augmente.

Globalement, ces résultats montrent que le module caméra fonctionne correctement et qu'il répond aux besoins du projet en termes de qualité d'image et de transmission des données.

Synthèse Les principaux éléments du WP1.3 sont les suivants :

- mise en place d'une chaîne complète d'acquisition d'image, depuis le déclenchement jusqu'à l'exploitation des données ;
- choix du capteur OV2640 pour la génération directe d'images JPEG ;
- étude du format JPEG et de son intérêt pour la réduction des données ;
- analyse de l'influence de la résolution sur la taille des images et le temps de capture ;
- validation expérimentale du fonctionnement du module caméra ;
- prise en compte du fonctionnement en conditions de jour et de nuit.

Dans l'ensemble, le module caméra constitue un élément fiable et cohérent du système, permettant une acquisition efficace des données visuelles tout en respectant les contraintes matérielles et énergétiques du projet.

2.3.4 Ce qu'il faut retenir

Cette section a permis de valider le module caméra comme un élément central du système d'acquisition de données. Voici les points essentiels à retenir de ce Work Package :

- **Choix du capteur (OV2640)** : Le capteur OV2640 a été retenu pour sa compatibilité avec l'ESP32, sa capacité à générer directement des images JPEG et son adéquation avec les contraintes d'un système embarqué basse consommation.
- **Validation par prototypage (ESP32-CAM)** : L'utilisation de la plateforme ESP32-CAM a permis de valider rapidement la chaîne complète d'acquisition (capture, compression, transmission), facilitant les tests expérimentaux.
- **Choix du format JPEG** : Le format JPEG a été privilégié pour réduire la taille des données tout en conservant une qualité visuelle acceptable, ce qui est essentiel pour le stockage et la transmission via WiFi.
- **Impact de la résolution** : Les tests montrent une augmentation significative de la taille des images avec la résolution (de quelques kB à ~ 30 kB), mettant en évidence un compromis entre qualité et performance.

- **Variabilité liée au contenu** : Pour une même résolution, la taille des images varie en fonction du contenu visuel (détails, contrastes), en raison du caractère adaptatif de la compression JPEG.
- **Performance globale** : Le temps de capture reste globalement faible, la latence étant principalement liée à la transmission WiFi et au traitement côté client.
- **Contraintes embarquées** : Le choix des paramètres d'acquisition est fortement influencé par les limitations du microcontrôleur (mémoire, puissance), nécessitant des compromis adaptés.
- **Validation fonctionnelle** : Le module caméra permet de générer des images exploitables dans différentes conditions (distance, luminosité), validant son intégration avec les autres modules du système.

2.4 WP 1.4 — Conception et fabrication du boîtier (responsable : Joris Menard)

Rappel des objectifs

L'objectif de ce work package était de concevoir une solution permettant d'abriter le matériel électronique installé sur le site de Saint-Mars-du-Désert, de le protéger contre l'humidité et les intempéries, tout en assurant une dissipation correcte de la chaleur générée par les composants électroniques, même si le système ne fait pas partie des plus énergivores.

Solution envisagée

Pour répondre à cet objectif, nous avons envisagé d'utiliser un logiciel de conception 3D comme SolidWorks et d'utiliser une imprimante 3D pour fabriquer un boîtier sur mesure. Cela facilite le développement de la solution, permet de réduire les coûts et d'obtenir un boîtier parfaitement adapté à notre projet.

Pour assurer une bonne étanchéité du boîtier, nous avons prévu d'utiliser des joints pour sceller les différentes ouvertures du boîtier, mais également d'utiliser des matériaux résistants à l'eau pour sa fabrication, tels que le PETG ou l'ABS.

L'objectif est également de garantir à l'utilisateur un accès simple et rapide à l'intérieur du boîtier pour permettre de changer la batterie.

Mise en œuvre, tests et validation

Le développement du boîtier s'est déroulé en deux principales étapes. La première a consisté à concevoir une partie du boîtier destinée à accueillir la batterie. Il s'agit d'une boîte rectangulaire dans laquelle la batterie est fixée à l'aide d'un scratch, permettant à la fois un maintien solide et un accès facile pour la retirer et la remplacer pour la recharger.

Sur cette partie, nous avons également prévu d'ajouter quatre encoches sur les côtés, qui permettront de fixer le boîtier à un support, tel qu'un arbre dans notre cas, à l'aide de sangles.

La seconde étape a été de concevoir la partie du boîtier destinée à accueillir la carte électronique et ses composants. Pour cela, nous avons placé deux ouvertures à l'avant du boîtier, la première étant prévue pour accueillir la caméra accompagnée de la couronne de LED et la seconde en forme de cône pour permettre de placer le capteur PIR et ensuite détecter les mouvements dans une zone définie selon l'angle du cône.

Une vitre transparente, collée et rendue étanche grâce à un joint, a été prévue pour recouvrir l'ouverture destinée à la caméra et au capteur PIR. Elle assure leur protection contre les intempéries tout en préservant les performances de chaque composant, la caméra peut continuer à capturer des images et le capteur PIR à détecter les mouvements

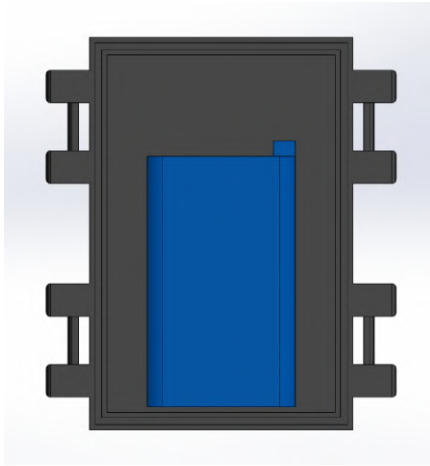


FIGURE 2.44 – Partie du boîtier accueillant la batterie

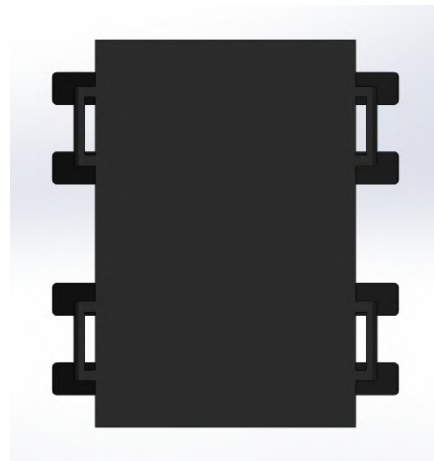


FIGURE 2.45 – Encoches pour les sangles de fixation

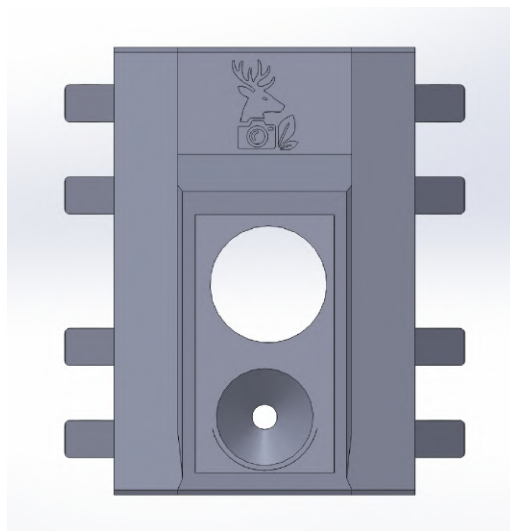


FIGURE 2.46 – Partie du boîtier accueillant la caméra et le capteur PIR

sans aucune obstruction.

À l'intérieur de cette partie du boîtier, nous avons ajouté des supports pour fixer la carte électronique à l'aide de vis, ainsi que des supports pour placer le module de communication 4G (TEL0177).

Pour protéger le matériel électronique lors de l'ouverture du boîtier, nous avons prévu d'ajouter une pièce fixée par des vis au bord de cette partie du boîtier, qui viendra recouvrir les composants électroniques et les protéger.

Pour refermer le boîtier, nous avons prévu de les assembler à l'aide d'un tenon sur l'une des parties et d'une mortaise sur l'autre partie, afin de les emboîter et de refermer le boîtier. Pour assurer une bonne étanchéité, nous avons prévu d'ajouter un joint en ca-

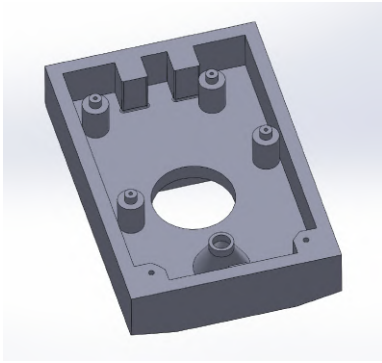


FIGURE 2.47 – Support pour la carte électronique

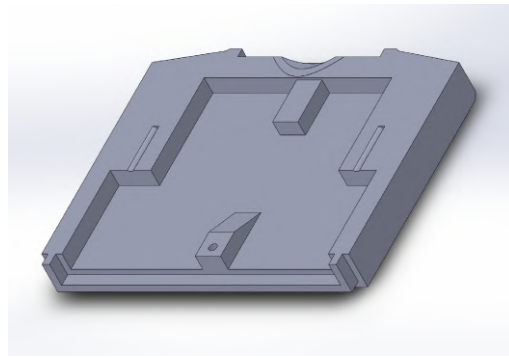


FIGURE 2.48 – Support pour le module de communication 4G

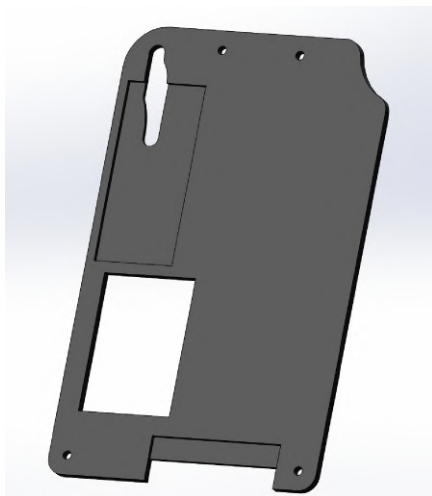


FIGURE 2.49 – Pièce de protection des composants électroniques

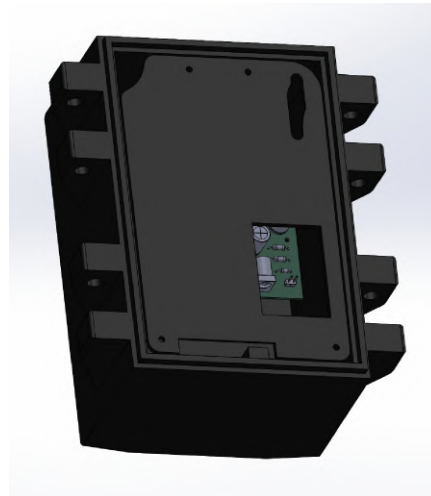


FIGURE 2.50 – Pièce de protection fixée sur le boîtier

outchouc sur le pourtour de la partie du boîtier accueillant la mortaise.

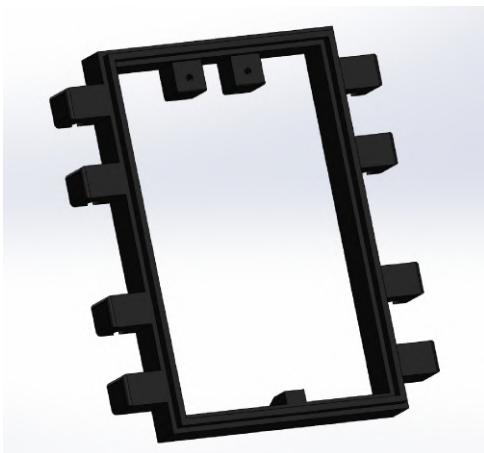


FIGURE 2.51 – Partie du boîtier avec le tenon

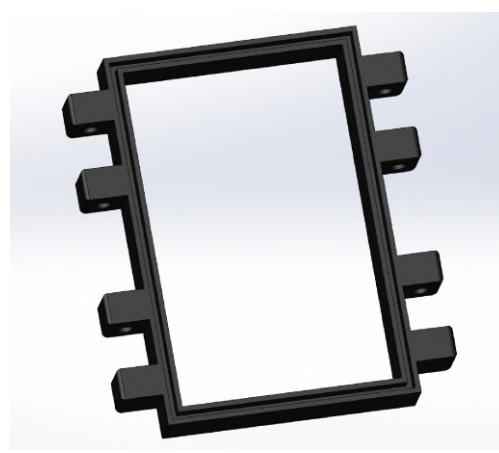


FIGURE 2.52 – Partie du boîtier avec la mortaise

Afin d'assurer une fermeture robuste du boîtier, nous avons prévu d'imprimer des clips venant se fixer sur ses côtés. Pour ce faire, des excroissances en relief seront intégrées sur les flancs des deux parties du boîtier, avec des trous destinés à accueillir des tubes en acier. Ces tubes serviront de maintien des clips sur l'une des parties du boîtier, tandis que sur l'autre partie les clips pourraient être clipsés et déclipés à volonté, permettant ainsi d'ouvrir ou de fermer le boîtier facilement.

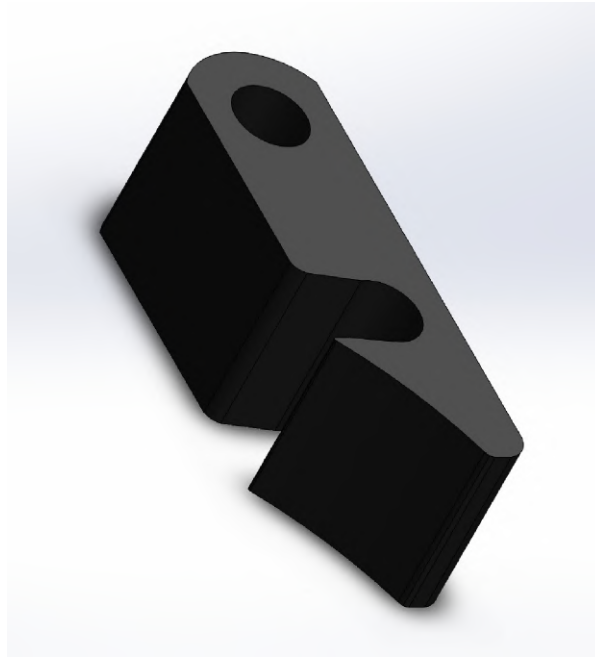


FIGURE 2.53 – Clips de fermeture du boîtier

Des problèmes d'approvisionnement en matériel nous ont malheureusement empêchés de recevoir les équipements nécessaires à la finalisation du boîtier. En l'absence de tubes en acier, le système de fermeture n'a pas pu être assemblé, ni testé, ni validé. Par ailleurs, sans vitre ni joint, l'étanchéité du boîtier ne peut être garantie, ce qui exclut toute validation dans des conditions réelles d'utilisation, c'est-à-dire en extérieur et exposé aux intempéries.

L'image du boîtier final envisagé est présentée ci-dessous :



FIGURE 2.54 – Image du boîtier final envisagé

Pour les tests, aucune stratégie de test formelle n’a été mise en place ; nous avons juste vérifié que les différentes parties du boîtier s’assemblent correctement et que le boîtier puisse accueillir les composants électroniques dont nous disposons.

Ce qu’il faut retenir

À retenir de cette partie du projet : les deux parties principales du boîtier ont été imprimées avec succès et peuvent être assemblées. Le boîtier est en mesure d’accueillir la carte électronique, cette dernière étant maintenue fermement en place.

Cependant, en raison des problèmes d’approvisionnement évoqués, le système de fermeture n’a pu être ni assemblé, ni testé, ni validé. L’absence de vitre et de joint empêche par ailleurs de garantir l’étanchéité du boîtier, excluant ainsi toute mise à l’épreuve du système dans des conditions réelles d’utilisation en extérieur.

2.5 WP bis — Transmission (responsable : Joris Menard)

2.5.1 Rappel des objectifs

Cette partie du projet vise à développer une solution de communication sans fil pour transmettre les données capturées par la caméra, ainsi que les métadonnées associées (localisation GPS, heure, température, etc...). L'objectif est de permettre l'envoi de ces informations à un serveur distant connecté au réseau internet et accessible à tous, où elles pourront être stockées et exploitées pour diverses applications.

2.5.2 Solution envisagée

Pour répondre à cet objectif, comme indiqué précédemment dans le rapport, nous avons choisi d'utiliser une carte de développement ESP32 intégrant un module de communication sans fil, pour implémenter la solution. Afin de déterminer la technologie de communication mobile la plus adaptée, nous avons élaboré un tableau comparatif des différentes options disponibles. Cette comparaison prend en compte plusieurs critères, tels que la portée, le débit, le coût, la consommation énergétique ou encore la simplicité de mise en œuvre, appliqués à diverses technologies comme la transmission HF, LoRa ou la 4G, 2G, 3G ou 5G.

Nous en avons déduit que la 4G, 2G, 3G et 5G sont les technologies les plus adaptées à notre projet. Elles offrent en effet une bonne couverture réseau dans la zone de Saint-Mars-du-Désert, des débits suffisants pour transmettre les images ainsi que les métadonnées, et une mise en œuvre relativement simple. En effet, de nombreux projets similaires sont déjà disponibles sur Internet, ce qui nous a permis de nous en inspirer pour faciliter le développement.

De plus, la 4G, 2G, 3G et 5G restent économiquement accessibles, avec des modules de communication et des forfaits de données à un coût raisonnable.

Les choix techniques concernant le développement de la solution utilisant la 4G, 2G, 3G et 5G seront détaillés plus bas dans le rapport.

2.5.3 Mise en œuvre, tests et validation

Avant de se lancer dans le développement de la solution de communication utilisant la 4G, 2G, 3G et 5G, il était nécessaire de tester notre architecture d'envoi des données en utilisant une connexion Wi-Fi.

Connexion de la carte en Wi-Fi

Un code C++ permettant de connecter l'ESP32 à un réseau à l'aide de son SSID et de son mot de passe a été développé. Pour cela, une bibliothèque nommée `WiFi.h` a été utilisée pour faciliter le développement de cette fonctionnalité.

Pour valider la connexion de la carte à un réseau, un petit réseau a été mis en place, un téléphone jouant le rôle de point d'accès. L'ESP32 se connecte alors au réseau ; la capture d'écran ci-dessous montre les appareils connectés à ce réseau local, la présence

de l'ESP32 permet donc de confirmer sa connexion.

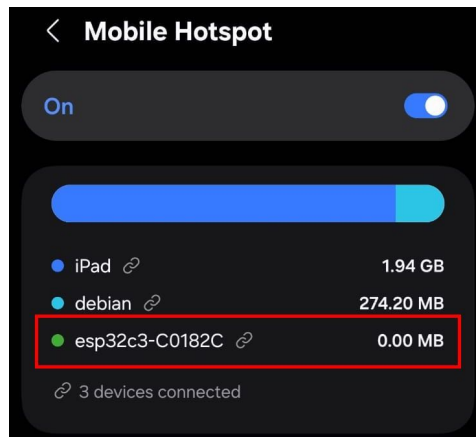


FIGURE 2.55 – Capture d'écran des paramètres du point d'accès du téléphone

Envoi de données à un autre appareil connecté sur le réseau

Une fois la connexion établie, une fonction supplémentaire a été mise en place. Elle permet l'envoi, pour le moment, d'un caractère vers un autre appareil sur le réseau connaissant son adresse IP. Pour l'envoi des données, il a été choisi de tester pour l'instant le protocole UDP. Ce protocole est utilisé dans le streaming vidéo sur Internet, ce qui nous fournit des exemples d'utilisation. Il permet également d'assurer une plus grande rapidité des échanges, car, contrairement à d'autres protocoles, il ne nécessite pas de confirmation de réception de chaque paquet.

Pour l'envoi des données, il est nécessaire de spécifier un port de destination. C'est comme une porte d'entrée qui doit être ouverte sur l'appareil récepteur pour permettre la réception des données. Il a également été nécessaire d'indiquer l'adresse IP de l'appareil destinataire.

Nous avons choisi d'utiliser le port numéro 5005, simplement parce qu'il est disponible sur l'appareil récepteur et qu'il n'est pas utilisé par d'autres services, ce qui permet d'éviter les conflits.

Pour valider le fonctionnement de la communication, un envoi de données simple a été effectué. Pour cela, les données envoyées par l'ESP32 ont été affichées dans le terminal de l'ordinateur portable auquel est connecté l'ESP32. Une application permettant de visualiser les trames UDP a également été installée sur le téléphone, qui symbolise l'appareil récepteur. Un caractère incrémenté dans la table ASCII est envoyé, ce qui permet de suivre l'envoi et la réception continue des informations. Les images ci-dessus confirment que la communication fonctionne correctement, les données envoyées depuis l'ESP32 sont bien reçues par l'appareil récepteur.

```

A
Connecté
SSID : Joris's A56
B
Connecté
SSID : Joris's A56
C
Connecté
SSID : Joris's A56
D
Connecté
SSID : Joris's A56
E
Connecté
SSID : Joris's A56
F
Connecté
SSID : Joris's A56
G
Connecté
SSID : Joris's A56
H
Connecté
SSID : Joris's A56
I
Connecté
SSID : Joris's A56
J
Connecté
SSID : Joris's A56
K
Connecté
SSID : Joris's A56
L
Connecté
SSID : Joris's A56
M
Connecté
SSID : Joris's A56
N

```

FIGURE 2.56 – Données envoyées depuis l'ESP32

UDP

IP / Host : 192.0.0.4 Port : 5005

Message: cocu

SEND

Listening Port : 5005

LISTEN

Response / Log

- o
- n
- m
- l
- k
- j
- i
- h
- g
- f
- e
- d
- c
- b
- a

FIGURE 2.57 – Données reçues depuis le téléphone

Implémentation de la solution en utilisant le protocole HTTP

Après discussion avec la personne en charge du work package **Stockage distant**, nous avons fait le choix d'utiliser le protocole HTTP. C'est un protocole de couche application de la suite de protocoles Internet, utilisé pour les systèmes d'information hypermédia distribués et collaboratifs. Ce choix s'explique par le fait qu'il est possible d'envoyer l'image capturée par la caméra et ses métadonnées en une seule requête. Les protocoles des couches inférieures du modèle ISO se chargent automatiquement de découper les données en paquets si nécessaire, ainsi que de gérer le format des trames, ce qui garantit une transmission fiable. Le protocole HTTP est donc simple à mettre en place. De plus, il permet de recevoir un code de retour du serveur, ce qui facilite la confirmation de la bonne réception des données. Il est également d'une simplicité d'installation sur l'appareil récepteur. Tous ces éléments font du protocole HTTP une solution adaptée à notre projet.

Pour mettre en place le protocole HTTP, nous avons configuré un serveur web sur l'appareil récepteur des données. L'envoi des données s'effectue via une requête POST adressée au

serveur web, en précisant l'URL correspondant à l'adresse IP et au port du serveur. Le port choisi est le 5050, il a été sélectionné arbitrairement après vérification de sa disponibilité afin d'éviter tout conflit avec d'autres services.



Une fois le protocole HTTP configuré, un test d'envoi d'une image simple a été réalisé. Pour ce test, une image de 20x20 pixels a été générée directement dans le code de l'ESP32, chaque pixel étant codé sur trois octets selon le format **RRGGBB**. L'image a ensuite été transmise au serveur web via une requête **POST**. Le serveur renvoie un code permettant de vérifier la bonne réception de l'image, si le code retourné est 200, l'envoi a été effectué avec succès.

Pour vérifier que l'image reçue correspond exactement à celle envoyée, sans octet manquant ou erroné et avec les octets dans le bon ordre, un calcul de la valeur **CRC32** a été effectué à la fois sur l'ESP32 avant l'envoi et sur le serveur après la réception.

Le **CRC32** (Cyclic Redundancy Check sur 32 bits) est une somme de contrôle largement utilisée pour détecter les erreurs dans les données transmises ou stockées. Il calcule un code unique à partir du contenu des données, même une modification d'un seul bit produit une valeur **CRC32** totalement différente.

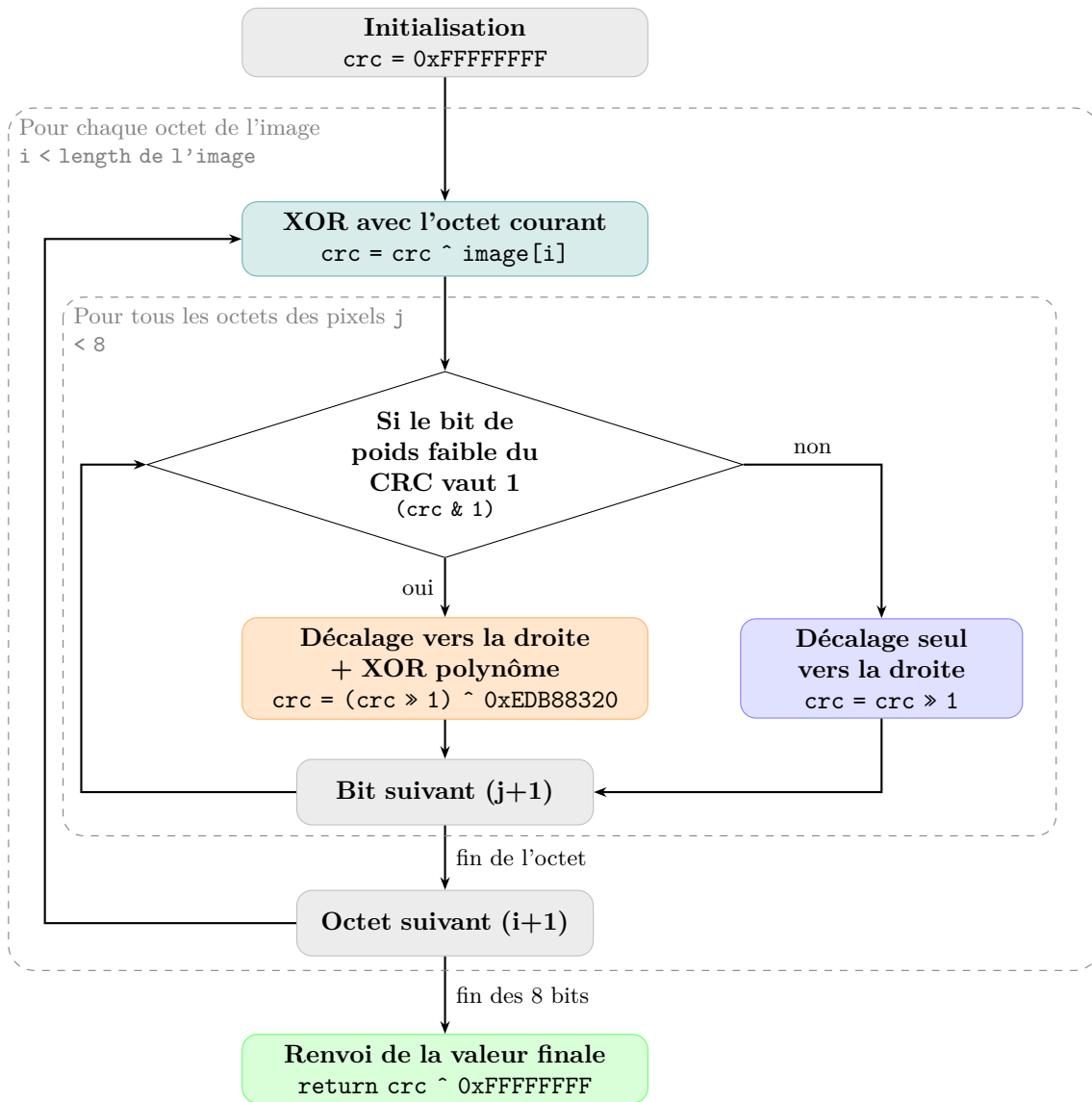


FIGURE 2.58 – Schéma de l'algorithme de calcul du CRC32

Les images ci-dessous montrent que la valeur CRC32 calculée pour l'image envoyée correspond à celle de l'image reçue, et que le code de retour est bien 200, confirmant ainsi que l'envoi s'est déroulé correctement et que l'image a été reçue sans erreur.

```

~/Doc/myData/C/S7/Projet_T/W/T/t/src main >2 *1 !720 ?16 > python3 im.py
Listening on /dev/ttyACM0...
Début de l'image
Numéro de l'image : 1
CRC32 image envoyé : 0xDF057787
Code de retour : 200
Fin de l'image
Début de l'image
Numéro de l'image : 2
CRC32 image envoyé : 0x7A09B38D
Code de retour : 200
Fin de l'image
Début de l'image
Numéro de l'image : 3
CRC32 image envoyé : 0xB122DFA9
Code de retour : 200
Fin de l'image
Début de l'image
Numéro de l'image : 4
CRC32 image envoyé : 0x70D4AD75
Code de retour : 200
Fin de l'image
Début de l'image
Numéro de l'image : 5
CRC32 image envoyé : 0xEC87614F
Code de retour : 200
Fin de l'image

```

FIGURE 2.59 – CRC32 de l'image envoyée et code de retour

```

PS C:\Users\joris\OneDrive\Documents\Ptrans> & C:\msys64\ucrt64\bin\python.exe c:/Users/joris/OneDrive/Documents/Ptrans/code_recep_data.py
* Serving Flask app 'code_recep_data'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5050
* Running on http://192.168.1.32:5050
Press CTRL+C to quit
CRC32 image reçue: DF057787
Numéro d'image : 1
192.168.1.37 - - [30/Nov/2025 13:43:05] "POST /upload HTTP/1.1" 200 -
CRC32 image reçue: 7A09B38D
Numéro d'image : 2
192.168.1.37 - - [30/Nov/2025 13:43:10] "POST /upload HTTP/1.1" 200 -
CRC32 image reçue: B122DFA9
Numéro d'image : 3
192.168.1.37 - - [30/Nov/2025 13:43:15] "POST /upload HTTP/1.1" 200 -
CRC32 image reçue: 70D4AD75
Numéro d'image : 4
192.168.1.37 - - [30/Nov/2025 13:43:21] "POST /upload HTTP/1.1" 200 -
CRC32 image reçue: EC87614F
Numéro d'image : 5
192.168.1.37 - - [30/Nov/2025 13:43:26] "POST /upload HTTP/1.1" 200 -

```

FIGURE 2.60 – CRC32 de l'image reçue

Cette stratégie de test et de validation a été appliquée de manière progressive sur plusieurs dimensions d'image. La taille des images transmises a été augmentée par paliers successifs, en vérifiant à chaque étape la correspondance des CRC32 calculés en émission et en réception. Cette approche a permis de valider avec succès la transmission d'images jusqu'à la résolution finale de 1200×1600 pixels, correspondant aux images capturées par la caméra.

Le calcul du CRC32 a été enlevé dans le code final implémenté sur l'ESP32, car il s'agit d'une étape de validation qui n'est pas nécessaire dans la version finale du projet.

Ajout des métadonnées à l'envoi des données

Le dernier développement a été d'ajouter les métadonnées à l'envoi des données. Pour cela, nous avons choisi d'utiliser le format JSON pour structurer les métadonnées, car il est à la fois léger et facile à manipuler. Le JSON est formé de paires clé-valeur, comme visible ci-dessous :

```
1 {  
2   "IMG": "cl'e image",  
3   "IMG.TYPE": "type image",  
4   "IMG.WIDTH": "largeur image",  
5   "IMG.HEIGHT": "hauteur image",  
6   "WEATHER.TEMP": "temperature",  
7   "WEATHER.HUM": "humidit'e",  
8   "GPS.LONG": "longitude",  
9   "GPS.LAT": "latitude",  
10  "DATE.YEAR": "annee",  
11  "DATE.MONTH": "mois",  
12  "DATE.DAY": "jour",  
13  "DATE.HOUR": "heure",  
14  "DATE.MINUTE": "minute",  
15  "DATE.SECOND": "seconde",  
16  "CAM.BATTERY": "niveau batterie",  
17  "CAM.ID": "identifiant de camera"  
18 }
```

Pour valider l'envoi des métadonnées, nous avons simplement comparé visuellement les données envoyées depuis l'ESP32 avec celles reçues par le serveur et affichées sur la page web. L'objectif était de vérifier que les clés étaient identiques entre les deux interfaces, émettrice et réceptrice. Cette vérification s'est révélée concluante dès le premier test.

Réalisation du premier prototype fonctionnel

Après toutes ces étapes de développement, de test et de validation, nous avons intégré ce work package aux autres afin de réaliser un premier prototype fonctionnel. Cela nous a permis d'identifier les points fonctionnels, les points non fonctionnels ainsi que les axes d'amélioration à apporter à chaque work package.

Les tests réalisés précédemment ont permis de valider l'ensemble des points de ce work package. Nous n'avons donc pas eu à effectuer de corrections supplémentaires sur celui-ci après l'assemblage du prototype.

Analyse comparative des technologies de communication mobile

Une fois la solution de communication sans fil mise en place, testée et validée avec le Wi-Fi, nous avons procédé à une analyse comparative entre les différentes technologies de communication mobile disponibles, à savoir la 2G, la 3G, la 4G et la 5G, pour déterminer laquelle serait la plus adaptée à notre projet.

La 2G

La 2G (GSM) a introduit les communications numériques et les SMS, mais avec des débits de données très limités.

Technologies :

- GSM
- GPRS
- EDGE

Bandes passantes principales :

- Bande 8 (B8) – 900 MHz
- Bande 3 (B3) – 1800 MHz

La 3G

La 3G a augmenté les débits par rapport à la 2G, rendant possibles la navigation web, le streaming et les appels vidéo.

Technologies :

- UMTS
- HSPA
- HSPA+

Bandes passantes principales :

- Bande 8 (B8) – 900 MHz
- Bande 1 (B1) – 2100 MHz

La 4G

La 4G offre des débits de données plus élevés que les générations précédentes, grâce à des techniques de modulation et de multiplexage optimisées.

Technologies :

- LTE
- FDD

Bandes passantes principales :

- Bande 28 (B28) – 700 MHz
- Bande 20 (B20) – 800 MHz
- Bande 3 (B3) – 1800 MHz
- Bande 1 (B1) – 2100 MHz
- Bande 7 (B7) – 2600 MHz

La 5G

La 5G offre des débits plus élevés, une latence plus faible et une meilleure prise en charge des objets connectés.

Technologies :

- NR (New Radio)
- NSA (Non-Standalone)
- SA (Standalone)

Bandes passantes principales :

- Bande n28 – 700 MHz
- Bande n78 – 3,5 GHz

Avant de choisir le type de technologie à utiliser, nous allons effectuer une analyse de la couverture réseau dans la zone de Saint-Mars-du-Désert, afin de sélectionner la meilleure technologie pour notre projet.

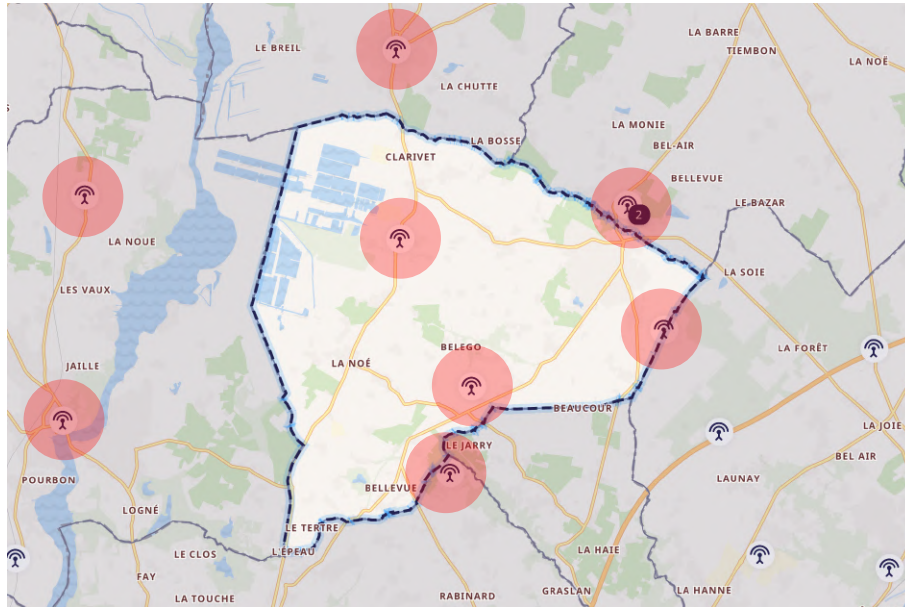


FIGURE 2.61 – Carte des antennes présentes dans la zone de Saint-Mars-du-Désert

Sur toutes les antennes (représentées par les cercles rouges) répertoriées sur la figure 2.61 et présentes dans la zone de Saint-Mars-du-Désert, les opérateurs disponibles sont :

- Orange
- SFR
- Free
- Bouygues

L'objectif est donc de prendre un forfait 4G chez l'un de ces opérateurs, ou chez un opérateur qui bénéficie de leur couverture réseau.

Les bandes de fréquences disponibles sur les antennes dans la zone de Saint-Mars-du-Désert sont les suivantes :

- 700 MHz – Bande 28 (B28)
- 800 MHz – Bande 20 (B20)
- 900 MHz – Bande 8 (B8)
- 1800 MHz – Bande 3 (B3)
- 2100 MHz – Bande 1 (B1)
- 2600 MHz – Bande 7 (B7)

Après analyse, nous avons choisi d'opter pour la 4G. En effet, elle offre une bonne couverture réseau dans la zone de Saint-Mars-du-Désert ainsi que des débits suffisants pour la transmission des images et des métadonnées. Par ailleurs, les bandes de fréquence 4G sont les plus utilisées par les antennes présentes dans cette zone. De plus, la 4G est aujourd'hui la technologie la plus répandue sur les modules de communication disponibles sur le marché, ce qui facilite le développement de la solution.

Choix du module de communication

La prochaine étape a été de choisir un module de communication compatible avec la technologie 4G.

Pour cela, nous avons recherché des modules de communication 4G, fonctionnant aux bandes de fréquences utilisées par les antennes présentes dans la zone de Saint-Mars-du-Désert, et compatibles avec une interface de communication série (UART) pour faciliter la communication avec l'ESP32.

Trouver un module de communication adapté, en particulier un module 4G, s'est révélé difficile sur les sites français, en raison à la fois des prix souvent élevés et des délais de livraison parfois importants.

Nous avons néanmoins identifié un module 4G, le **TEL0177**, basé sur le microcontrôleur **Air780EU**. Ce module est disponible auprès de distributeurs spécialisés en composants électroniques tels que Farnell, Mouser, Digi-Key ou Gotronic. Ces fournisseurs disposent de dépôts en France ou en Europe, ce qui permet de bénéficier de délais de livraison plus courts.

Le module TEL0177 est également disponible à un prix raisonnable. Il est compatible avec les bandes de fréquences utilisées par les antennes présentes dans la zone de Saint-Mars-du-Désert et comporte une interface de communication série (UART).



FIGURE 2.62 – Module de communication TEL0177

En résumé, le module TEL0177 constitue une solution pertinente pour notre système embarqué : il est compatible 4G, accepte une carte SIM, communique avec l'ESP32 via l'interface UART et prend en charge les bandes passantes nécessaires dans la zone de Saint-Mars-du-Désert (B1, B3, B7, B8, B20 et B28).

Bandes passantes du module TEL0177 :

- B1 (2100 MHz)
- B3 (1800 MHz)
- B7 (2600 MHz)
- B8 (900 MHz)
- B20 (800 MHz)
- B28 (700 MHz)

Sélection du forfait mobile

La dernière étape pour la mise en place de la solution de communication mobile est de choisir un forfait mobile adapté à notre module de communication 4G TEL0177 et à nos besoins en termes de volume de données mensuel.

Malheureusement, par manque de temps, nous n'avons pas pu finaliser cette étape ni sélectionner un forfait mobile adapté. Nous avons néanmoins envisagé, dans un premier temps, de réaliser des tests en utilisant nos propres forfaits et donc nos propres cartes SIM.

Cette solution n'a toutefois pas pu être testée, car les problèmes de commande de matériel ont entraîné une réception très tardive des équipements. Cela ne nous a donc pas laissé suffisamment de temps pour tester la solution avec nos propres forfaits mobiles.

En résumé, le développement de la solution de communication mobile n'a pas été finalisé.

2.5.4 Ce qu'il faut retenir

Ce qu'il faut retenir de cette partie du projet est que la communication sans fil utilisant le Wi-Fi a été mise en place, testée et validée sur un prototype, en concordance avec les autres work packages. Nous sommes ainsi capables de transmettre automatiquement une image et les métadonnées associées vers un serveur web distant dès qu'une détection est effectuée par le capteur PIR.

En revanche, la communication sans fil utilisant la 4G n'a pas pu être finalisée en raison de problèmes de commande de matériel, qui ont entraîné une réception tardive des équipements nécessaires au développement et aux tests. Cependant, une analyse comparative des différentes technologies de communication mobile a été réalisée, ainsi que la sélection d'un module de communication 4G adapté à notre projet.

2.6 WP 2.1 — Stockage distant (responsable : Davy Clark)

2.6.1 Rappel des objectifs

Le work package WP 2.1 a pour objectif de mettre en place une solution de stockage distant permettant de centraliser les données issues du piège caméra, d'en assurer la conservation et d'en faciliter l'exploitation.

Les objectifs principaux sont les suivants :

- recevoir automatiquement les images capturées par le piège caméra ;
- enregistrer chaque image avec ses métadonnées associées (date, heure, température, localisation GPS, type de détection, etc.) ;
- permettre l'accès aux données en lecture, en consultation en ligne et en téléchargement ;
- produire automatiquement des statistiques sur les données collectées, notamment sur l'occurrence des animaux détectés.

Les tâches associées à ce work package sont organisées comme suit :

1. Spécification du work package

- (a) Définir les objectifs du stockage distant.
- (b) Identifier les besoins en capacité et en sécurité.
- (c) Choisir le support d'hébergement le plus adapté (cloud, NAS ou serveur dédié).

2. Conception de la structure des données

- (a) Définir les informations principales à conserver.
- (b) Définir les relations entre les différentes données.
- (c) Définir les champs et les types de données.
- (d) Concevoir l'organisation générale du stockage.

3. Développement des interfaces

- (a) Créer l'interface de réception des données.
- (b) Créer l'interface de traitement et de tri.
- (c) Créer l'interface de consultation et d'export.
- (d) Créer l'interface d'administration (utilisateurs, journaux, supervision).

4. Maintenance et documentation

- (a) Rédiger la documentation technique.
- (b) Former les utilisateurs.
- (c) Mettre en place la supervision et la maintenance.

2.6.2 Planification du projet

La capacité de stockage visée pour cette première version est de l'ordre de 100 images. Ce dimensionnement a permis de définir une solution simple à déployer, tout en restant suffisante pour les démonstrations et les premiers essais du système.

Un premier calcul empirique a été réalisé à partir de l'ensemble des images reçues lors des différents tests. La taille moyenne observée pour une image est d'environ 1,04 Mo. Sur cette base, une capacité de 100 images représente un volume total estimé à 104 Mo, ce

qui reste compatible avec une architecture matérielle simple pour une première version du système.

En retenant une moyenne de 20 images reçues par jour, la croissance journalière du stockage peut être estimée à 20,8 Mo par jour. Dans ces conditions, une autonomie d'environ un mois nécessite au minimum 624 Mo d'espace disponible. Il conviendra de définir un seuil d'alerte afin d'anticiper une saturation du disque et de déclencher, si nécessaire, une action de maintenance ou d'archivage.

Le support retenu pour cette phase est un serveur dédié. Ce choix permet de centraliser les données et de maîtriser l'architecture logicielle mise en place pour la réception, le traitement et la mise à disposition des informations. Le flux général du stockage distant est représenté sur la figure 2.63.

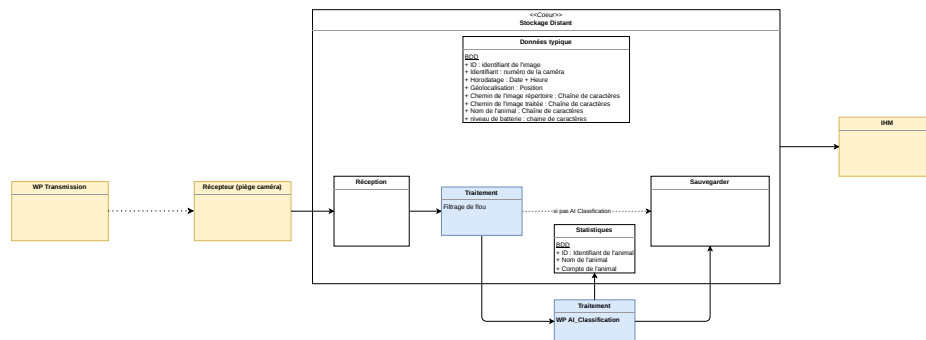


FIGURE 2.63 – Vue d'ensemble du flux de stockage distant

2.6.3 Conception de la structure des données

D'après la figure de spécification (figure 2.63), on peut déduire la structure principale des données à conserver pour chaque image.

TABLE 2.13 – Structure des données stockées pour chaque image

Indice	Champ	Type
0	ID	Entier
1	DATE_SERVER	Date/heure
2	DATE_TRAP	Date/heure
3	GEOLOCALISATION_LAT	Texte
4	GEOLOCALISATION_LONG	Texte
5	TEMPERATURE	Texte
6	HUMIDITE	Texte
7	IMAGE_REPERTOIRE	Texte
8	IMAGE_TRAITEE	Texte
9	NOM_ANIMAL	Texte
10	ID_ANIMAL	Entier
11	BATTERIE	Texte
12	CAMERA_ID	Entier
13	ETAT	Entier
14	CONFIANCE	Texte

Cette structure regroupe les informations de réception, les métadonnées environnementales, les données de localisation ainsi que les résultats de traitement associés à chaque image.

2.6.4 Solution envisagée

La solution retenue repose sur une architecture web permettant de recevoir les données envoyées par le piège caméra, de les traiter, puis de les enregistrer dans un espace de stockage centralisé. Les choix principaux sont les suivants :

- utilisation d’une interface web comme support global d’exploitation ;
- utilisation du protocole HTTP pour les échanges entre les modules ;
- enregistrement des données dans une base de données SQL, organisée de façon à être directement exploitable par l’application.

Cette approche permet de séparer clairement la réception des données, leur traitement et leur mise à disposition à travers une interface unique.

2.6.5 Réalisation

Méthodologie

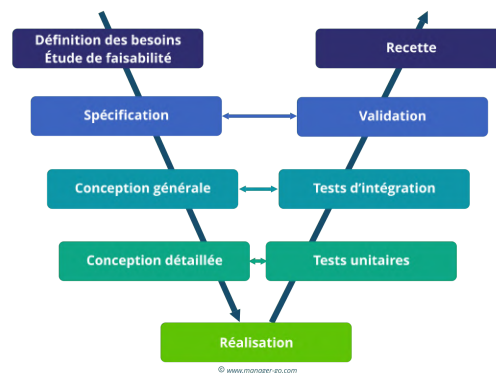


FIGURE 2.64 – Schéma de la méthode en cycle en V

La méthodologie suivie pour la réalisation du stockage distant repose sur une décomposition en sous-fonctions correspondant aux principales fonctionnalités attendues du système. Chaque sous-fonction a été développée, testée indépendamment, puis intégrée progressivement dans l’ensemble, conformément à la démarche du cycle en V illustrée sur la figure 2.64.

Réception des données

Cette partie est en lien direct avec le WP Bis (chapitre WP Bis). Le format des données échangées est présenté en Annexe A.3.

La réception s’effectue en deux étapes successives. Le microcontrôleur transmet d’abord les fragments de l’image via une requête HTTP POST vers `/api/image`, avec un contenu binaire. Ces fragments sont placés dans un buffer temporaire, pris en charge par un thread dédié afin de ne pas bloquer la réception. Une seconde requête HTTP POST est

ensuite envoyée vers `/api/data` pour transmettre les métadonnées associées, sous un format structuré de type **JSON**. Cette deuxième étape déclenche la vérification de la cohérence entre l'image et ses métadonnées, la reconstruction du buffer, puis le traitement applicatif.

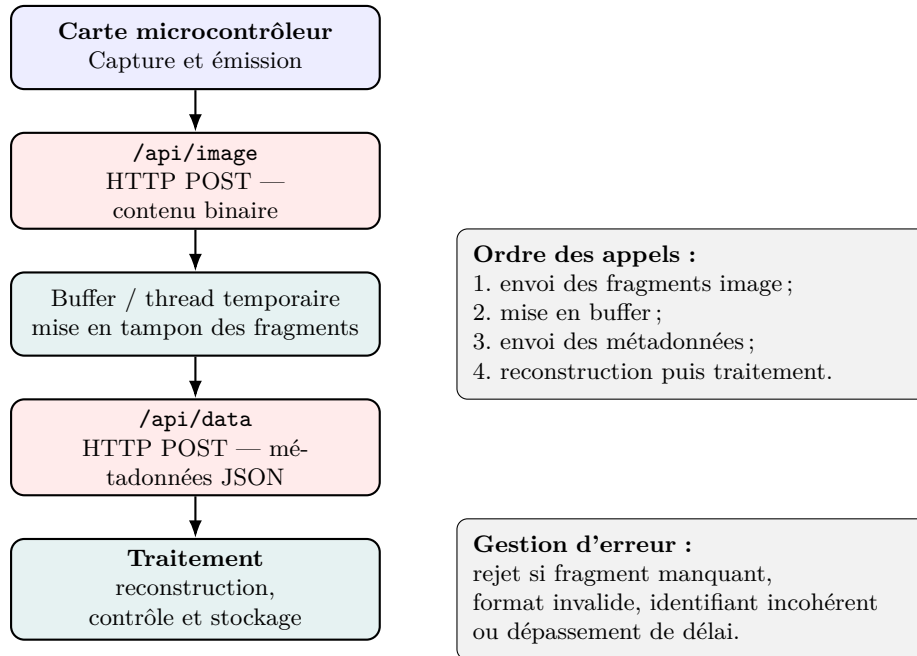


FIGURE 2.65 – Diagramme technique du flux de réception et de traitement

En cas d'anomalie, plusieurs situations sont prises en compte : fragment manquant, ordre de réception incohérent, format invalide, identifiant de capture non reconnu ou dépassement d'un délai d'attente. Dans ces cas, la requête est rejetée, l'événement est journalisé et les données incomplètes sont écartées afin d'éviter l'enregistrement d'un jeu de données corrompu.

Stockage

Le stockage est organisé autour d'un schéma relationnel articulé autour de cinq tables : **MAIN**, **IA**, **CAMERA**, **JOURNAL** et **STATS**. La table **MAIN** constitue le noyau du modèle ; elle regroupe les captures reçues et leurs métadonnées, dont la structure détaillée est présentée dans le tableau 2.13. Les autres tables viennent compléter cette structure pour le suivi matériel, l'analyse automatique, la journalisation des événements et la production de statistiques.

Les clés primaires retenues sont les champs **ID** de chacune des tables. La table **MAIN** référence **CAMERA** via **CAMERA_ID**, ce qui permet de rattacher chaque capture à un équipement identifié. La table **IA** référence **MAIN** via **IMAGE_ID** afin d'associer les résultats d'analyse à une image donnée. La table **JOURNAL** peut, selon le type d'événement, référencer à la fois une caméra via **CAM_ID** et une image via **IMAGE_ID**. La table **STATS** est une table d'agrégation destinée à stocker les comptages par nom d'animal.

La table **JOURNAL** assure la traçabilité du système. Elle enregistre les événements significatifs : réception d'une image, erreur de transmission, échec de reconstruction, incident applicatif, tentative d'accès non autorisée ou action d'administration. Les champs

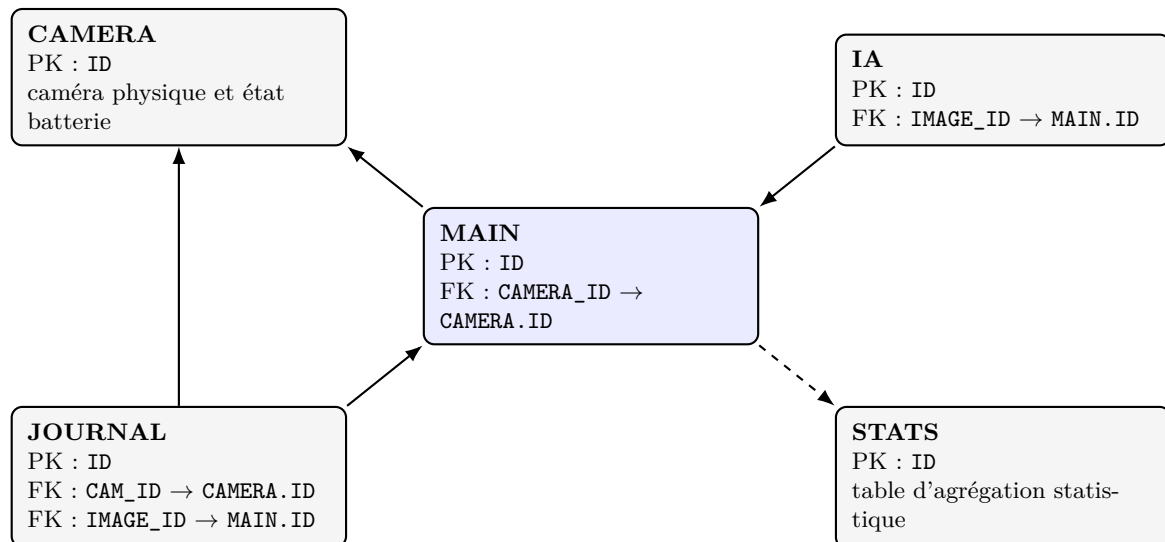


FIGURE 2.66 – Schéma relationnel simplifié du stockage distant

EVENT_TYPE, DESCRIPTION et TIMESTAMP permettent de qualifier, décrire et dater chaque événement. Lorsque l'information est disponible, les références CAM_ID et IMAGE_ID le rattachent à une caméra ou une capture précise. Cette table constitue un support essentiel de supervision, de diagnostic et d'audit.

La table CAMERA permet de suivre l'état des équipements dans le temps. Outre l'identifiant de la caméra, elle conserve le niveau de batterie, la dernière position connue et la date de mise à jour associée. Elle ne se limite donc pas à une valeur instantanée : elle retrace l'évolution de l'état de chaque caméra au fil des remontées reçues, ce qui est particulièrement utile pour détecter une baisse d'autonomie ou anticiper une intervention de maintenance.

Le champ ETAT de la table MAIN constitue un registre d'état de l'image. La valeur 0 indique qu'aucune anomalie n'a été détectée ; la valeur 1 signale une image floue ; la valeur 2 indique la présence d'un humain dans l'image. Lorsque ce champ est différent de 0, l'image est exclue des résultats principaux et isolée dans une vue dédiée, accessible uniquement aux rôles autorisés.

Hébergement

Plusieurs solutions d'hébergement ont été envisagées au cours du projet, de façon progressive. Dans un premier temps, l'application a été hébergée localement sur un ordinateur afin de faciliter le développement et les essais initiaux. Une seconde solution a consisté à l'exposer depuis un réseau domestique avec ouverture de port sur le routeur. Enfin, un hébergement sur un serveur AWS a été retenu lorsque cette ouverture de port n'a plus été possible en raison des contraintes imposées par l'opérateur.

Fonctions annexes

Sécurité et gestion des accès Dans un premier temps, les aspects de sécurité n'ont pas été traités, car ils n'étaient pas considérés comme prioritaires pour les essais initiaux. Toutefois, la question de la confidentialité s'est rapidement imposée, certaines images pouvant contenir des personnes. Un mécanisme d'authentification a donc été mis en

place afin de restreindre l'accès aux données aux seuls utilisateurs autorisés. Des mesures complémentaires ont ensuite été ajoutées pour limiter les points d'entrée exposés et réduire le risque d'envoi abusif de données pouvant saturer le stockage.

Au-delà de cette authentification, le système met en œuvre trois rôles distincts — **user**, **dev** et **admin** — afin de contrôler les droits d'accès selon le profil d'utilisation. Le rôle **user** dispose d'un accès en lecture aux données courantes. Le rôle **dev** permet en outre d'accéder aux images filtrées et au journal d'événements. Le rôle **admin** dispose de l'ensemble des droits, y compris la gestion des comptes utilisateurs.

Export des données Une fonctionnalité d'export complète le stockage distant afin de permettre la restitution des données collectées dans un format directement exploitable. La route `/export` produit un fichier **XLSX** accompagné d'une archive **ZIP** contenant les images associées. En l'absence de la bibliothèque `openpyxl`, un export de secours au format **CSV** est automatiquement proposé.

2.6.6 Limites actuelles et axes d'amélioration

Le tableau suivant synthétise les principaux points de vigilance identifiés ainsi que les actions envisagées pour y remédier.

TABLE 2.15 – Synthèse des limites actuelles et des actions prévues

Limite actuelle	Impact	Action prévue
Perte réseau	Une transmission peut être interrompue avant la réception complète de l'image ou des métadonnées.	Ajouter un contrôle de complétude, des temporisations et des mécanismes de reprise sur erreur.
Ordre des paquets ou des appels	Un buffer incomplet ou des appels reçus dans le mauvais ordre peuvent produire des données incohérentes.	Associer un identifiant unique à chaque envoi et vérifier explicitement l'ordre attendu des fragments et des appels.
Faux positifs de l'IA	Les résultats de classification peuvent fausser les statistiques si les prédictions sont peu fiables.	Définir des seuils de confiance, conserver les scores détaillés et prévoir une validation humaine sur les cas ambigus.
Saturation du stockage	L'augmentation du nombre d'images ou de la durée d'exploitation peut conduire à un manque d'espace disque.	Mettre en place une supervision du volume occupé, un seuil d'alerte et une stratégie d'archivage ou de purge.

2.6.7 Tests et validation

Transmission des données avec le protocole UDP

Les essais UDP ont été conduits en collaboration avec le WP Transmission (dirigé par Joris), en trois étapes progressives allant de la transmission de caractères simples jusqu'à la tentative d'envoi d'une image complète.

Résultats Étape 1 : Transmission de caractères ASCII. Un premier test a été effectué en émettant des caractères ASCII depuis la carte microcontrôleur. La réception a été assurée par le script `recepteur_udp.py` côté serveur. Les caractères reçus sont conformes à ceux qui ont été émis. **Validation :** la transmission simple de données textuelles fonctionne correctement.

Étape 2 : Transmission de fragments d'image. L'étape suivante a consisté à envoyer des paquets de données représentant des fragments d'image. Le même script de réception a été réutilisé sans modification majeure, et les transmissions se sont déroulées sans erreur. **Validation :** la transmission par paquets UDP est fonctionnelle.

Étape 3 : Transmission d'image complète. L'étape finale visait la reconstruction d'une image complète à partir des paquets UDP reçus. La faisabilité a été étudiée, mais la reconstruction fiable s'est avérée plus complexe à garantir qu'en HTTP, en raison de l'absence de mécanisme natif de contrôle des pertes et de l'ordre de livraison.

TABLE 2.17 – Résultats synthétiques des essais de transmission UDP

Test	Attendu	Obtenu	Statut
Transmission de caractères ASCII	Vérifier la communication de base	Caractères reçus conformes à ceux émis	Validé
Transmission de fragments d'image	Vérifier la réception de paquets binaires	Fragments reçus sans erreur majeure	Validé
Transmission d'image complète	Vérifier la reconstruction complète	Faisabilité étudiée ; reconstruction fiable non garantie	Partiel
Robustesse du protocole	Analyser les limites face aux pertes	UDP rapide mais nécessite une gestion applicative supplémentaire	Limité

Bilan

Remarques Le protocole UDP peut présenter un avantage de vitesse pour des flux continus (vidéo notamment), mais son implémentation est plus complexe pour assurer une reconstruction fiable des données. Pour les besoins du projet, la solution HTTP avec requête POST s'est révélée plus adaptée. Pour la phase de test, il serait par ailleurs préférable d'utiliser un switch ou un routeur pour relier directement la carte microcontrôleur au serveur, plutôt qu'un partage de connexion mobile.

Transmission des données avec le protocole HTTP

Les essais HTTP ont été conduits selon la même progression que pour UDP, en trois étapes allant de la transmission de données simples jusqu'à l'envoi complet d'une image accompagnée de ses métadonnées.

Résultats Étape 1 : Transmission de données textuelles. Un premier test a consisté à envoyer des données textuelles simples afin de valider la communication entre

l'émetteur et le serveur. Les données reçues correspondent aux valeurs attendues. **Validation** : la transmission de données simples via HTTP fonctionne correctement.

Étape 2 : Transmission de fragments d'image. Des fragments binaires correspondant au contenu de l'image ont été transmis via les points d'entrée décrits dans la partie réalisation. Ils ont été correctement reçus et intégrés dans le flux de traitement. **Validation** : la transmission de blocs de données via HTTP est fonctionnelle.

Étape 3 : Transmission complète de l'image et des métadonnées. L'image complète et ses métadonnées ont été transmises conjointement, permettant de finaliser le traitement côté serveur et d'enregistrer directement les données dans la structure prévue. **Validation** : HTTP constitue une solution adaptée, robuste et simple à intégrer dans l'architecture globale du projet.

TABLE 2.19 – Résultats synthétiques des essais de transmission HTTP

Test	Attendu	Obtenu	Statut
Transmission de données textuelles	Vérifier la communication HTTP	Données reçues conformes aux valeurs attendues	Validé
Transmission de fragments d'image	Vérifier la réception de blocs binaires	Fragments reçus et intégrés dans le flux de traitement	Validé
Transmission complète image + métadonnées	Vérifier l'envoi complet	Image et métadonnées transmises et traitées	Validé
Simplicité d'intégration	Vérifier l'adéquation avec le stockage	Intégration plus simple et plus robuste qu'UDP	Validé

Bilan

Remarques L'utilisation de HTTP simplifie le développement grâce à une meilleure structuration des échanges entre client et serveur. L'envoi par requête POST est particulièrement adapté à la transmission conjointe d'images et de métadonnées. HTTP apparaît ainsi comme le protocole le plus pertinent pour garantir une intégration simple et fiable dans l'architecture globale du projet.

2.7 WP 2.2 — Interface web (responsable : Xueying Xu)

2.7.1 Rappel des objectifs

Le work package WP2.2 correspond à l'interface web du système. Son objectif principal est de permettre à l'utilisateur de consulter et gérer les données collectées par le piège caméra de manière simple et accessible via un navigateur web. Plus concrètement, les tâches associées sont :

- Afficher les images capturées par le module caméra
- Permettre un accès au système via une interface web (adresse IP / site)
- Associer aux images certaines informations (date, heure, etc.)
- Visualiser l'état du système (caméra, batterie, etc.)
- Proposer différentes fonctionnalités (visualisation des images, gestion des caméras, journal système, etc.)
- Offrir une interface claire et intuitive pour l'utilisateur

Cette interface constitue le lien entre le système embarqué et l'utilisateur final.

2.7.2 Solution envisagée

Afin de répondre aux objectifs du WP2.2, une solution basée sur une interface web légère a été mise en place.

Cette interface a été développée en utilisant le framework Python **Flask**, qui permet de créer rapidement un serveur web capable de communiquer avec le système embarqué.

Fonctionnalités principales L'interface web développée ne se limite pas à un simple affichage d'images, mais propose un ensemble de fonctionnalités permettant la gestion et l'exploitation des données collectées.

- **Visualisation des images** : Les images capturées sont affichées sous forme de galerie, accompagnées de métadonnées telles que la date, la température et l'humidité.
- **Filtrage et tri des données** : L'utilisateur peut filtrer les images selon différents critères (date, identifiant de caméra) et trier les résultats (par exemple du plus récent au plus ancien).
- **Gestion des données** : Des actions simples sont disponibles, telles que la suppression ou le masquage d'images, permettant de gérer efficacement les données stockées.
- **Export des données** : Une fonctionnalité d'export permet de générer un fichier au format CSV ou XLSX contenant les informations associées aux images. Cela facilite l'analyse externe des données.
- **Journal système** : Un journal permet de suivre les événements du système (réception d'image, suppression, etc.), assurant une traçabilité des actions.
- **Interface multi-pages** : L'application est organisée en plusieurs pages (images, carte, journal, batterie), facilitant la navigation et la structuration des informations.

La chaîne de fonctionnement de l'interface web est illustrée ci-dessous.

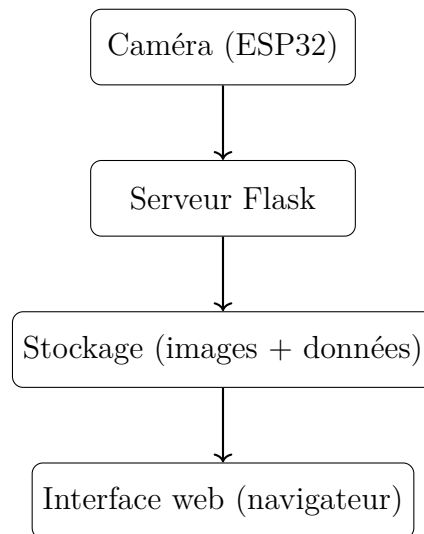


FIGURE 2.67 – Architecture simplifiée de l'interface web

L'interface web permet plusieurs fonctionnalités essentielles :

- visualisation des images capturées par la caméra ;
- affichage des informations associées (date, heure, etc.) ;
- consultation de l'état du système (connexion, activité) ;
- navigation simple entre les différentes pages ;
- accès au système via une adresse IP ou un site web.

Une attention particulière a été portée à la simplicité de l'interface afin de garantir une utilisation intuitive, même pour un utilisateur non technique.

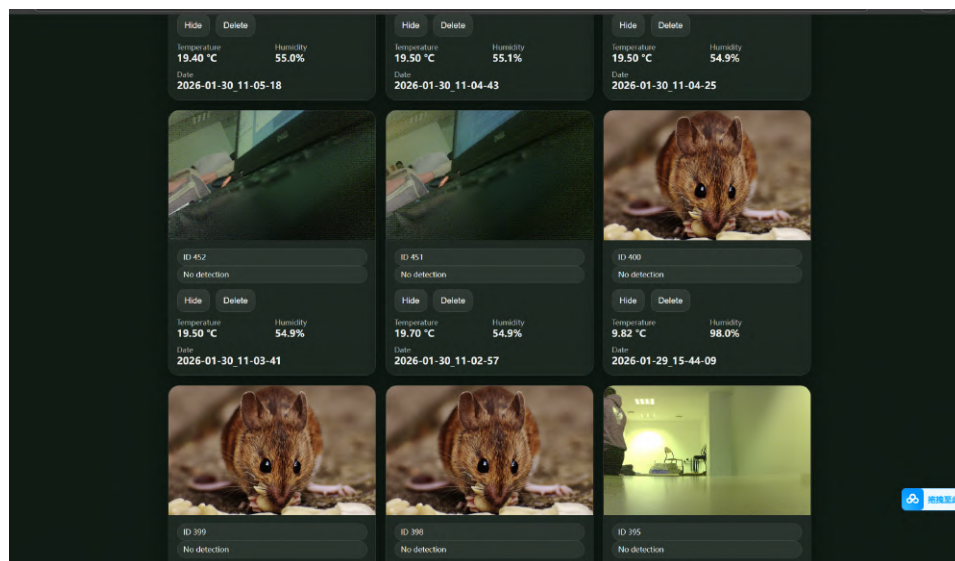


FIGURE 2.68 – Exemple d'interface web développée pour le projet

2.7.3 Tests et Validation

Afin de valider le bon fonctionnement de l'interface web ainsi que son intégration avec les autres modules du système (notamment WP1.2 et WP1.3), plusieurs tests fonctionnels ont été réalisés.

Accès et déploiement Dans un premier temps, l'application a été déployée en local à l'aide du serveur Flask. L'interface est accessible via une adresse IP locale (127.0.0.1 :5000), ce qui permet de simuler le fonctionnement du système sans infrastructure externe. Les tests montrent que le serveur démarre correctement et que les différentes pages sont accessibles sans erreur.

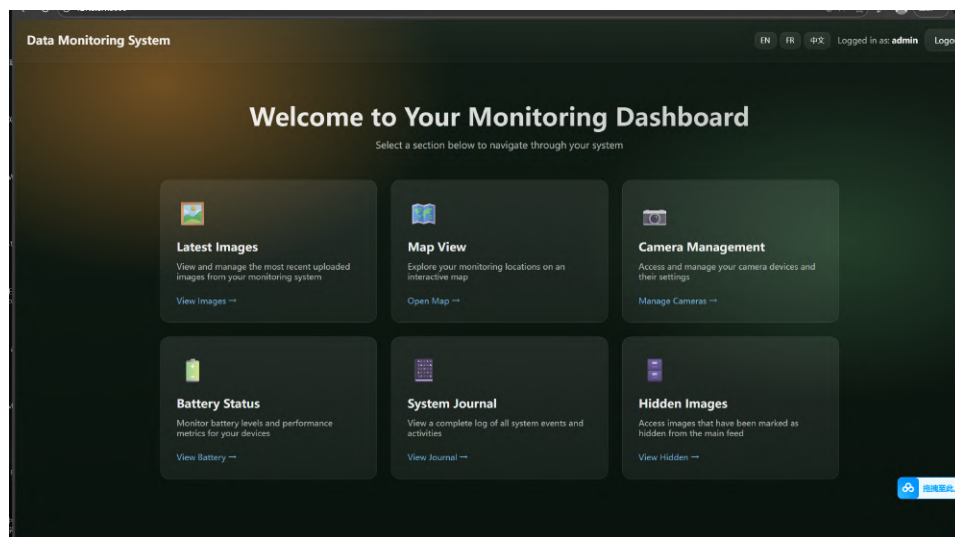


FIGURE 2.69 – Page d'accueil de l'interface web (dashboard)

Affichage et gestion des images Des tests ont été réalisés pour vérifier la réception et l'affichage des images issues du module caméra. Les résultats montrent que les images sont correctement chargées et affichées sous forme de galerie, accompagnées de leurs métadonnées (date, température, humidité, identifiant).

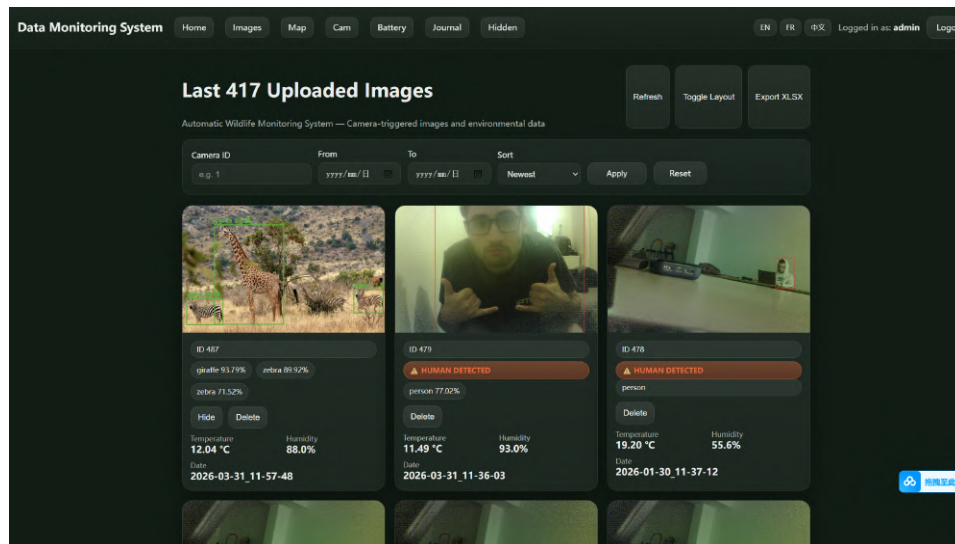


FIGURE 2.70 – Visualisation des images avec métadonnées et détection

Des fonctionnalités de gestion ont également été validées, notamment :

- suppression d'images ;
- masquage (hidden) de certaines images ;
- mise en évidence automatique de certaines détections (ex : présence humaine).

Filtrage et organisation des données L'interface permet de filtrer et trier les images selon différents critères (date, identifiant de caméra, ordre chronologique).

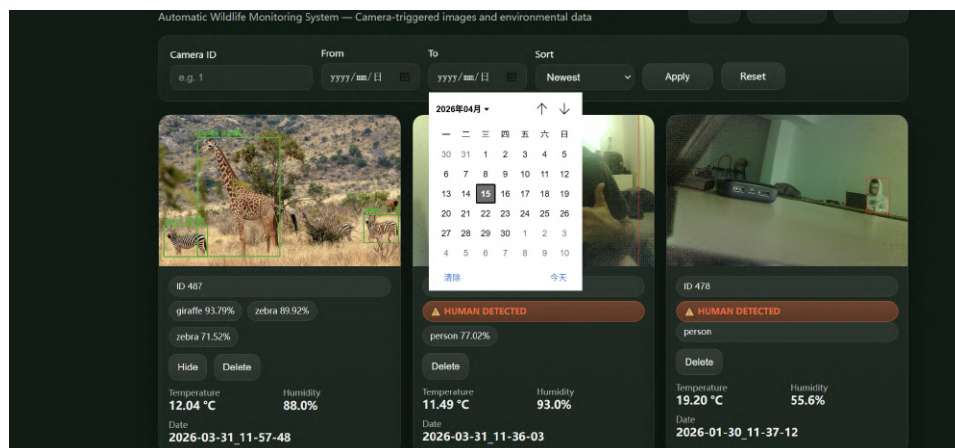


FIGURE 2.71 – Interface de filtrage et tri des images

Les tests montrent que ces fonctionnalités permettent d'améliorer significativement l'accès aux données, en particulier lorsque le nombre d'images devient important.

Export des données Une fonctionnalité d'export a été testée, permettant de générer un fichier contenant les informations associées aux images (format CSV ou XLSX).

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W
	ID	DATE	TYPE	ORIGIN	GEOLOGICAL	TIME	DEPTH	DEPTH	DEPTH	DEPTH	DEPTH	DEPTH	DEPTH	DEPTH	DEPTH	DEPTH	DEPTH	DEPTH	DEPTH	DEPTH	DEPTH	DEPTH	DEPTH
1	447	2020-03-01	30-02020-03-01	47.28119	-1.51599	12.04	88	images/2020/images/traitve	48	2	0												
2	469	2020-03-01	30-02020-03-01	47.28119	-1.51678	19.3	55.6	images/2020/images/traitve	100	1	0												
3	462	2020-03-01	30-02020-03-01	47.28119	-1.51678	19.3	55.3	images/2020/images/traitve	100	1	0												
4	461	2020-03-01	30-02020-03-01	47.28119	-1.51678	19.3	55.3	images/2020/images/traitve	100	1	0												
5	460	2020-03-01	30-02020-03-01	47.28119	-1.51678	19.3	55.3	images/2020/images/traitve	100	1	0												
6	459	2020-03-01	30-02020-03-01	47.28119	-1.51678	19.3	55.3	images/2020/images/traitve	100	1	0												
7	458	2020-03-01	30-02020-03-01	47.28119	-1.51678	19.3	55.3	images/2020/images/traitve	100	1	0												
8	457	2020-03-01	30-02020-03-01	47.28119	-1.51678	19.3	55.3	images/2020/images/traitve	100	1	0												
9	456	2020-03-01	30-02020-03-01	47.28119	-1.51678	19.3	55.3	images/2020/images/traitve	100	1	0												
10	455	2020-03-01	30-02020-03-01	47.28119	-1.51678	19.3	55.3	images/2020/images/traitve	100	1	0												
11	454	2020-03-01	30-02020-03-01	47.28119	-1.51678	19.3	55.1	images/2020/images/traitve	100	1	0												
12	453	2020-03-01	30-02020-03-01	47.28119	-1.51678	19.5	54.9	images/2020/images/traitve	100	1	0												
13	452	2020-03-01	30-02020-03-01	47.28119	-1.51678	19.5	54.9	images/2020/images/traitve	100	1	0												
14	451	2020-03-01	30-02020-03-01	47.28119	-1.51678	19.5	54.9	images/2020/images/traitve	100	1	0												
15	450	2020-03-01	30-02020-03-01	47.28119	-1.51678	19.5	54.9	images/2020/images/traitve	100	1	0												
16	449	2020-03-01	30-02020-03-01	47.28119	-1.51599	9.82	98	images/2020-01-29-13-40-00.png	5	2	0												
17	399	2020-03-01	30-02020-03-01	47.28119	-1.51599	9.82	98	images/2020-01-29-13-40-00.png	8	1	0												
18	398	2020-03-01	30-02020-03-01	47.28119	-1.51599	9.82	98	images/2020-01-29-15-42-25.png	28	2	0												
19	395	2020-03-01	30-02020-03-01	47.28119	-1.516016	13.9	84.3	images/2020-01-19-16-23-14.png	0	1	0												
20	394	2020-03-01	30-02020-03-01	47.28119	-1.516016	13.9	84.3	images/2020-01-19-16-24-24.png	0	1	0												
21	393	2020-03-01	30-02020-03-01	47.28119	-1.516016	13.8	84.8	images/2020-01-19-16-21-22.png	0	1	0												
22	392	2020-03-01	30-02020-03-01	47.28119	-1.516016	13.8	85	images/2020-01-19-16-21-09.png	0	1	0												
23	391	2020-03-01	30-02020-03-01	47.28119	-1.516016	13.9	84.5	images/2020-01-19-16-20-20.png	0	1	0												
24	390	2020-03-01	30-02020-03-01	47.28119	-1.516016	13.9	84.1	images/2020-01-19-16-19-24.png	0	1	0												
25	389	2020-03-01	30-02020-03-01	47.28119	-1.516016	14.1	83.5	images/2020-01-19-16-17-13.png	0	1	0												
26	388	2020-03-01	30-02020-03-01	47.28119	-1.516016	14.1	83.1	images/2020-01-19-16-16-36.png	3.66667	0	1												
27	387	2020-03-01	30-02020-03-01	47.28119	-1.516016	14.1	82.9	images/2020-01-19-16-16-18.png	1.83334	0	1												
28	386	2020-03-01	30-02020-03-01	47.28119	-1.516016	14.2	82.7	images/2020-01-19-16-13-18.png	0	1	0												
29	385	2020-03-01	30-02020-03-01	47.28119	-1.516016	14.2	82.5	images/2020-01-19-16-14-34.png	0	0.00004	1												
30	384	2020-03-01	30-02020-03-01	47.28119	-1.516016	14.3	82.1	images/2020-01-19-16-15-15.png	0.06667	0	1												
31	383	2020-03-01	30-02020-03-01	47.28119	-1.516016	14.3	82.5	images/2020-01-19-16-13-58.png	1.00007	0	1												
32	382	2020-03-01	30-02020-03-01	47.28119	-1.516016	14.4	82.9	images/2020-01-19-16-12-48.png	0	1	0												
33	381	2020-03-01	30-02020-03-01	47.28119	-1.516016	14.4	83.1	images/2020-01-19-16-12-30.png	0	1	0												
34	380	2020-03-01	30-02020-03-01	47.28119	-1.516016	14.3	82.7	images/2020-01-19-16-12-08.png	2.33333	1	0												

FIGURE 2.72 – Fonction d’export des données

Journal et traçabilité Le module de journalisation a été validé en vérifiant l'enregistrement des événements (réception d'image, suppression, etc.).

FIGURE 2.73 – Journal des événements du système

Visualisation cartographique Une interface de visualisation géographique a également été intégrée, permettant d'afficher la position des caméras sur une carte.

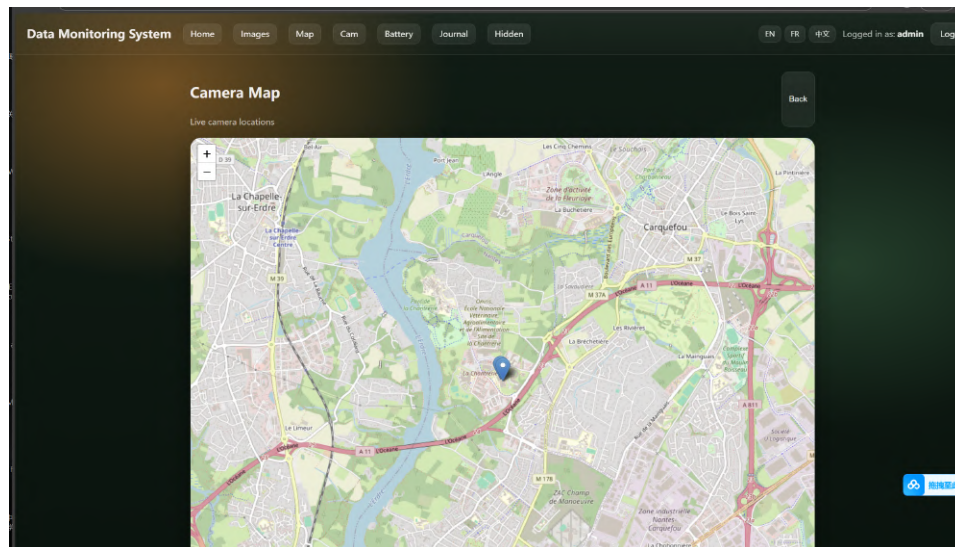


FIGURE 2.74 – Visualisation des caméras sur une carte

Cette fonctionnalité facilite la compréhension de la répartition spatiale du système.

Robustesse et utilisation Enfin, des tests de robustesse ont été réalisés via des requêtes répétées. Le système reste fonctionnel et stable dans des conditions d'utilisation normales.

L'interface a également été testée sur différents navigateurs et supports (ordinateur), confirmant sa compatibilité.

Certaines fonctionnalités liées à l'état des caméras ou de la batterie sont actuellement à un stade de prototype. Toutefois, leur intégration dans l'interface permet déjà de valider l'architecture globale du système.

Synthèse Globalement, ces tests confirment que l'interface web constitue un outil fiable pour la consultation et la gestion des données issues du système embarqué. Elle permet non seulement la visualisation des images, mais aussi leur organisation, leur export et leur traçabilité, ce qui en fait un élément central du système.

2.7.4 Ce qu'il faut retenir

Cette section a permis de valider l'interface web comme un élément central pour l'exploitation des données du système. Voici les points essentiels à retenir de ce Work Package :

- **Architecture client-serveur** : Une architecture basée sur Flask a été mise en place, permettant la réception, le stockage et la visualisation des données issues du système embarqué.
- **Validation par déploiement local** : Le déploiement en local a permis de tester l'ensemble des fonctionnalités sans dépendre d'une infrastructure externe, garantissant la reproductibilité.
- **Visualisation enrichie des données** : Les images sont affichées sous forme de galerie avec leurs métadonnées (date, température, humidité), facilitant leur interprétation.

- **Organisation et filtrage** : Des outils de tri et de filtrage permettent de gérer efficacement un grand volume de données, améliorant l’accessibilité pour l’utilisateur.
- **Fonctionnalités de gestion** : Des actions telles que la suppression, le masquage et la mise en évidence de détections permettent une gestion dynamique des images.
- **Export des données** : La possibilité d’exporter les données (CSV/XLSX) constitue un point clé pour l’analyse externe et l’exploitation scientifique.
- **Traçabilité du système** : L’intégration d’un journal permet de suivre les événements du système, apportant une dimension de diagnostic et de fiabilité.
- **Visualisation spatiale** : L’ajout d’une interface cartographique améliore la compréhension de la répartition des capteurs sur le terrain.
- **Modularité et évolutivité** : Certaines fonctionnalités (état caméra, batterie) sont encore à un stade de prototype, mais leur intégration valide l’architecture globale et ouvre la voie à des extensions futures.
- **Validation fonctionnelle globale** : L’ensemble des tests confirme que l’interface web permet une consultation fiable, structurée et exploitable des données du système.

2.8 WP 2.3 — Classification animale par IA (responsable : Ewen Le Callet)

2.8.1 Rappel des objectifs

Avant de commencer cette partie, il est important de rappeler que le concept de "Classification animale par IA" est en quelque sorte la genèse du projet transversal entier... Pour la petite histoire, nous nous amusons entre les cours de 3A à tester différents modèles de classification IA ; ce fut en quelque sorte une de nos premières rencontres avec le machine learning. Ces petites expériences nous ont tellement plu que nous en avons parlé aux professeurs et ainsi de suite jusqu'à aujourd'hui avec ce projet transversal et cette section "Classification animale par IA".

Il est important à souligner que cette appétence vient aussi de mon stage de 3A au Zoo des Sables d'Olonne. En effet, on m'avait demandé de pouvoir identifier les écureuils roux des écureuils gris d'Amérique (espèce invasive en Europe, affaiblissant la population d'écureuil roux), encore un exemple de notre intérêt pour ce domaine. Ainsi, cela nous tenait vraiment à cœur que ce soit une partie intégrante du projet...



FIGURE 2.75 – Image piège caméra du Zoo des Sables d'olonne.

Bien que nous étions très excités par cette fonctionnalité, nous avons dû la faire passer en fonctionnalités optionnelles. En effet, il était logique de d'abord se focaliser sur la conception totale du piège caméra avant de rajouter une feature comme celle-là. Cependant, nous avons réussi à trouver le temps pour le faire et nous considérons que cela apporte une grande valeur ajoutée au projet.

Afin de résumer les objectifs, ces derniers collaient totalement avec ceux de l'ABC : aider et automatiser la classification animale des images capturées par notre piège caméra, permettant également d'écarter automatiquement les images ne contenant aucune détection... Le but est encore une fois d'aider les naturalistes.

Localisation de l'inférence

Nous avons également soulevé la question dans les premières semaines du projet de l'endroit où effectuer l'inférence IA dans le processus du piège caméra (sur le piège en

direct vs sur le serveur).

TABLE 2.21 – Analyse comparative de l'emplacement de l'inférence

Solution	Avantages	Défauts
Sur le piège (In-place)	Gain de bande passante (envoi uniquement des données utiles).	Environnement contraint, complexité logicielle, consommation batterie.
Sur le serveur	Aucune contrainte de puissance, facilité de mise à jour des modèles.	Dépendance à l'envoi de l'image complète sur le cloud.

Bien que nous ayons repéré avec le client quelques possibilités de le faire in-place (notamment à l'aide de microcontrôleurs tels que Google Coral), cette idée a été écartée pour des raisons de simplification de la preuve de concept... En effet, sur le serveur nous n'aurons presque aucune contrainte alors que sur la partie embarquée, on est en environnement contraint...

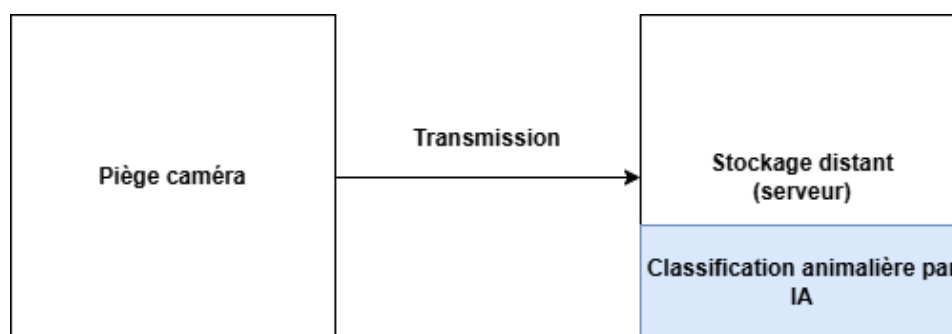


FIGURE 2.76 – Emplacement du WP "Classification animale par IA" dans notre architecture.

Il est important également à noter que le choix de la résolution de la caméra sélectionnée pour le WP 1.3 a été aiguillé par la capacité d'une IA de classification animale à détecter un animal... Un critère que nous avons trouvé judicieux afin de se poser des limites en termes de résolution pour la sélection de la caméra.



FIGURE 2.77 – Image de référence (1920*1080).



FIGURE 2.78 – Résolution limite (700*395).

L'utilisation d'une approche IA répond à plusieurs besoins concrets :

- **Filtrage automatique** des images ne contenant aucun animal, afin de limiter la charge sur le stockage distant et de faciliter la consultation ;
- **Détection d'humains** pour éviter de rendre publiques des images sensibles (protection de la vie privée) ;
- **Classification des espèces** présentes sur les images, dans l'objectif de produire des statistiques et des indicateurs automatiquement (comptages, répartition temporelle, etc.).

L'objectif principal est de permettre la classification (ou, a minima, la détection) des animaux présents sur les images capturées, sans intervention humaine. Les critères de succès retenus sont : la capacité à détecter la présence d'animaux, la détection d'humains pour la confidentialité, et un traitement fiable intégré au workflow.

Étant donné le caractère optionnel de ce WP, les attentes ont été cadrées de manière pragmatique : fournir une solution fonctionnelle, facilement intégrable, permettant d'obtenir rapidement des résultats exploitables (même imparfaits), plutôt que viser une performance maximale.

2.8.2 Solution envisagée

Pour la solution envisagée, grâce au choix de faire l'inférence IA côté serveur, nous avons un certain matelas en termes de puissance et de vitesse. La question était donc de trouver un modèle performant, open source et déjà entraîné sur un dataset cohérent avec notre utilisation (typiquement image de piège caméra). Ces trois critères nous ont orientés à tester différents modèles...

YOLOv8 (pré-entraîné COCO)

Un premier test a été réalisé avec YOLOv8 pré-entraîné sur COCO. Le modèle est efficace pour la détection d'humains et présente de bonnes performances d'inférence. En revanche, il reste limité pour notre cas d'usage : le nombre de classes animales disponibles est restreint et le modèle n'est pas spécialisé pour les images de pièges photographiques. *Inference time : 2.4721 s (exemple)*

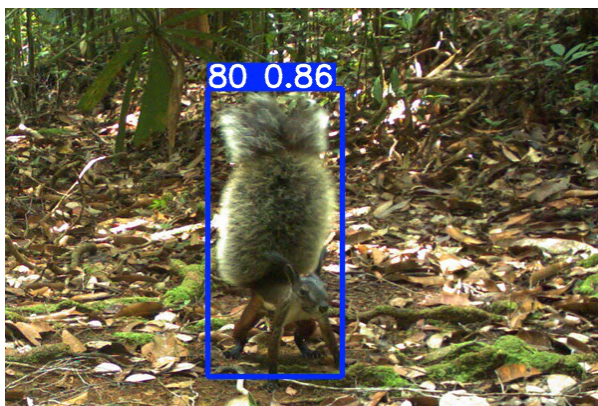


FIGURE 2.79 – Inférence effectuée sur YOLOv8 (le label 80 correspondant à "animal").

Ainsi, pour ce modèle, c'est plus le troisième critère qui pèche (entraîné sur un dataset cohérent). Une question d'effectuer nous-mêmes un entraînement s'est posée, compte

tenu du peu de ressources matérielles que nous avons (même si on aurait pu faire peut-être un entraînement LoRA).

Note sur l'entraînement LoRA : Le LoRA (*Low-Rank Adaptation*) est une technique permettant d'adapter des modèles d'IA massifs à des tâches spécifiques sans avoir à ré-entraîner tous les paramètres du modèle, ce qui permet de gagner énormément de temps et de puissance de calcul.

Cependant, nous ne trouvons pas forcément de dataset cohérent (images de piège caméra + faune ouest-européenne). hercher un bon dataset est en réalité une partie énorme du travail avec l'IA...

SpeciesNet (pipeline MegaDetector + classification)

Dans un second temps, nous avons évalué SpeciesNet, un pipeline spécialisé pour la faune. Le principe est le suivant : une première étape de détection (type MegaDetector) identifie des catégories générales (animal, humain, véhicule), puis une seconde étape effectue la classification fine des espèces lorsque la présence d'un animal est détectée.

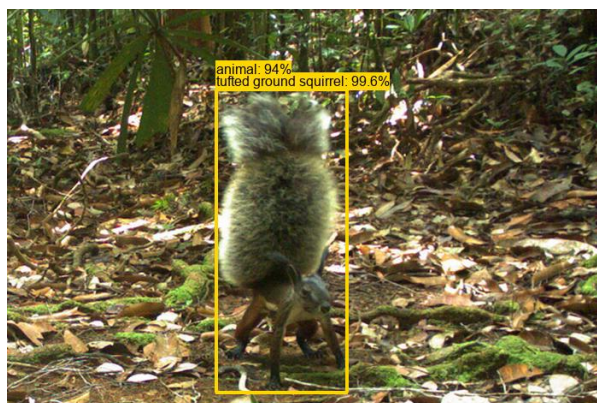


FIGURE 2.80 – Même image que celle effectuée sur YOLOv8 mais cette fois-ci inférée sur SpeciesNet.

Résumé du fonctionnement de SpeciesNet : L'ensemble SpeciesNet combine un détecteur d'objets (MegaDetector) et un classifieur d'images. Le processus suit plusieurs étapes clés :

- **Détection :** Identifie les animaux, humains ou véhicules et fournit des boîtes englobantes.
- **Classification :** Analyse l'image pour identifier l'espèce parmi une liste fixe (Top-5 scores).
- **Décisions basées sur la détection :** Si un humain ou un véhicule est détecté avec une forte confiance, ce label est prioritaire pour protéger la vie privée.
- **Décision "Blank" :** Si le modèle est très confiant sur l'absence d'animal (score > 0.99), l'image est classée comme vide.
- **Rollup taxonomique :** Si le modèle n'est pas sûr de l'espèce exacte, il remonte dans la classification (ex : Genre ou Famille) pour garantir une prédiction fiable plutôt qu'une espèce erronée.

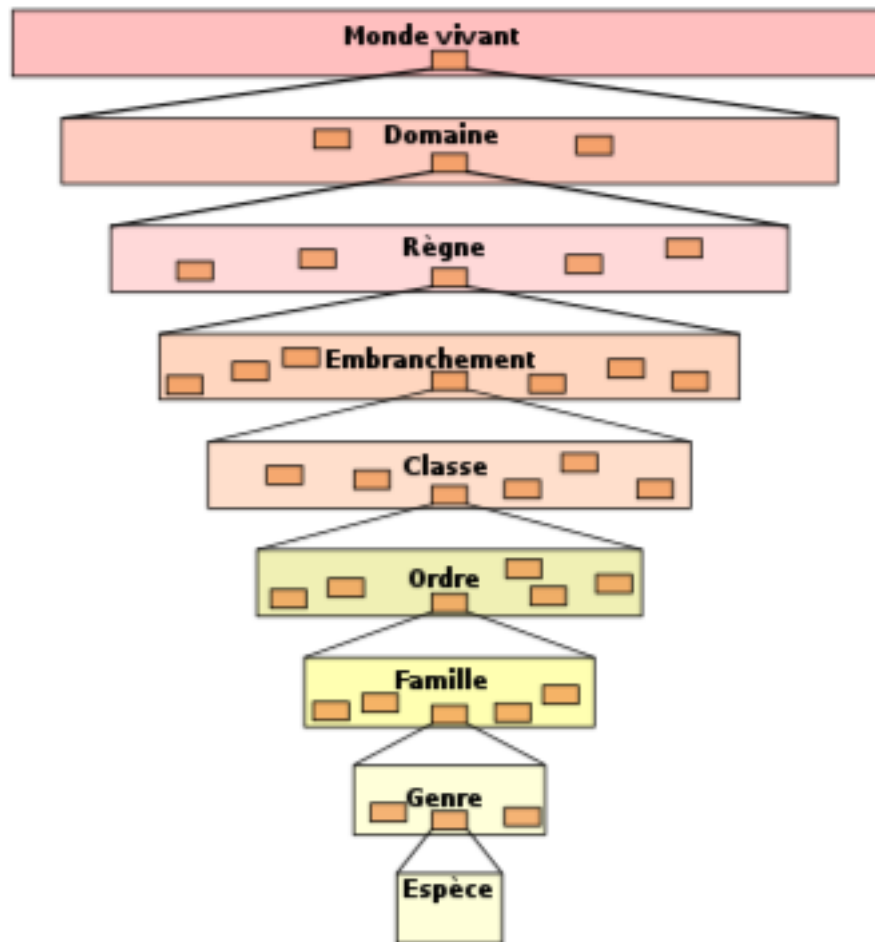


FIGURE 2.81 – Méthodologie de taxonomie suivie par le modèle SpeciesNet.

Ce modèle est très performant sur la classification, mais son intégration induit une latence plus élevée, ce qui pose des contraintes en termes de temps de traitement.

Temps d'exécution : 11.939 s (exemple)

Après avoir réfléchi, nous avons décidé de faire le choix du modèle SpeciesNet dans notre PoC finale, en mettant comme argument que le côté "temps réel" admet un temps réel souple pour notre projet...

2.8.3 Tests et Validation

La méthodologie finale appliquée dans le serveur distant est la suivante : Les images sont stockées sur le stockage distant et sont traitées automatiquement dès qu'un nouveau fichier est ajouté. Le pipeline IA s'exécute alors et produit une liste d'objets et un état global.

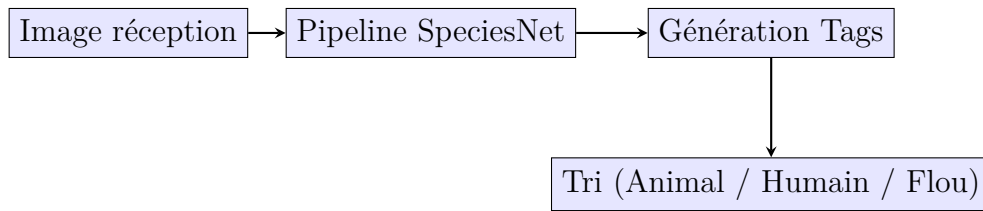


FIGURE 2.82 – Schéma résumé du traitement de l'image par l'IA.

Le format de sortie peut se résumer ainsi :

- **objets** : liste de dictionnaires (`NOM_ANIMAL`, `CONFIANCE`).
- **meta** : dictionnaire (`ETAT` : 0-OK, 1-Flou, 2-Humain, `IMAGE_TRAITEE`).

Malheureusement nous n'avons toujours pas pu tester notre solution fonctionnelle totale sur des vrais animaux, mais les tests faits dans la rue Ireste démontrent bien la qualité de la détection...



FIGURE 2.83 – Image tirée du site web, déplacée dans la section "indésirable" suite à la détection d'un humain.

2.8.4 Ce qu'il faut retenir

Cette section a permis de valider l'intégration de l'intelligence artificielle comme une brique logicielle à forte valeur ajoutée pour le projet. Voici les points essentiels à retenir de ce Work Package :

- **Genèse passionnelle** : Le projet est né d'une volonté d'appliquer le *machine learning* à la conservation de la faune sauvage (expérience au Zoo des Sables d'Olonne).
- **Arbitrage technique** : Le choix d'une **inférence côté serveur** a été privilégié pour s'affranchir des contraintes matérielles de l'embarqué (puissance, batterie) et permettre l'utilisation de modèles plus robustes.

- **Critère de résolution** : La sélection de la caméra (WP 1.3) a été validée par des tests d'IA, fixant une limite de résolution de **700x395 pixels** pour garantir une détection fiable.
- **Solution retenue (SpeciesNet)** : Malgré un temps d'exécution plus long (~12s), ce pipeline spécialisé a été choisi pour sa précision et sa capacité de **rollup taxonomique** (classification par famille si l'espèce est incertaine).
- **Fonctionnalités "Métier"** :
 - *Filtrage automatique* des images vides (gain de stockage).
 - *Protection de la vie privée* par la détection et l'écartement systématique des images contenant des humains.
 - *Prétraitement naturaliste* pour l'ABC (comptage et statistiques).
- **Validation** : Bien que les tests sur faune sauvage restent à réaliser, la preuve de concept est fonctionnelle et le pipeline de tri automatique est opérationnel sur le serveur.

3

Bilan du projet

3.1 Industrialisation du Projet : Du Prototype à la Production en Série

L'industrialisation représente une étape essentielle dans la transformation d'un prototype fonctionnel en un produit commercialisable à grande échelle. Dans notre projet, cette transition consiste à adapter notre piège caméra, initialement conçu dans un cadre académique, afin de répondre aux contraintes réelles de production : reproductibilité, fiabilité, réduction des coûts et capacité à produire en volume.

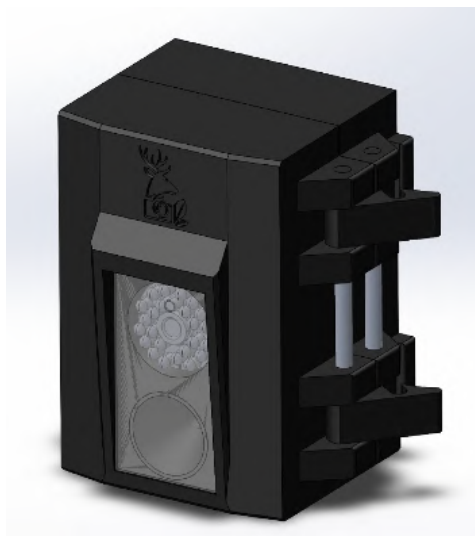


FIGURE 3.1 – Version 3D finale de notre PoC.

Cette évolution ne se limite pas à une simple duplication du prototype. Elle implique une réflexion globale sur l'organisation du travail, la conception technique du produit ainsi que sur son modèle économique.

3.1.1 Organisation Industrielle

Dans un contexte industriel, la réalisation du produit repose sur une structuration en plusieurs pôles complémentaires. Le bureau d'études joue un rôle central en assurant l'optimisation du prototype. Il ne s'agit plus seulement de faire fonctionner le système,

mais de le rendre facilement fabriquable. Cette démarche, inspirée des principes du *Design for Manufacturing*, nous a déjà conduit dans notre PTRANS à essayer d'intégrer l'ensemble des composants électroniques sur une carte unique (PCB), limitant ainsi les erreurs de câblage et réduisant le temps d'assemblage. De la même manière, le boîtier pourrait être repensé pour une fabrication en série, typiquement par injection plastique, garantissant robustesse et étanchéité.

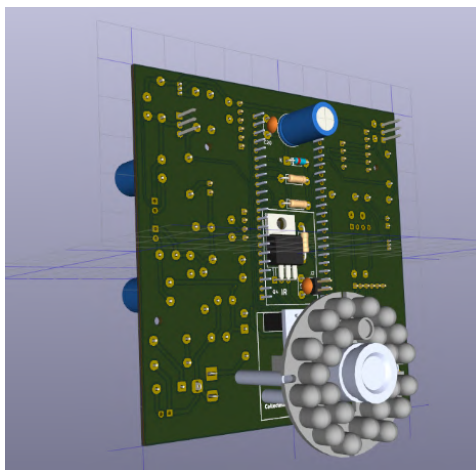


FIGURE 3.2 – Visualisation 3D de notre PCB avec les composants.

Une fois la conception stabilisée, la production est réalisée en chaîne. L'assemblage du produit est découpé en étapes successives, chacune étant effectuée par des opérateurs spécialisés. Cette organisation permet de diminuer le temps de fabrication par unité tout en assurant une qualité constante. Parallèlement, une gestion rigoureuse des approvisionnements est nécessaire afin de garantir la disponibilité des composants et d'éviter toute interruption de production. Les achats en grande quantité deviennent alors un levier majeur de réduction des coûts.

Enfin, contrairement au prototype reposant sur une preuve de concept, l'industrialisation nécessite une infrastructure numérique robuste. Le stockage des données, la gestion des serveurs ainsi que le maintien de l'interface web doivent être assurés de manière fiable pour un grand nombre d'utilisateurs, ce qui implique l'intervention de compétences spécifiques en développement et en administration système. En effet, nous utilisons pour le PTRANS une location gratuite de serveur AWS. Cependant, cette location gratuite ne l'est que selon un certain nombre de requête. En imaginant un nombre important de piège caméra produit et donc un nombre important d'utilisateurs sur l'interface web, il est important de prendre en considération ce coût supplémentaire.



FIGURE 3.3 – Serveur AWS (datacenter Amazon).

3.1.2 Validation et Mise en Production

Avant d'atteindre une production en série, plusieurs phases de validation sont indispensables. Une première série limitée permet de vérifier la conception et les outils de fabrication. Elle est suivie d'une phase de production pilote, réalisée dans des conditions proches de l'environnement industriel réel, afin d'ajuster les procédures. Ce n'est qu'après ces validations que la production en série peut être lancée, avec des contrôles qualité systématiques garantissant la conformité des produits. On peut comprendre en résumé ces étapes :

1. **Engineering Run (ER)** : validation technique.
2. **Pilot Run (PR)** : test en conditions réelles.
3. **Serial Run (SR)** : production industrielle.

3.1.3 Analyse Économique

L'un des enjeux majeurs de l'industrialisation réside dans la maîtrise des coûts. Le prototype développé présente un coût matériel total de **239.62 € HT**, principalement en raison d'achats unitaires et de choix techniques non optimisés pour la production en série

RS														
Repere TYPO	Famille	Description	Value	Quantity	Fournisseur 1	Lien Fournisseur 1	Ref Fournisseur 1	Fabricant	Ref Fabricant	Prix 1 Unitaire	UDV	Quantité Commandé	Prix	Status
WP1	PASSIF	MOSFET N	IRF540N	2	RS	lien	907-5028	INFINEON	IRF540NLPBF	14,21	10	1	14,21	A COMMANDER
WP1	PASSIF	MOSFET P	IRF9540N	1	RS	lien	541-1225	INFINEON	IRF9540NPBF	1,48	1	3	4,44	A COMMANDER
Total												18,65		
Castorama														
Repere TYPO	Famille	Description	Value	Quantity	Fournisseur 1	Lien Fournisseur 1	Ref Fournisseur 1	Fabricant	Ref Fabricant	Prix 1 Unitaire	UDV	Quantité Commandé	Prix	Status
Construction boîtier	MATERIAUX	Barre aluminium	pleine, 8 mm ,0,5m	1	CASTORAMA	lien	3701102734356	N/A	N/A	1,71	1	1	1,71	A COMMANDER
Total												1,71		
Gotronic														
Repere TYPO	Famille	Description	Value	Quantity	Fournisseur 1	Lien Fournisseur 1	Ref Fournisseur 1	Fabricant	Ref Fabricant	Prix 1 Unitaire	UDV	Quantité Commandé	Prix	Status
Transmission	MODULE	Module 4G	TELO177	1	GOTRONIC	lien	38823	DFRobot	TELO177	23,9	1	1	23,9	A COMMANDER
Capteur	MODULE	PIR	IT15IR	1	GOTRONIC	lien	32452		IT15IR	3,9	1	1	3,9	A COMMANDER
Capteur	MODULE	GPS	ADA4415	1	GOTRONIC	lien	37028	Adafruit	PA1010D	33,9	1	1	33,9	A COMMANDER
Capteur	MODULE	BME680	BME680	1	GOTRONIC	lien	36351	Joy-It	SEN-BME680	22,9	1	1	22,9	A COMMANDER
Capteur	MODULE	Temp. Hum	DHT22	1	GOTRONIC	lien	36685	Joy-It	SEN-DHT22	8,55	1	1	8,55	A COMMANDER
CONNECTIQUE	MODULE	CONNECTIQUE	FIT0513	1	GOTRONIC	lien	35830	DFRobot	FIT0513	0,75	1	1	0,75	A COMMANDER
Total												93,9		
Amazon														
Repere TYPO	Famille	Description	Value	Quantity	Fournisseur 1	Lien Fournisseur 1	Ref Fournisseur 1	Fabricant	Ref Fabricant	Prix 1 Unitaire	UDV	Quantité Commandé	Prix	Status
Stockage	MODULE	Module SD		1	AMAZON	lien	N/A	N/A	N/A	8,99	5	1	8,99	A COMMANDER
Stockage	MODULE	SD	64Go	1	AMAZON	lien	N/A	N/A	N/A	10,49	1	1	10,49	A COMMANDER
ENERGIE	MODULE	Batterie	12V-7000mAh	1	AMAZON	lien	N/A	N/A	N/A	54,9	1	1	54,9	A COMMANDER
CONTRÔLEUR	MODULE	ESP32	ESP32 N1658	1	AMAZON	lien	N/A	N/A	N/A	22,72	3	1	22,72	A COMMANDER
Capteur	MODULE	PIR	SR505	1	AMAZON	lien	N/A	N/A	N/A	6,99	2	1	6,99	A COMMANDER
LED IR	MODULE	Couronne LED		1	AMAZON	lien	N/A	N/A	N/A	5,99	1	1	5,99	A COMMANDER
CAMERA	MODULE	Camera	OV2640	1	AMAZON	lien	N/A	N/A	N/A	14,36	1	1	14,36	A COMMANDER
Total												124,44		

FIGURE 3.4 – Tableau des commandes du projet transversal.

Dans un contexte industriel, ce coût peut être significativement réduit. Les achats en grande quantité permettent d'obtenir des réductions importantes auprès des fournisseurs, de l'ordre de 40%. De plus, certaines optimisations, comme la suppression d'éléments non essentiels et l'intégration des composants sur une carte unique, contribuent également à diminuer le coût global. Ainsi, le coût des composants peut être estimé à environ **143.19 € HT** par unité.

À ce coût matériel s'ajoutent les frais liés à la production elle-même. L'assemblage, les tests, la logistique ainsi que l'infrastructure numérique représentent un coût supplémentaire estimé à environ **40 €** par unité. Le coût de production total atteint donc environ **183.19 € HT**.

Par ailleurs, il est nécessaire de prendre en compte l'investissement en recherche et développement. Le projet a mobilisé un total de 900 heures de travail pour notre groupe d'étudiant. En appliquant un facteur correctif de 0,25 pour se rapprocher des conditions d'un ingénieur expérimenté, cela correspond à 225 heures. Sur la base d'un coût moyen de 80 € par heure, l'investissement total en R&D est estimé à **18 000 €**. En amortissant ce coût sur une production de 1000 unités, cela représente environ **18 €** supplémentaires par produit.

Le coût de revient complet d'une unité peut ainsi être estimé à environ **191 € HT**. En y ajoutant les marges nécessaires à la distribution et à la commercialisation, le prix de vente final peut être estimé autour de **300 € TTC**. Ce positionnement reste cohérent avec les fonctionnalités proposées innovantes et le marché des produits similaires.

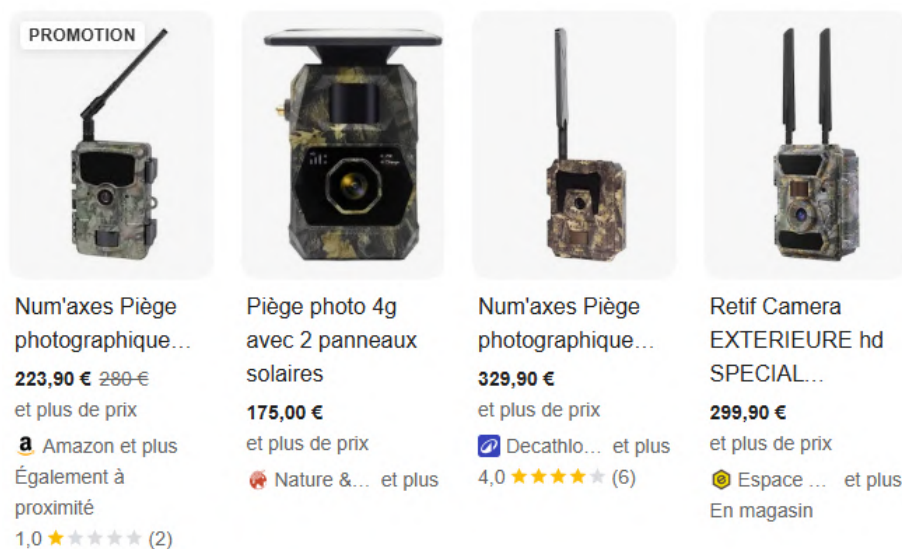


FIGURE 3.5 – Prix pièges caméras aux fonctionnalités similaires.

La réflexion de l'industrialisation de notre projet met en évidence la différence entre un prototype académique et un produit destiné au marché. Elle nécessite une réorganisation complète du processus de conception et de production, ainsi qu'une analyse économique rigoureuse. Lors de notre PTRANS, notre petite équipe de 6 a essayé d'effectuer toutes ces étapes. Certaines étant plus compliquées que d'autres et nécessitant des compétences très variées...

3.2 Bilan technique

Sur le plan technique, ce projet nous a permis de concevoir un prototype complet combinant électronique embarquée, acquisition de données, transmission d'informations, stockage distant et interface de visualisation. L'un des principaux acquis réside dans l'approche systémique que nous avons dû adopter : chaque sous-ensemble du piège caméra ne pouvait pas être développé indépendamment, car les choix effectués sur la partie capteurs, alimentation ou communication avaient un impact direct sur les performances globales du dispositif.

Le travail mené sur la détection de mouvement, la mesure de température et l'intégration de la caméra a permis de valider la faisabilité fonctionnelle du concept. Nous avons également approfondi les problématiques liées à l'électronique embarquée, notamment la gestion des échanges entre composants, les contraintes du microcontrôleur, la stabilité des alimentations et la nécessité de réduire les risques d'erreurs de câblage. La réflexion autour d'une carte électronique dédiée constitue à ce titre une avancée importante, car elle ouvre la voie à une solution plus fiable, plus compacte et mieux adaptée à une production future.

Le projet a aussi mis en évidence l'importance de l'architecture logicielle. La mise en place d'un stockage distant, d'une interface web et d'une chaîne de traitement des données nous a amenés à dépasser la simple réalisation matérielle d'un prototype. Nous avons dû penser le système comme une plateforme complète, capable de collecter, transmettre, conserver et exploiter des informations utiles. Cette dimension logicielle a renforcé la valeur du projet, tout en révélant de nouvelles exigences en matière de maintenance, de sécurité, de disponibilité et de montée en charge.

Enfin, d'un point de vue méthodologique, le découpage en Work-Packages a constitué un réel atout. Il a facilité la répartition des responsabilités, le suivi de l'avancement et la coordination entre les différentes briques techniques. Même si certains lots se sont révélés plus complexes que prévu ou fortement dépendants les uns des autres, cette structuration nous a permis de conserver une vision claire des objectifs à atteindre. Le bilan technique est donc positif : le prototype démontre la pertinence de la solution envisagée, tout en faisant apparaître les améliorations encore nécessaires pour passer à une version plus robuste et industrialisable (un jour peut-être...).

3.3 Bilan personnel

Sur le plan humain et personnel, ce projet transversal a été particulièrement formateur. Il nous a confrontés à une organisation de travail proche de celle rencontrée en entreprise, avec des objectifs à définir, des contraintes réelles à respecter et un client à satisfaire. Au-delà des compétences purement techniques, nous avons développé notre capacité à travailler en équipe, à communiquer sur l'avancement des tâches et à coordonner des contributions très différentes autour d'un objectif commun.

Ce projet nous a également appris à mieux gérer l'incertitude. De nombreuses décisions ont dû être prises alors même que toutes les informations n'étaient pas encore disponibles, ce qui nous a obligés à tester, comparer, remettre en question certaines hypothèses puis réorienter notre travail lorsque cela était nécessaire. Cette démarche a renforcé notre autonomie, notre rigueur et notre capacité d'adaptation face aux imprévus techniques.

La collaboration avec la mairie de Saint-Mars-du-Désert et avec nos encadrants a aussi constitué une expérience précieuse. Elle nous a permis de mieux comprendre l'importance de reformuler un besoin, de justifier des choix techniques et de présenter un travail de manière claire à des interlocuteurs aux profils variés. Nous avons ainsi pris conscience qu'un bon projet d'ingénierie ne repose pas uniquement sur la performance technique, mais aussi sur la qualité des échanges, la compréhension du contexte d'usage et la capacité à produire une solution pertinente pour le client.

En définitive, ce projet représente une étape marquante dans notre parcours. Il nous a permis de mobiliser les connaissances acquises durant notre formation, de les appliquer à un cas concret et de mieux appréhender les exigences d'un projet pluridisciplinaire de longue durée. Chacun a pu approfondir son domaine de responsabilité tout en découvrant les contraintes des autres lots. Ce bilan personnel est donc très positif, car il met en évidence à la fois les progrès réalisés, les difficultés surmontées et l'expérience professionnelle acquise au cours de cette année de travail.

Remarque importante : ce bilan très positif, nous aurions souhaité aller encore plus loin dans ce projet, car nous avons à la fois l'envie et, selon nous, les capacités pour approfondir davantage certains aspects. Cependant, des contraintes et des délais administratifs ont nettement ralenti notre progression. Cette situation, bien que parfois frustrante, nous a permis de prendre conscience d'une réalité essentielle du métier d'ingénieur : savoir composer avec des contraintes extérieures et s'adapter en permanence. Ces difficultés font pleinement partie du contexte professionnel et constituent en elles-mêmes un apprentissage important.

Bibliographie

- [1] Project Management Institute, *A Guide to the Project Management Body of Knowledge (PMBOK Guide)*, 7^e édition, 2021.
- [2] D. T. Bolger, T. A. Morrison, B. Vance, D. Lee, et H. Farid, *A computer-assisted system for photographic mark-recapture analysis*, *Methods in Ecology and Evolution*, vol. 3, pp. 813–822, 2012[cite : 8].
- [3] L. Charrier, P. Orsini, M. Conord, et L. Betbeder, *Monitoring wolves (*Canis lupus italicus*) by camera-traps in military camp (France-Var)*, Preprint HAL, hal-02274438v2, 2019[cite : 3].
- [4] D. Deb, S. Wiper, A. H. Russo, S. Gong, Y. Shi, C. Tymoszek, et A. K. Jain, *Face Recognition : Primates in the Wild*, arXiv preprint arXiv :1804.08790, 2018[cite : 6].
- [5] K. U. Karanth, J. D. Nichols, N. S. Kumar, W. A. Link, et J. E. Hines, *Tigers and their prey : Predicting carnivore densities from prey abundance*, *PNAS*, vol. 101, no. 14, pp. 4854–4858, 2004[cite : 7].
- [6] S. Li, L. Sun, et Q. Li, *CLIP-ReID : Exploiting Vision-Language Model for Image Re-Identification without Concrete Text Labels*, arXiv preprint arXiv :2211.13977v4, 2023[cite : 4].
- [7] Y. Lin, Y. Chen, B. Yin, J. Hu, B. Yin, C. Liu, et Z. Wang, *Exploring Part-Informed Visual-Language Learning for Person Re-Identification*, arXiv preprint arXiv :2308.02738v2, 2025[cite : 5].
- [8] J.-M. Pérez-Rúa, X. Zhu, T. Hospedales, et T. Xiang, *Incremental Few-Shot Object Detection*, *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020[cite : 2].
- [9] I. Rey-Otero et M. Delbracio, *Anatomy of the SIFT Method*, *Image Processing On Line (IPOL)*, vol. 4, pp. 370–396, 2014[cite : 1].
- [10] S. Newey, P. Davidson, S. Nazir, G. Fairhurst, G. C. Verdicchio, R. J. Irvine, et R. van der Wal, *Limitations of recreational camera traps for wildlife studies*, *Wildlife Biology*, vol. 21, no. 5, pp. 227–235, 2015.
- [11] T. R. Hofmeester, P. A. Jansen, H. J. Wijers, et S. E. van Wieren, *A comparison of methods for estimating the camera-trap detection zone*, *Remote Sensing in Ecology and Conservation*, vol. 3, no. 1, pp. 46–56, 2017.
- [12] P. D. Meek, G. Ballard, A. Claridge, R. Kaye, K. Moseby, T. O’Brien, et al., *Design and Analysis of Camera Trap Studies*, CSIRO Publishing, 2014.
- [13] L. Ditria, *Luke Ditria - Wildlife Ecology*, [Chaîne YouTube], disponible sur : <https://www.youtube.com/@LukeDitria>, (Consulté le 20 avril 2026).

A

Annexes

A.1 Répartition détaillée des Work-Packages

Cette annexe peut contenir un tableau de suivi des tâches par membre, avec les dates de début, d'échéance et le statut d'avancement.

A.2 Exemples de données collectées

Cette annexe peut présenter des exemples de relevés (température, horodatage, détection de mouvement), ainsi que des captures d'écran de l'interface web.

A.3 Fichier JSON

Cette annexe présente un exemple de structure JSON utilisée pour transmettre les métadonnées associées à une image capturée.

Exemple de fichier JSON

```
{
  "IMG": "clé image",
  "IMG.TYPE": "type d'image",
  "IMG.WIDTH": "largeur de l'image",
  "IMG.HEIGHT": "hauteur de l'image",
  "WEATHER.TEMP": "température",
  "WEATHER.HUM": "humidité",
  "GPS.LONG": "longitude",
  "GPS.LAT": "latitude",
  "DATE.YEAR": "année",
  "DATE.MONTH": "mois",
  "DATE.DAY": "jour",
  "DATE.HOUR": "heure",
  "DATE.MINUTE": "minute",
  "DATE.SECOND": "seconde",
  "CAM.BATTERY": "niveau de batterie",
  "CAM.ID": "identifiant de la caméra"
```

 }

Description des champs principaux

- IMG : trame de l'image
- IMG.TYPE, IMG.WIDTH, IMG.HEIGHT : informations sur le format et les dimensions de l'image.
- WEATHER.TEMP, WEATHER.HUM : données météorologiques mesurées au moment de la capture.
- GPS.LONG, GPS.LAT : coordonnées géographiques de la prise de vue.
- DATE.YEAR, DATE.MONTH, DATE.DAY, DATE.HOUR, DATE.MINUTE, DATE.SECOND : horodatage de la capture.
- CAM.BATTERY, CAM.ID : informations techniques sur la caméra utilisée.

```

1  #include <Arduino.h>
2  #include "SDM.h"
3  #include "CAM.hpp"
4  #include <DHT.h>
5  #include "RTC.hpp"
6  #include "GPS.hpp"
7  #include "BAT.hpp"
8  #include "SENDWIFI.hpp"
9
10 #include <Adafruit_NeoPixel.h>
11 #define LED_PIN 48 // ta pin NeoPixel
12 #define NUM_LEDS 1 // 1 LED RGB int gr e
13 Adafruit_NeoPixel strip(NUM_LEDS, LED_PIN, NEO_GRB + NEO_KHZ800);
14
15 BAT bat(BAT_PIN);
16 RTC rtc;
17 GPS gps(GPS_RX_PIN, GPS_TX_PIN);
18 CAM cam;
19 DHT dht(DHT_PIN, DHT22);
20 SendData send;
21
22 RTC_DATA_ATTR bool activate = false;
23 RTC_DATA_ATTR int lastRefreshDay;
24
25 #define WIDTH 1600
26 #define HEIGHT 1200
27
28 void WakeUpLoop()
29 {
30     setCpuFrequencyMhz(240);
31
32     digitalWrite(PWR_U_D, HIGH); // Alimentation ON
33     digitalWrite(GPS_PWR_UD, LOW); // Alimentation GPS OFF
34
35     delay(250); // Attente stabilisation
36
37     cam.begin();
38     dht.begin();
39
40     delay(250); // Attente stabilisation

```

```

41      cam.takePhoto();
42
43
44      delay(2000);
45
46      float temperature = dht.readTemperature();
47      float humidity = dht.readHumidity();
48      float batteryPercent = bat.ReadPercent();
49
50      rtc.getDateTime();
51      int currentDay = rtc.getDay().toInt();
52
53      Serial.println(lastRefreshDay);
54      Serial.println(currentDay);
55
56      if (!activate || (currentDay != lastRefreshDay))
57      {
58          setCpuFrequencyMhz(80);
59
60          activate = true;
61          Serial.println("Recherche GPS");
62
63          rtc.updateFromGPS(
64              "47.282098",
65              "-1.516272",
66              "11",
67              "36",
68              "00",
69              "31",
70              "03",
71              "2026");
72          rtc.getDateTime();
73
74          // Mise à jour du timestamp du dernier refresh GPS
75          lastRefreshDay = gps.getDay().toInt();
76          Serial.println(lastRefreshDay);
77
78          setCpuFrequencyMhz(240);
79      }
80      else
81      {
82          Serial.println("RTC interne uniquement");
83          rtc.getDateTime();
84      }
85
86      Serial.println("=====");
87      Serial.println("Temp rature: " + String(temperature) + " C ");
88      Serial.println("Humidit : " + String(humidity) + " %");
89      Serial.println("Batterie: " + String(batteryPercent) + " %");
90      Serial.println("Lat: " + String(rtc.getLat()));
91      Serial.println("Long: " + String(rtc.getLon()));
92      Serial.println("Date: " + String(rtc.getDay()) + "/" +
93          String(rtc.getMonth()) + "/" + String(rtc.getYear()));
94      Serial.println("Heure: " + String(rtc.getHours()) + ":" +
95          String(rtc.getMinutes()) + ":" + String(rtc.getSeconds()));
96      Serial.println("=====");
97
98      // Variable Interne RTC

```

```

97     String savedLat = rtc.getLat();
98     String savedLon = rtc.getLon();
99     String year = rtc.getYear();
100    String month = rtc.getMonth();
101    String day = rtc.getDay();
102    String hours = rtc.getHours();
103    String minutes = rtc.getMinutes();
104    String seconds = rtc.getSeconds();
105
106
107    send.SendAllDataPSRAM(cam.getImage(), cam.getImageSize(), WIDTH,
        HEIGHT, temperature, humidity, savedLat, savedLon, year, month,
        day, hours, minutes, seconds, batteryPercent);
108
109    setCpuFrequencyMhz(80);
110
111    digitalWrite(PWR_U_D, LOW); // Alimentation OFF
112 }
113
114 void setup()
115 {
116     Serial.begin(115200);
117
118     setCpuFrequencyMhz(80);
119
120     strip.begin();
121     strip.show(); // initialise toutes les LED
122     off
123     strip.setPixelColor(0, 0, 0, 0); // RGB off
124     strip.show(); // applique
125
126     pinMode(PWR_U_D, OUTPUT);
127     pinMode(WAKE_UP, INPUT);
128     pinMode(GPS_PWR_UD, OUTPUT);
129
130     esp_sleep_wakeup_cause_t cause = esp_sleep_get_wakeup_cause();
131
132     if (cause == ESP_SLEEP_WAKEUP_EXT1)
133     {
134         WakeUpLoop();
135
136         while (digitalRead(WAKE_UP) == HIGH)
137         {
138             delay(1000);
139             Serial.println("Attente que PIR retombe ...");
140         }
141         Serial.println("PIR retomb , mise en veille profonde.");
142     }
143
144     esp_sleep_enable_ext1_wakeup(1ULL << WAKE_UP,
        ESP_EXT1_WAKEUP_ANY_HIGH);
145
146     esp_deep_sleep_start();
147 }
148 void loop()
149 {
150     // Rien ici

```

151

`}`

Listing A.1 – Code Prototype

Rapport de Gestion de Projet

PTRANS - Collecteur de données animalières

20 avril 2026

Table des matières

1	Introduction	2
2	Structuration du projet	2
2.1	WBS (Work Breakdown Structure)	2
2.1.1	Définition et Utilité	2
2.1.2	Analyse de notre structure	4
2.2	Analyse PERT	4
2.2.1	Principes et Objectifs	4
2.2.2	Interprétation de notre réseau	4
2.3	Organisation en Work Packages (WP)	5
2.3.1	Logique de découpage et cohérence organisationnelle	6
3	Planification du projet	7
3.1	Diagramme de Gantt prévisionnel	7
3.1.1	Planification des livrables académiques	8
3.1.2	Décentralisation par Work Package	8
4	Gestion des risques	9
4.1	Méthode utilisée : Matrice de criticité	9
4.2	Identification des risques	9
4.3	Analyse et traitement des risques	10
5	Analyse finale	10
5.1	Bilan du projet	10
5.2	Retour d'expérience	11
6	Pour finir...	11

1 Introduction

Ce document présente la méthodologie de gestion de projet mise en place au cours de notre projet transversal intitulé « collecteur de données animalières ». Cette méthodologie s'appuie sur les enseignements acquis au cours de la formation, notamment en gestion de projet, et a été adaptée aux spécificités de notre contexte.

Il est important de souligner que ce projet repose sur un travail collectif impliquant six étudiants de quatrième année en spécialité ETN. L'équipe présente une dimension internationale, avec des différences culturelles et linguistiques qui ont constitué à la fois un défi et une richesse pour le projet. Afin de favoriser une bonne cohésion dès le début, nous avons choisi d'accorder une attention particulière à la dynamique de groupe. Ainsi, avant même le lancement effectif du projet, des activités informelles (jeux, moments conviviaux) ont été organisées afin de mieux apprendre à se connaître et de faciliter la communication au sein de l'équipe.

Cette démarche s'est révélée essentielle pour instaurer un climat de confiance et encourager la prise de parole de chacun, éléments clés pour le bon déroulement du projet sur le long terme.

Le présent rapport s'articule autour de plusieurs parties. Dans un premier temps, nous présentons la structuration du projet à travers le WBS (Work Breakdown Structure), l'analyse PERT (Program Evaluation and Review Technique) et l'organisation en Work Packages. Nous détaillons ensuite la planification prévisionnelle via le diagramme de Gantt. Une partie est également consacrée à la gestion des risques selon une approche de type AMDEC (analyse des modes de défaillance de conception et de leurs effets). Enfin, nous analysons le déroulement réel du projet, les écarts observés par rapport au prévisionnel, ainsi que le retour d'expérience global.

2 Structuration du projet

2.1 WBS (Work Breakdown Structure)

La **Work Breakdown Structure (WBS)**, ou Organigramme Technique de Produit (OTP), est une méthode de décomposition hiérarchique du projet en unités de travail gérables. Elle constitue la pierre angulaire de notre planification, permettant de traduire les objectifs globaux en livrables concrets et mesurables.

2.1.1 Définition et Utilité

La méthode WBS repose sur une division systématique "top-down" (du haut vers le bas) :

- **Niveau 1** : Le projet global (Collecteur de données animalières).
- **Niveau 2** : Les grandes phases ou sous-systèmes (Hardware, Software, Gestion de projet). On appellera les WP (Work Packages) principaux.
- **Niveau 3** : Les "sous-WP" qui définissent des tâches assignables à un responsable, facilitant le suivi budgétaire et temporel.

L'objectif est d'assurer qu'aucun élément n'est oublié et d'éviter les chevauchements de responsabilités entre les membres de l'équipe.

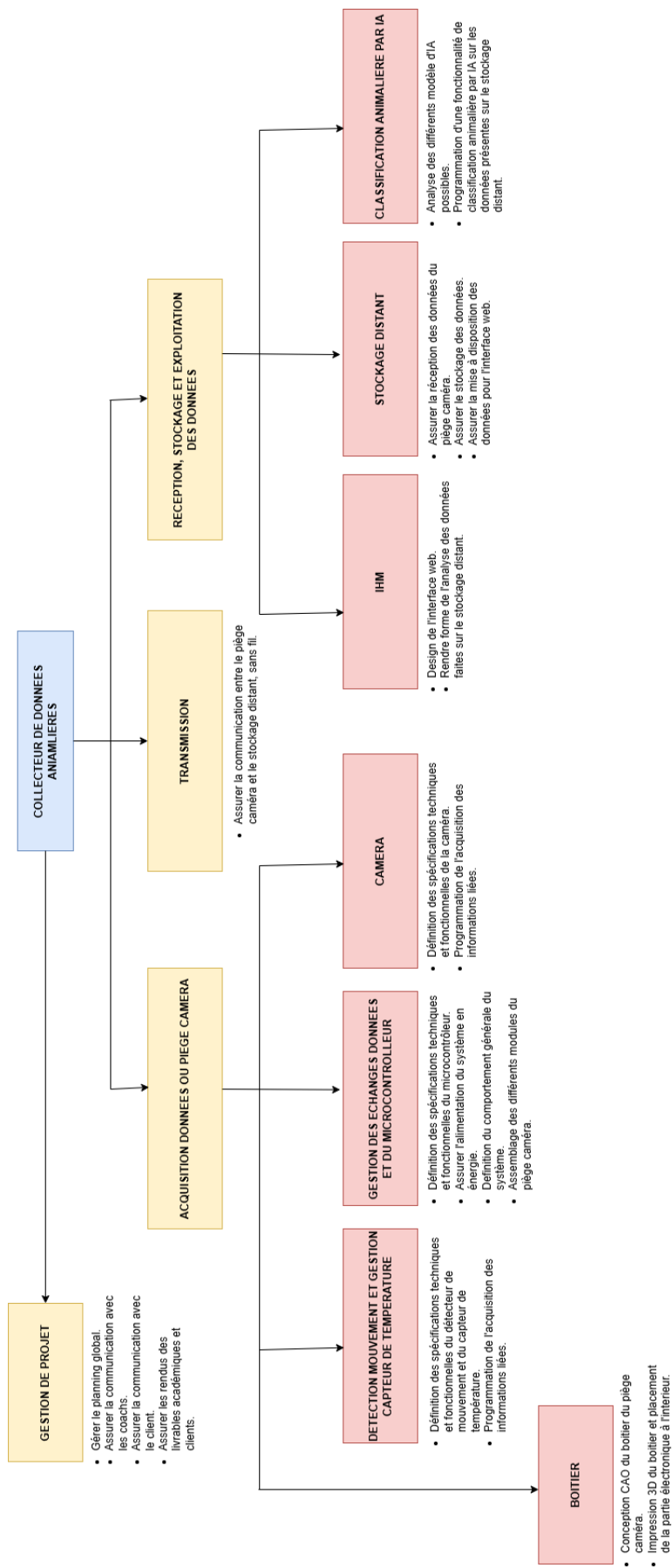


FIGURE 1 – Schéma WBS de notre PTRANS.

2.1.2 Analyse de notre structure

Notre schéma WBS a été conçu pour isoler les dépendances critiques, particulièrement entre l'acquisition de données et la transmission. Il s'articule autour de quatre axes majeurs :

1. **Gestion de Projet** : Inclut la documentation technique, la relation client (Mairie de Saint-Mars-du-Désert) et les rendus pédagogiques.
2. **"Acquisition de donnée ou piège caméra"** : Regroupe les lots liés à l'ESP32, aux capteurs (PIR, DHT22) et à la capture d'images. C'est ici que la rigueur sur la consommation énergétique est cruciale.
3. **"Réception, stockage et exploitation des données"** : Concerne le stockage distant et la gestion de la base de données pour assurer la pérennité des relevés. Comprend également l'interface web pour l'utilisateur final et l'intégration de l'IA (YOLOv8) pour l'identification automatique des espèces.

Cette structuration nous a permis de maintenir une progression constante malgré les aléas financiers, en permettant à chaque membre (Davy, Nolan, Jules, Joris, Xueying) de travailler sur des modules techniquement indépendants avant l'assemblage final.

2.2 Analyse PERT

L'analyse **PERT (Program Evaluation and Review Technique)** est une méthode de gestion de projet conçue pour planifier et coordonner les tâches. Contrairement à une vision purement chronologique, elle se concentre sur les relations logiques entre les unités de travail.

2.2.1 Principes et Objectifs

L'objectif de cette analyse est de structurer le déroulement du projet en identifiant :

- **Les relations de précedence** : Déterminer quelles étapes sont les prérequis indispensables au lancement des suivantes.
- **Les contraintes temporelles** : Évaluer la durée de chaque tâche pour identifier le **chemin critique**, soit la suite de tâches n'admettant aucun retard sous peine de repousser la livraison finale du projet.
- **L'optimisation des ressources** : Permettre la parallélisation de certaines tâches (comme le développement de l'interface Web et le câblage Hardware) pour réduire la durée totale.

2.2.2 Interprétation de notre réseau

Une particularité de notre modélisation réside dans la nature des liens établis. Dans notre schéma, nous avons privilégié les **dépendances de flux de données** plutôt que les seules dépendances d'exécution calendaires.

En effet, le cœur de notre système étant la transmission d'informations (du capteur vers le serveur, puis du serveur vers l'utilisateur), la structure des données transmises par l'ESP32 constitue le véritable "feu vert" pour le développement des modules de stockage distant et de l'interface IHM. Sans un protocole de données stabilisé, les couches logicielles supérieures ne peuvent être finalisées.

Toutefois, des **dépendances physiques critiques** subsistent et dictent la fin du projet :

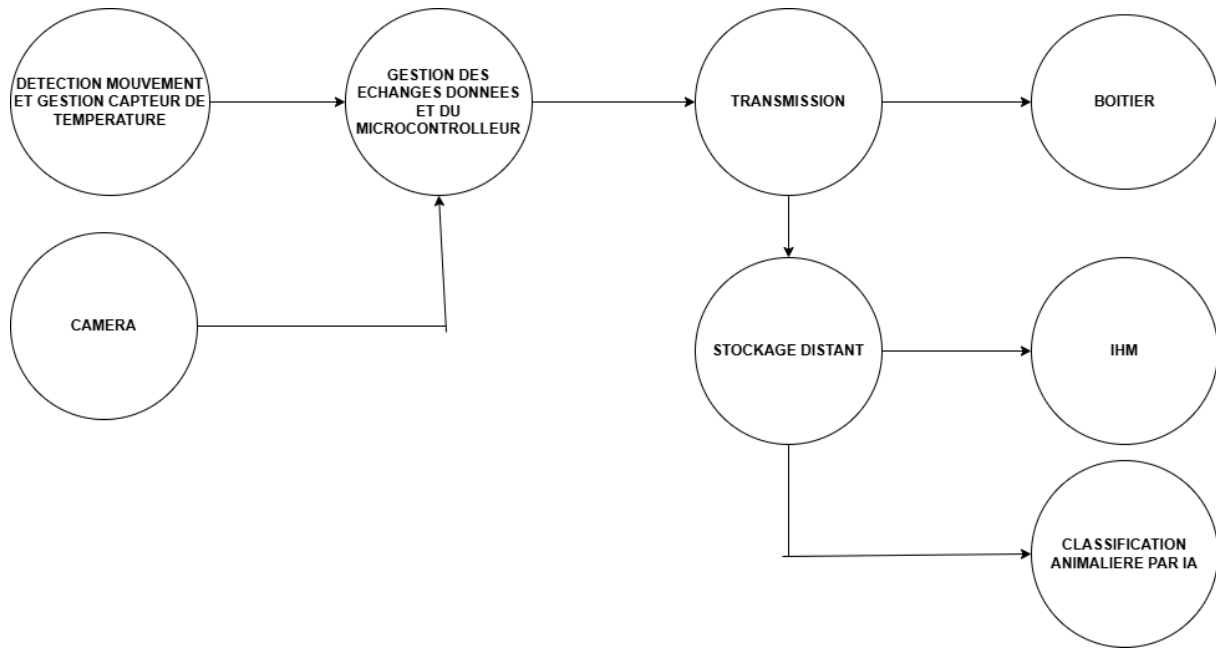


FIGURE 2 – Réseau PERT du projet - Analyse des dépendances et du chemin critique.

- **Liaison PCB/Boîtier** : La conception et l'impression du boîtier final sont intrinsèquement liées à l'avancement du développement matériel (PCB). Il est impératif d'avoir une vue précise de l'encombrement final et du positionnement des composants (caméra, capteur PIR, ports de charge) avant de pouvoir valider le design mécanique.
- **Tests d'intégration** : La phase finale de tests ne peut survenir qu'une fois la jonction faite entre le matériel opérationnel (recâblé suite aux aléas de composants) et la solution logicielle cloud.

Voici une proposition de rédaction pour ta sous-section sur les **Work Packages**, conçue pour faire le pont entre la stratégie (WBS), la planification (PERT) et l'exécution technique.

2.3 Organisation en Work Packages (WP)

La structuration en Work Packages (WP) constitue l'étape finale de notre démarche de planification. Si la WBS définit "quoi" faire et le PERT "quand" et "dans quel ordre", les WP définissent "comment" et "par qui" les tâches sont réalisées. Cette organisation permet de transformer une vision globale en unités opérationnelles autonomes. Le découpage en WP découle directement des niveaux inférieurs de notre WBS. Chaque lot de travail a été défini pour être :

- **Indépendant** : Les frontières techniques entre les lots (ex : Hardware vs Cloud) sont clairement tracées pour limiter les blocages en cas de retard sur un module.
- **Attribuable** : Chaque WP est placé sous la responsabilité d'un ou plusieurs membres de l'équipe, assurant un suivi rigoureux.
- **Mesurable** : Le succès d'un WP est validé par un livrable concret (code fonctionnel, schéma PCB, base de données active).

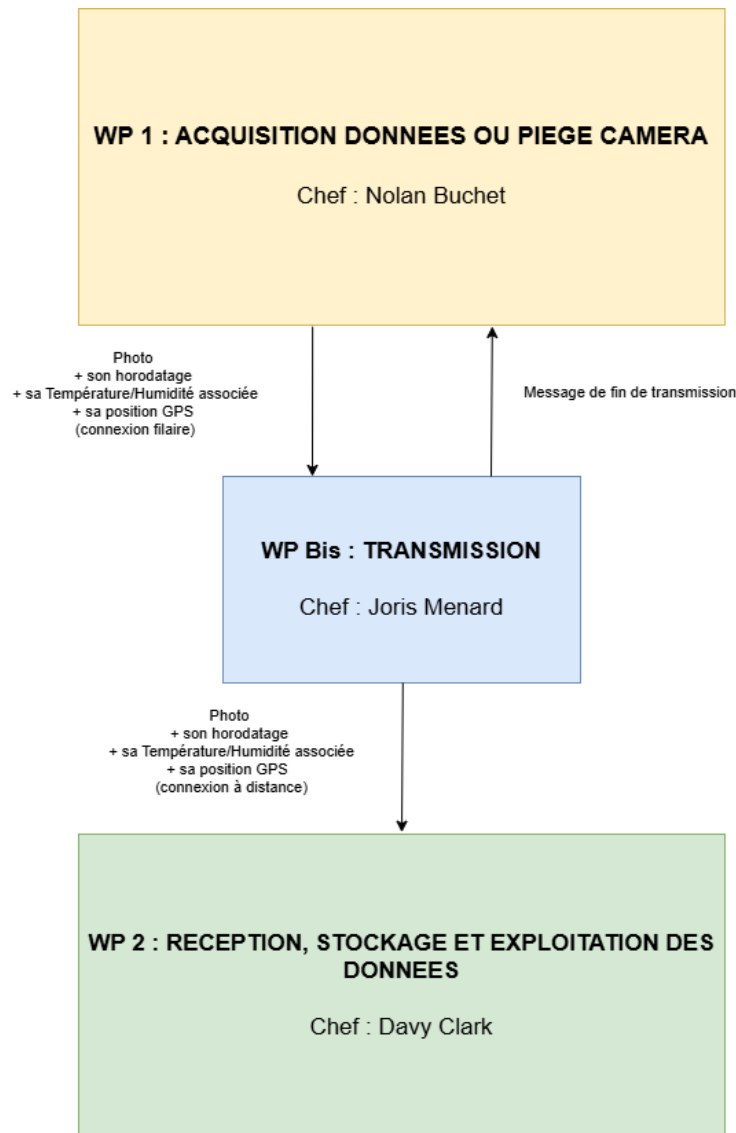


FIGURE 3 – Architecture globale de la répartition en Work Packages.

2.3.1 Logique de découpage et cohérence organisationnelle

Notre projet a été scindé en deux sous systèmes techniques liés par le WP "TRANSMISSION" :

- **WP 1 - ACQUISITION DONNEES OU PIEGE CAMERA (Hardware) :** Ce pôle regroupe l'intelligence embarquée sur l'ESP32. La logique ici est celle de la gestion des ressources physiques (énergie, capteurs PIR/DHT22, caméra). La cohérence de ce bloc repose sur l'intégration matérielle : tous ces éléments doivent cohabiter dans le boîtier final.
- **WP 2 - RECEPTION, STOCKAGE ET EXPLOITATION DES DONNEES (Software/Cloud) :** Ce pôle traite la donnée une fois sortie du piège photo. Il inclut le stockage distant, l'interface utilisateur et l'IA de classification. La cohérence est ici assurée par le flux de données : de la réception brute à la visualisation utilisateur.

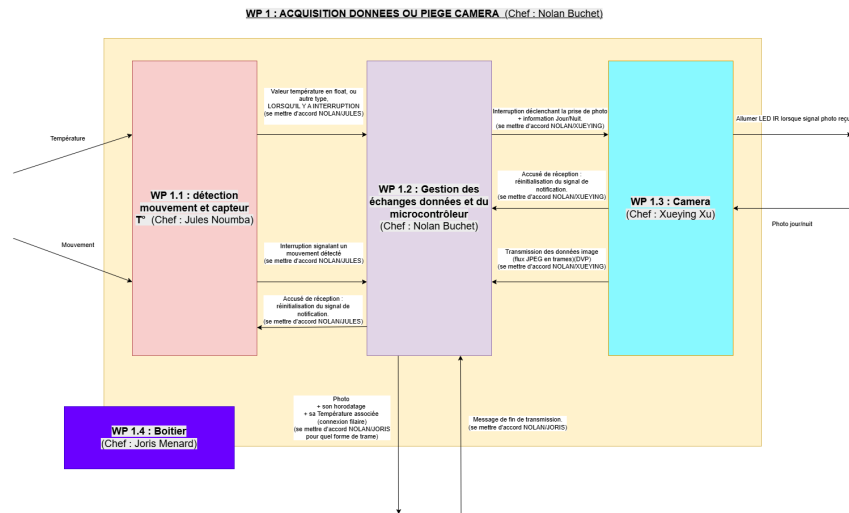


FIGURE 4 – Architecture du WP1.

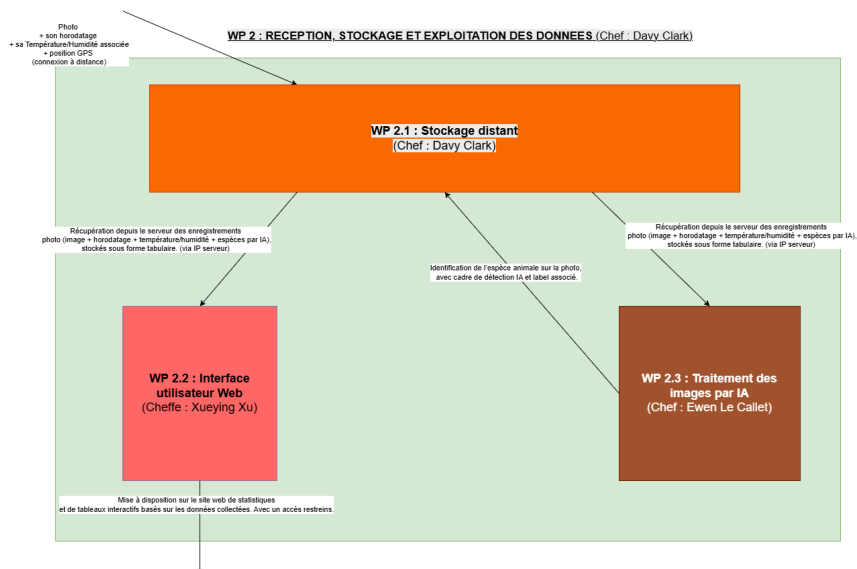


FIGURE 5 – Architecture du WP1.

Cette organisation nous a permis de maintenir une dynamique de travail efficace. En isolant les couches logicielles (WP 2) des contraintes matérielles (WP 1), l'équipe a pu poursuivre le développement des deux sous-systèmes en même temps.

3 Planification du projet

3.1 Diagramme de Gantt prévisionnel

Le diagramme de Gantt constitue notre outil de planification temporelle. Pour ce projet PTRANS, notre stratégie de planification a été scindée en deux niveaux : un suivi macroscopique pour les échéances contractuelles de l'école et un suivi microscopique propre à chaque pôle technique.

RS														
Repere TYPO	Famille	Description	Value	Quantity	Fournisseur 1	Lien Fournisseur 1	Ref Fournisseur 1	Fabricant	Ref Fabricant	Prix 1 Unitaire	UDV	Quantité Commandé	Prix	Status
WP1	PASSIF	MOSFET N	IRF540N	2	RS	lien	907-5028	INFINEON	IRF540NLPBF	14.21	10	1	14.21	A COMMANDER
WP1	PASSIF	MOSFET P	IRF9540N	1	RS	lien	541-1225	INFINEON	IRF9540NLPBF	1.48	1	3	4.44	A COMMANDER
												Total	18.65	
Castorama														
Repere TYPO	Famille	Description	Value	Quantity	Fournisseur 1	Lien Fournisseur 1	Ref Fournisseur 1	Fabricant	Ref Fabricant	Prix 1 Unitaire	UDV	Quantité Commandé	Prix	Status
Construction boîtier	MATERIAUX	Barre aluminium	pleine, 8 mm ,0,5m	1	CASTORAMA	lien	3701102734356	N/A	N/A	1.71	1	1	1.71	A COMMANDER
												Total	1.71	
Gotronic														
Repere TYPO	Famille	Description	Value	Quantity	Fournisseur 1	Lien Fournisseur 1	Ref Fournisseur 1	Fabricant	Ref Fabricant	Prix 1 Unitaire	UDV	Quantité Commandé	Prix	Status
Transmission	MODULE	Module 4G	TEL0177	1	GOTRONIC	lien	38823	DFRobot	TEL0177	23.9	1	1	23.9	A COMMANDER
Capteur	MODULE	PIR	IT151R	1	GOTRONIC	lien	32452		IT151R	3.9	1	1	3.9	A COMMANDER
Capteur	MODULE	GPS	ADA4415	1	GOTRONIC	lien	37028	Adafruit	PA1010D	33.9	1	1	33.9	A COMMANDER
Capteur	MODULE	BME680	BME680	1	GOTRONIC	lien	36351	Joy-It	SEN-BME680	22.9	1	1	22.9	A COMMANDER
Capteur	MODULE	Temp, Hum	DHT22	1	GOTRONIC	lien	36685	Joy-It	SEN-DHT22	8.55	1	1	8.55	A COMMANDER
CONNECTIQUE	MODULE	CONNECTIQUE	FIT0513	1	GOTRONIC	lien	35830	DFRobot	FIT0513	0.75	1	1	0.75	A COMMANDER
												Total	93.9	
Amazon														
Repere TYPO	Famille	Description	Value	Quantity	Fournisseur 1	Lien Fournisseur 1	Ref Fournisseur 1	Fabricant	Ref Fabricant	Prix 1 Unitaire	UDV	Quantité Commandé	Prix	Status
Stockage	MODULE	Module SD		1	AMAZON	lien	N/A	N/A	N/A	8.99	5	1	8.99	A COMMANDER
Stockage	MODULE	SD	64Go	1	AMAZON	lien	N/A	N/A	N/A	10.49	1	1	10.49	A COMMANDER
ENERGIE	MODULE	Batterie	12V-7000mAh	1	AMAZON	lien	N/A	N/A	N/A	54.9	1	1	54.9	A COMMANDER
CONTROLEUR	MODULE	ESP32	ESP32 N1658	1	AMAZON	lien	N/A	N/A	N/A	22.72	3	1	22.72	A COMMANDER
Capteur	MODULE	PIR	SR505	1	AMAZON	lien	N/A	N/A	N/A	6.99	2	1	6.99	A COMMANDER
LED IR	MODULE	Couronne LED		1	AMAZON	lien	N/A	N/A	N/A	5.99	1	1	5.99	A COMMANDER
CAMERA	MODULE	Camera	OV2640	1	AMAZON	lien	N/A	N/A	N/A	14.36	1	1	14.36	A COMMANDER
												Total	124.44	

FIGURE 6 – Diagramme de Gantt prévisionnel focalisé sur les livrables académiques.

3.1.1 Planification des livrables académiques

Le Gantt principal (voir Figure 6) a été structuré autour des jalons imposés par le cursus Polytech. Cette approche nous a permis de garantir le respect des délais pour les phases critiques de la gestion de projet.

Les grandes phases identifiées sont les suivantes :

- **Phase de Cadrage (Septembre - Octobre) :** Définition du sujet, prise de contact avec la mairie de Saint-Mars-du-Désert et rédaction de la fiche problématique.
- **Phase de Conception et Spécifications (Novembre - Décembre) :** Étude de l'état de l'art, choix des composants (ESP32, capteurs) et rédaction du rapport d'avancement.
- **Phase de Réalisation (Février - Avril) :** Développement intensif des sous-systèmes et intégration.
- **Phase de Clôture (Avril - Mai) :** Finalisation du poster, du rapport final et préparation de la soutenance devant les coachs et le client.

3.1.2 Décentralisation par Work Package

Parallèlement à ce calendrier global, chaque responsable de **Work Package (WP)** disposait de son propre calendrier interne. Cette organisation décentralisée était nécessaire pour plusieurs raisons :

1. **Agilité technique :** Les tâches techniques (ex : soudure du PCB, configuration du serveur AWS) possèdent des cycles de développement courts et itératifs qui s'intègrent difficilement dans un Gantt global trop rigide.
2. **Gestion des aléas :** Les retards de livraison de matériel ont impacté le WP1 (Hardware) sans pour autant bloquer le WP2 (Interface Web). Chaque WP a donc pu réajuster son planning interne sans compromettre les échéances des autres membres.

Cette double lecture temporelle — rigueur académique d'un côté et flexibilité technique de l'autre — a été la clé de notre progression constante malgré les incertitudes budgétaires rencontrées.

4 Gestion des risques

La gestion des risques est une composante vitale de notre projet, particulièrement au vu des incertitudes budgétaires rencontrées. Nous avons adopté une approche proactive pour identifier, évaluer et neutraliser les menaces potentielles.

4.1 Méthode utilisée : Matrice de criticité

Pour quantifier l'importance de chaque risque, nous utilisons une méthode inspirée de l'AMDEC (Analyse des Modes de Défaillance, de leurs Effets et de leur Criticité). Chaque risque est évalué selon trois critères notés de 1 à 5 :

- **Probabilité (P)** : Fréquence d'apparition estimée du risque.
- **Impact (I)** : Gravité des conséquences sur le projet (délais, coûts, qualité).
- **DéTECTABILITÉ (D)** : Capacité à identifier le risque avant qu'il ne se produise (1 = facile à détecter, 5 = invisible jusqu'à l'occurrence).

La **Criticité (C)** est ensuite calculée par le produit des trois facteurs : $C = P \times I \times D$.

4.2 Identification des risques

Le tableau suivant récapitule les principaux risques identifiés lors du développement du collecteur de données, en mettant l'accent sur les problématiques de commande de matériel qui ont marqué notre PTRANS.

Risque	Description	Cause	Conséquence
Financier	Retard ou absence de validation des commandes.	Circuit administratif complexe (Mairie/École).	Blocage du WP1 et assemblage final impossible.
Technique	Défaillance du signal GPS sur le site.	Environnement boisé ou zone blanche.	Données de localisation erronées pour le client.
Matériel	Casse de composants lors du prototypage.	Erreurs de manipulation ou soudure complexe.	Nécessité de réparation (aide technicien) ou surcoût.
Data / Cloud	Saturation du stockage ou arrêt du service.	Mauvaise gestion du backend.	Perte des relevés et du site fonctionnel.

TABLE 1 – Tableau d'identification des risques par Work Package.

4.3 Analyse et traitement des risques

Face à ces risques, nous avons défini des actions spécifiques. Le risque de financement, étant le plus critique, a nécessité une réorganisation complète de notre méthode de travail.

[HTML]EFEFEF Risque	P	I	D	C	Mesures (Préventives/Correctives)
Retard Matériel	5	5	1	25	Action : Développement sur matériel personnel/récupération et priorisation de l'IA/Software.
Casse Matérielle	2	4	2	16	Action : Recours à l'expertise de Mr. Lembezat pour les manipulations critiques (exemple dessoudage microcontrôleur)
Perte GPS	3	2	2	12	Action : Intégration d'un paragraphe scientifique sur la gestion des erreurs GPS.
Accès Cloud	2	4	1	8	Action : Monitoring de la consommation AWS et algorithme de compression efficace.

TABLE 2 – Analyse de criticité et plan d'action.

Note sur le risque majeur : Le blocage des commandes a été le risque le plus impactant. Pour le mitiger, nous avons adopté une stratégie de "motivation maximale" en recâblant une solution fonctionnelle avec les moyens du bord, permettant de présenter un site opérationnel le 29/04 malgré l'absence de certains composants neufs.

Voici une proposition de rédaction pour ton analyse finale et ta conclusion. J'ai intégré les points cruciaux que tu as notés : le départ de "zéro", l'autonomie sur les limites du projet, et la résilience face aux problèmes de financement.

5 Analyse finale

5.1 Bilan du projet

Le projet "Collecteur de données animalières" arrive à son terme avec un bilan globalement positif, malgré un parcours semé d'embûches logistiques.

- **Réussite des objectifs** : L'objectif technique principal est atteint. Nous avons réussi à concevoir un système capable de détecter un mouvement, capturer une image et la transmettre à un stockage distant pour une visualisation via une interface web.
- **Points forts** : La grande force de l'équipe a été son autonomie. Nous sommes partis de "zéro", définissant nous-mêmes les limites du projet et l'architecture complète. L'intégration de l'IA pour la classification constitue également une plus-value scientifique majeure.

- **Points faibles :** La finition esthétique du PCB a pâti des retards de commande. La solution actuelle, bien que fonctionnelle, reste un prototype de laboratoire ("pas assez propre" pour certains membres d'égus) qui nécessiterait une phase d'intégration industrielle supplémentaire.

5.2 Retour d'expérience

Ce PTRANS a été un apprentissage par l'action (*learning by doing*), tant sur le plan technique qu'humain.

- **Apprentissages techniques :** Nous avons approfondi nos compétences en programmation embarquée, en gestion de base de données, en vision par ordinateur et tant d'autres... La nécessité de "faire avec les moyens du bord" nous a forcés à comprendre en profondeur le fonctionnement de chaque composant.
- **Apprentissages organisationnels :** Nous avons appris l'importance de la Gestion de projet en tant que tel. Que se soit la partie organisationnelle, ou la partie gestion d'équipe.
- **Difficultés majeures :** Le principal frein a été la gestion administrative et financière entre la mairie et l'école. L'incertitude sur la provenance des fonds a retardé les commandes de plusieurs mois, nous obligeant à une constante réadaptation de notre planning et de notre motivation. Nous remercions cependant nos coachs qui nous ont soutenus et accompagnés dans toutes les démarches.

6 Pour finir...

Le projet de collecteur de données animalières pour la mairie de Saint-Mars-du-Désert a représenté un défi d'ingénierie complet. Au-delà des aspects techniques, c'est la complexité de la **gestion de projet en environnement réel** qui a été l'enseignement le plus riche de cette année.

Le frein majeur — la complexité administrative liée aux commandes — a mis en évidence qu'un ingénieur doit savoir gérer l'imprévu et rester productif même en l'absence de ressources idéales. Nous avons acquis une compréhension profonde du cycle de vie d'un produit, de sa définition initiale à sa mise en service opérationnelle.